Neural Network Verification

A Guide for Researchers and Practitioners

ThanhVu (Vu) Nguyen and **Hai Duong**

George Mason University

Contents and Summary

Preface

1 Neural Networks

1.1 Basics of Neural Networks

A *neural network* (**NN**) [1] consists of an input layer, multiple hidden layers, and an output layer. Each layer has a number of neurons, each connected to neurons in the next layer through a predefined set of weights (computed during training). A *Deep Neural Network* (**DNN**) is an NN with *two* or more hidden layers.

The output of an NN is obtained by iteratively computing the values of neurons in each layer. The value of a neuron in the input layer is the input data. The value of a neuron in the hidden layers is computed by applying an *affine transformation* (Sec. 1.3) to values of neurons in the previous layers, then followed by an *activation function* (Sec. 1.4) such as ReLU and Sigmoid. The value of a neuron in the output layer is computed similarly but may skip the activation function.

1.2 NN as a Function

We can view an NN as a function that maps input vectors to output vectors:

$$f : \mathbb{R}^n \to \mathbb{R}^m \tag{1}$$

where n is the number of input neurons and m is the number of output neurons. The neurons in the input layer are the inputs to the function, and the neurons in the output layer are the outputs of the function. The neurons in the hidden layers are also functions that transform the inputs from the previous layer to produce outputs for the next layer.

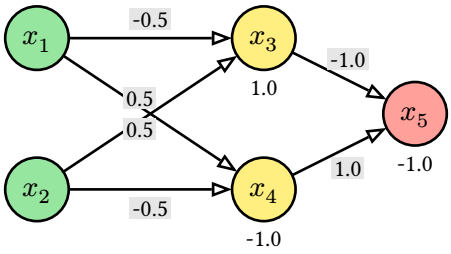


Fig. 1: A simple network with two inputs x_1, x_2, two hidden neurons x_3, x_4, and one output neuron x_5.

Example 1.2.1

Fig. 1 shows an NN with two inputs $x_1, x_2 \in \mathbb{R}$, two hidden neurons x_3, x_4 in one hidden layer, and one output neuron x_5. The connections between the neurons are weighted edges, and the biases are shown below each neuron.

- The hidden neurons compute:

$$x_3 = \text{relu}(-0.5x_1 + 0.5x_2 + 1.0), x_4 = \text{relu}(0.5x_1 - 0.5x_2 - 1.0), \tag{2}$$

  where $\text{relu}(x) = \max(x, 0)$ is the ReLU activation function.
- The output neuron computes:

$$x_5 = -1.0 \cdot x_3 + 1.0 \cdot x_4 - 1.0. \tag{3}$$

Thus, this NN computes a function $f : \mathbb{R}^2 \to \mathbb{R}$ where:

$$f(x_1, x_2) = -\text{ relu}(-0.5x_1 + 0.5x_2 + 1.0) + \text{relu}(0.5x_1 - 0.5x_2 - 1.0) - 1.0. \tag{4}$$

◊

1.3 Affine Transformation

The affine transformation (AF) of a neuron consists of a *linear combination*—a weighted sum $\sum$—of its inputs, followed by the addition of a bias term. More specifically, for a neuron with weights $w_1, ..., w_n$, bias b, and inputs $v_1, ..., v_n$ from the previous layer, the AF computes:

$$f(v_1, v_2, ..., v_n) = \sum_{i=1}^{n} w_i v_i + b. \tag{5}$$

Example 1.3.1

In Fig. 1, neuron x_3 receives inputs x_1 and x_2 with weights -0.5, 0.5, and bias 1.0, so its AF is $x_3 = -0.5x_1 + 0.5x_2 + 1.0$.

◊

1.4 Activation Functions

Activation functions in NNs are mathematical functions applied to the output of a neuron after the affine transformation. They introduce non-linearity to the network, allowing it to learn complex patterns in the data. Without activation functions, an NN would simply be a linear model, regardless of the number of layers, and would not be able to capture intricate relationships in the data.

Tab. 1 summarizes the most common activation functions used in NNs, their equations, output ranges, and key uses.

Name	Equation	Output Range	Key Use
ReLU	$\max(0, x)$	$[0, \infty)$	Hidden layers, fast train
Sigmoid	$\frac{1}{1+e^{-x}}$	$(0, 1)$	Binary classification
Tanh	$\tanh(x)$	$(-1, 1)$	Hidden layers, zero-centered
Softmax	$\frac{e^{x_i}}{\sum_j e^{x_j}}$	$(0, 1), \sum_i = 1$	Multi-class output

Tab. 1: Summary of Common Neural Network Activation Functions

1.4.1 ReLU (Rectified Linear Unit)

ReLU is a widely used activation function in neural networks. It is defined as:

$$\text{relu}(x) = \max(0, x) = \begin{cases} 0 \text{ if } & x \leq 0 \\ x \text{ if } & x > 0 \end{cases} \tag{6}$$

ReLU(x) is called piecewise linear because it consists of two linear segments as shown in Fig. 3: (i) a constant function that always return 0 when $x \leq 0$, (ii) an identity function that returns x when $x > 0$. A ReLU activated neuron is said to be *active* if its input is greater than zero and *inactive* otherwise.

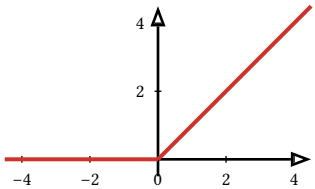


Fig. 3: ReLU (Rectified Linear Unit) function.

Logical encoding ReLU can be encoded using the following logical formula:

$$y = \text{relu}(x) \iff (x \leq 0 \land y = 0) \lor (x > 0 \land y = x) \tag{7}$$

In other words, if $x \leq 0$, y must be 0; otherwise, y must equal x.

Example 1.4.1.1

In Z3, we can declare ReLU using `If()`

```
import z3
def relu(x): return z3.If(x <= 0, 0, x)

relu(-1.2)  # returns 0
relu(0)     # returns 0
relu(2.8)   # returns 2.8
```

◊

Nonlinear Property Despite being piecewise linear, ReLU is **nonlinear** because it does not satisfy the two core properties of a linear function:

- *Additivity:* $\text{relu}(x + y) \neq \text{relu}(x) + \text{relu}(y)$ in general.
- *Homogeneity:* $\text{relu}(\alpha x) \neq \alpha \cdot \text{relu}(x)$ when $\alpha < 0$.

In simpler terms, ReLU is nonlinear because it does not form a straight line. It has a **kink**—a sharp bend—when $x = 0$, where the slope changes abruptly from 0 to 1. This discontinuity in the derivative prevents the function from being globally linear.

This non-linearity makes NNV difficult. In fact, verifying networks with ReLU is NP-complete, as shown in Sec. 3.2. We will use ReLU throughout this book as the default activation function for hidden neurons in an NN.

1.4.2 Sigmoid

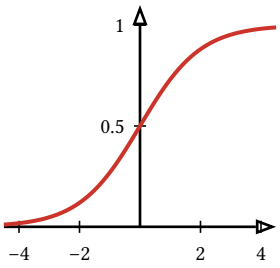


Fig. 4: Sigmoid function.

Sigmoid, shown in Fig. 4, is a smooth—i.e., continuous and differentiable—non-linear activation function that maps any real value to the range $(0, 1)$. It is continuous, meaning that small changes in the input will result in small changes in the output, and differentiable, meaning that it has a well-defined derivative at every point. Sigmoid is often used in the output layer of a binary classification problem.

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}} \tag{8}$$

Example 1.4.2.1

$\text{sigmoid}(-1.2) \approx 0.23$, $\text{sigmoid}(0) = 0.5$, and $\text{sigmoid}(2.8) \approx 0.94$. This means that the sigmoid function maps -1.2 to a value close to 0, 0 to 0.5, and 2.8 to a value close to 1. ◊

1.4.3 Hyperbolic Tangent (Tanh)

Tanh, shown in Fig. 5, is similar to sigmoid (Sec. 1.4.2) but maps any real value to the range (-1,1). It is often used in the output layer of a multi-class classification problem.

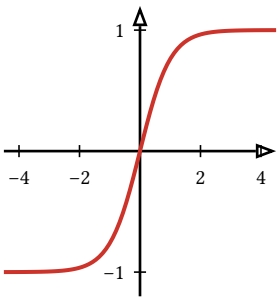


Fig. 5: tanh (hyperbolic tangent) activation function.

Example 1.4.3.1

$\tanh(-1.2) \approx -0.83$, $\tanh(0) = 0$, and $\tanh(2.8) \approx 0.99$. This means that the tanh function maps -1.2 to a value close to -1, 0 to 0, and 2.8 to a value close to 1.

◊

1.4.4 Softmax

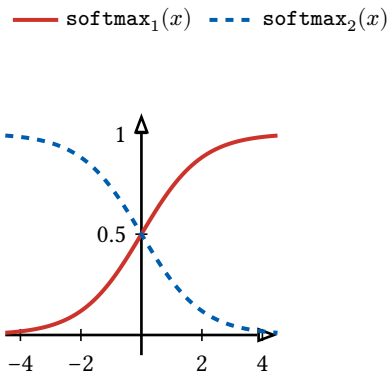


Fig. 6: Softmax function for 2 classes.

Softmax, shown in Fig. 6, is a generalization of sigmoid (Sec. 1.4.2) that maps any real value to the range (0,1) and ensures that the sum of the output values is 1. It is often used in the output layer of a multi-class classification problem.

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}} \tag{9}$$

Example 1.4.4.1 (Softmax Computation)

For a vector $x = [2, 1, 0]$, softmax computes:

$$\begin{aligned} \text{softmax}(x) &= \left[\frac{e^2}{e^2+e^1+e^0}, \frac{e^1}{e^2+e^1+e^0}, \frac{e^0}{e^2+e^1+e^0}\right] \\ &= \left[\frac{7.389}{7.389+2.718+1}, \frac{2.718}{7.389+2.718+1}, \frac{1}{7.389+2.718+1}\right] \\ &\approx [0.71, 0.24, 0.05] \end{aligned} \tag{10}$$

Softmax maps the input vector x to a probability distribution: the first class has probability 0.71, the second 0.24, and the third 0.05.

◊

Example 1.4.4.2

Use Z3 to encode the network in Fig. 1 and compute its output x_5 for the input $(x_1, x_2) = (2.0, 0.5)$ and find an input that maximizes the output x_5.

```
from z3 import Real, If, Solver, Optimize, sat

x1, x2, = Real('x1'), Real('x2')
x3, x3_, = Real('x3'), Real('x3_')
x4, x4_ = Real('x4'), Real('x4_')
x5 = Real('x5')

l2a = x3 = -0.5 * x1 + 0.5 * x2 + 1.0
l2a_ = x3_ = If(x3 > 0, x3, 0)
l2b = x4 =  0.5 * x1 - 0.5 * x2 - 1.0
l2b_ = x4_ = If(x4 > 0, x4, 0)
l3 = x5 = -1.0 * x3_ + 1.0 * x4_ - 1.0

# output for input (x1, x2) = (2.0, 0.5)
s = Solver()
s.add(l2a); s.add(l2b)
s.add(l2a_); s.add(l2b_); s.add(l3)

# specify input values
s.add(x1 = 2.0); s.add(x2 = 0.5)
if s.check() = sat:
  print(s.model())

# find an input that maximizes the output x5
o = Optimize()
o.add(l2a); o.add(l2b)
o.add(l2a_); o.add(l2b_); o.add(l3)

# maximize x5
o.maximize(x5)
if o.check() = sat:
  print(o.model())
```

◊

Exercise 1.4.4.3

Use Example 1.4.4.2 as an example and try the following exercises:

1. Use Z3 to encode the network in Fig. 1 and compute the outputs of *all* neurons for the input $(x_1, x_2) = (-1.3, 3.5)$.
2. Use Z3 to find an input (x_1, x_2) that *maximizes* the output x_5 of the network in Fig. 1.
3. Add another input x_{new} and update the network accordingly. Then, use Z3 to find an input $(x_1, x_2, x_{\text{new}})$ that *minimizes* the output x_5 of the updated network.

♡

1.5 Neural Network Architectures and Layers

NNs vary in architecture depending on how information flows through them and how computations are structured. Most common models are variations of the *feedforward network*, with additional structures or constraints layered on top. Tab. 2 summarizes several common NN architectures and their typical application domains.

Name	Acronym	Typical Applications
Feedforward NN	FNN / MLP	General function approximation, tabular data
Convolutional NN	CNN	Image processing, video analysis
Residual NN	ResNet	Deep image recognition, medical imaging
Recurrent NN	RNN	Sequence modeling, NLP, time series
Transformer	–	NLP, summarization, code generation, vision
Graph NN	GNN	Graph-structured data, molecule modeling, recommendation

Tab. 2: Popular NN Architectures and Applications

1.5.1 Feedforward Neural Networks (FNNs)

In an FNN, information flows in one direction: from the input layer, through one or more hidden layers, and finally to the output layer. There are no loops or cycles in the computation graph.

Widely used feedforward architectures include fully connected, convolutional, and residual networks. Each architecture has its own strengths and is suited for different types of tasks.

Fully Connected NNs (FCNs) In FCNs, each neuron in a layer is connected to every neuron in the next layer. Thus, every neuron in the input layer is connected to every neuron in the first hidden layer, every neuron in the first hidden layer is connected to every neuron in the second hidden layer, and so on, until the output layer. Fully connected NNs, sometimes called *dense networks*, are the most basic type of FNNs and are commonly used for tasks like classification.

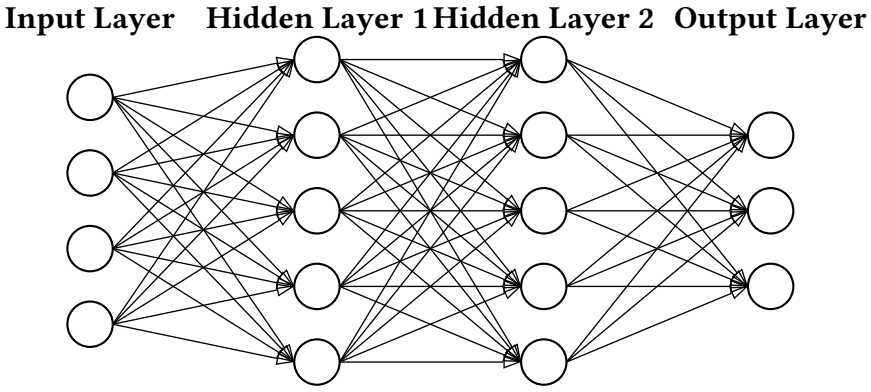


Fig. 7: A fully connected NN with two hidden layers.

Example 1.5.1.1 (Classifier)

A common use case for FCNs is classification. They take an input, e.g., pixels of an image, and predict what the image is (e.g., cat, dog, car). The output layer represents the probabilities of each class (e.g., y_1 is the probability of cat, y_2 is the probability of dog). Moreover, the outputs are often passed through a `softmax` activation function (Sec. 1.4.4) to convert them into probabilities that sum to 1, and the class with the highest probability is chosen as the predicted label.

◊

Convolutional NNs (CNNs) CNNs replace fully connected layers with *convolutional layers*, which apply local filters across the input space. In CNNs, each neuron receives several inputs, takes a weighted sum over them, passes it through an activation function, and responds with an output. CNNs are commonly used in computer vision and image processing. Despite their local structure, CNNs remain feedforward: data flows forward without cycles.

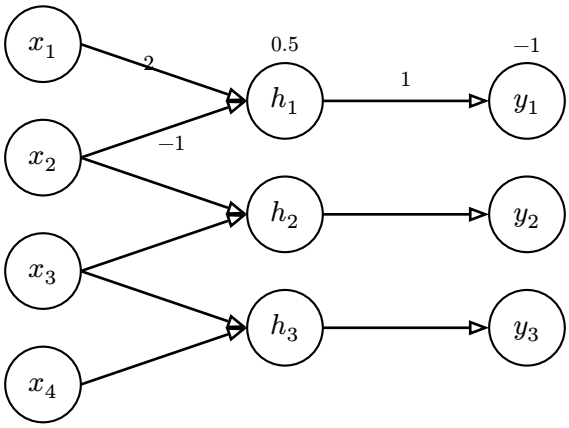


Fig. 8: 1-dimensional CNN

Example 1.5.1.2 (CNN Computation)

Fig. 8 shows a simple CNN with four inputs, three hidden neurons, and three outputs. Given an input vector $\boldsymbol{x} = [x_1, x_2, x_3, x_4]$, the computation of the first output proceeds as follows. The first hidden unit forms a linear combination of its inputs as $h_1 = 2x_1 - x_2 + 0.5$. This value is then passed through the ReLU activation function, resulting in $\hat{h}_1 = \text{relu}(h_1) = \max(0, 2x_1 - x_2 + 0.5)$. Finally, the first output is simply $y_1 = \hat{h}_1 - 1$.

◊

Residual Networks (ResNets) ResNets extend FNNs by adding *skip connections* — direct links that bypass one or more layers. ResNets are often used in image recognition and classification.

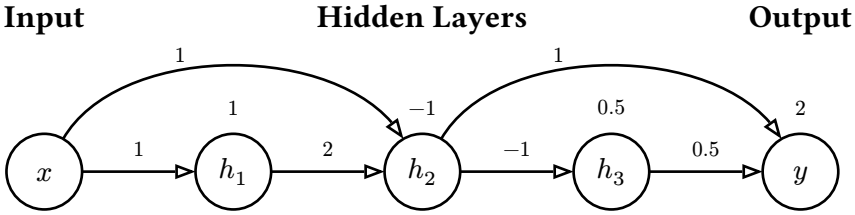


Fig. 9: ResNet block with weights on all connections and biases above nodes. Each node is labeled and typical ReLU is applied after each hidden sum.

Example 1.5.1.3 (ResNet Computation)

Fig. 9 shows an example of a ResNet. Assuming each hidden node applies the ReLU activation, with the weights and biases shown in the diagram, the outputs are:

$$\begin{aligned} h_1 &= \text{relu}(x+1) & h_2 &= \text{relu}(2h_1 + x - 1) \\ h_3 &= \text{relu}(-h_2 + 0.5) & y &= 0.5h_3 + h_2 + 2 \end{aligned} \tag{11}$$

◊

1.5.2 Other NN Architectures

Not all neural networks are feedforward. Some architectures introduce cycles, dynamic connections, or non-Euclidean[1] data structures (e.g., graphs).

1.5.2.1 Recurrent Neural Networks (RNNs)

RNNs, often used in natural language processing (NLP) and speech recognition, are designed to recognize patterns in sequences of data. RNNs have *loops* in them, allowing information to be sent forward and backward.

Currently, most NNV work tackles RNNs by *unrolling* them into an equivalent feedforward network. While this allows us to apply feedforward verification techniques, it can lead to an exponential increase in the size of the network, making verification more challenging.

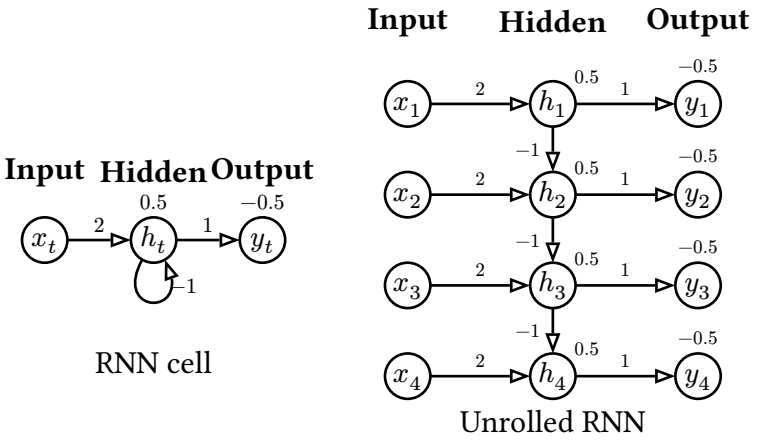


Fig. 10: RNN cell (left) and unrolled RNN sequence structure (right). Weights on connections; biases above hidden and output nodes.

Example 1.5.2.1 (RNN Computation)

Fig. 10 shows a simple RNN cell. Assume the input sequence is $\boldsymbol{x} = [x_1, x_2, x_3, x_4]$ and the initial hidden state is h_0. For the first time step, $h_1 = \text{relu}(2x_1 - h_0 + 0.5)$ and $y_1 = h_1 - 0.5$. For the second time step, $h_2 = \text{relu}(2x_2 - h_1 + 0.5)$ and $y_2 = h_2 - 0.5$.

◊

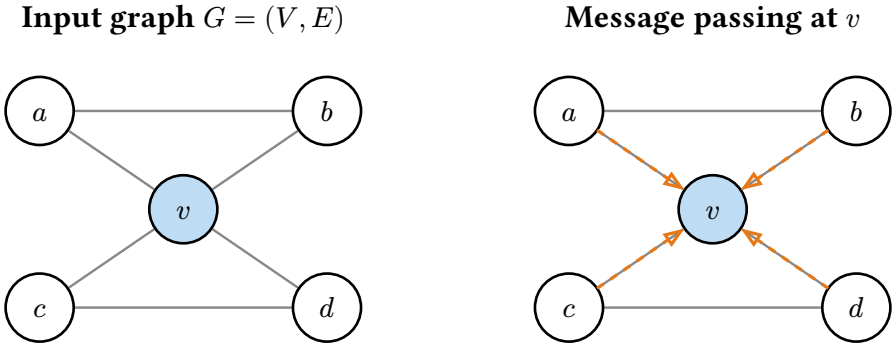
[1] Euclidean data refers to data that can be represented in a flat, two-dimensional space, such as images.

1.5.2.2 Transformers

Transformers are designed for long-range dependencies using *self-attention* rather than recurrence. They dominate applications in natural language processing and are increasingly used in vision and reinforcement learning.

1.5.2.3 Graph Neural Networks (GNNs)

Graph Neural Networks (GNNs) operate on graphs (instead of vectors like FNNs or sequences like RNNs). It uses message passing between nodes in the graph and allows each node to aggregate information from its neighbors. GNNs are used in applications involving structured data like molecules or social networks. Currently, most NNV tools do not support GNNs, and we will not focus on them in this book. However, they are an important area of research in the broader field of NNV.



$$h_v^{(k)} = \mathrm{UPDATE}\Big(h_v^{(k-1)},\ \mathrm{AGG}_{u \in N(v)}\ \mathrm{MSG}\Big(h_u^{(k-1)}\Big)\Big) \quad (12)$$

Fig. 11: Graph neural network message passing. Left: an input graph $G = (V, E)$. Right: node v updates its representation by aggregating messages (dashed orange) from its neighbors $N(v) = \{a, b, c, d\}$, then combining them with its own previous state. Stacking k such layers lets information propagate over k-hop neighborhoods.

1.6 Representing Neural Networks with ONNX

ONNX (Open Neural Network Exchange) [2] is an open source and widely adopted standard for representing neural networks. It provides a common format for representing the structure and parameters of neural networks, enabling interoperability between different ML frameworks and tools.

ONNX operators that cover most sequential feedforward networks include:

- `Add, Sub, Gemm, MatMul`: basic arithmetic and matrix operations.
- `ReLU, Sigmoid, SoftMax`: activation functions.
- `AveragePool, MaxPool, Flatten, Reshape`: tensor manipulation.
- `Conv, BatchNormalization, LRN`: CNN layers and normalizations.
- `Concat, Dropout, Unsqueeze`: tensor operations.

Example 1.6.1

For the network in Fig. 1, the ONNX representation would include:

- Input: x_1, x_2.
- Hidden layer: $x_3 = \text{relu}(-0.5x_1 + 0.5x_2 + 1.0)$, $x_4 = \text{relu}(0.5x_1 - 0.5x_2 + 1.0)$.
- Output: $x_5 = -x_3 + x_4 - 1$.

The ONNX representation would look like:

```
ir_version: 9
opset_import { version: 13 }
graph example_nn {
  input: "x"
  output: "x5"

  node { op_type: "Gemm" input: "x" input: "W1" input: "b1" output: "h1" }   #
-0.5*x1+0.5*x2+1
  node { op_type:  "relu" input: "h1" output: "x3" }

  node { op_type: "Gemm" input: "x" input: "W2" input: "b2" output: "h2" }   #
0.5*x1-0.5*x2+1
  node { op_type:  "relu" input: "h2" output: "x4" }

  node { op_type: "Neg" input: "x3" output: "nx3" }
  node { op_type: "Add" input: "nx3" input: "x4" output: "s1" }
  node { op_type: "Add" input: "s1" input: "c_minus_1" output: "x5" }

  initializer { name: "W1" values: [-0.5, 0.5] }
  initializer { name: "b1" values: [1.0] }
  initializer { name: "W2" values: [0.5, -0.5] }
  initializer { name: "b2" values: [1.0] }
  initializer { name: "c_minus_1" values: [-1.0] }
}
```

◊

2 Properties

Similar to traditional software systems, neural networks (NNs) have desirable properties to ensure the network behaves as expected. These could be specific to the applications modeled by the network, e.g., safety properties for a network modelling a collision avoidance system, or general properties that are desired by all networks, e.g., robustness to small perturbations in the input data.

Below we will discuss properties that are relevant to the verification of NNs. Specifically, these properties can be expressed in a formal language supported by a NNV tool. Additional properties can be found in the literature cite{seshia2018formal}.

2.1 Definition

As described in Sec. 1, NNs define functions of the form $f : \mathbb{R}^n \to \mathbb{R}^m$, where n is the dimension of the input and m is the dimension of the output. Thus, the properties or specifications of an NN—similarly to properties of software program—are defined in terms of its input and output:

"*For any input $x \in \mathbb{R}^n$ satisfying a precondition P, the neural network should produce an output $f(x) \in \mathbb{R}^m$ that satisfies a postcondition Q.*"

This says that if the input x satisfies the precondition P, then the output $f(x)$ should satisfy the postcondition Q.

2.2 Common Properties in Neural Networks

We now define some commonly studied properties in NNs verification.

2.2.1 Robustness

Robustness ensures that small changes in the input do not drastically change the output. This is a desirable property for all neural networks, especially classifiers, where we want to ensure that similar inputs yield similar outputs. For example, a slightly blurred image of a red light should still be classified as a red light.

There are two types of robustness properties: emph{local} (robustness around a chosen point) and emph{global} (robustness everywhere).

2.2.1.1 Local Robustness

A neural network f is ε-locally robust at point x with respect to norm $\|\cdot\|$ if

$$\forall x', \quad \|x - x'\| \leq \varepsilon \Rightarrow f(x) = f(x'). \tag{13}$$

where $|x - x'| \leq \varepsilon$ indicates that the difference between the two points is within a certain (small) threshold ε.

Thus local robustness says that a network is robust if all nearby inputs x' (within radius ε) are classified the same as x. In other words, no small perturbation around this specific point x will fool the classifier.

Local robustness is what most adversarial robustness papers mean, e.g., checking whether an image of a cat is still classified as a cat under small pixel noise.

Example 2.2.1.1 (Local Robustness: Image Classification)

Consider a neural network f that classifies images into different categories (e.g., dog, cat, etc.). A robustness property requires that if an input image c is classified as a dog, then any perturbed image x that is visually similar to c should also be classified as a dog. This can be expressed as:

$$\forall x, \quad \|c - x\| \leq \varepsilon \Rightarrow f(x) = f(c) \tag{14}$$

◊

2.2.1.2 Global Robustness

A neural network f is ε-globally robust with respect to norm $\|\cdot\|$ if

$$\forall x_1, x_2, \quad \|x_1 - x_2\| \leq \varepsilon \Rightarrow f(x_1) = f(x_2). \tag{15}$$

This says the property must hold for all pairs of inputs in the domain: whenever two points are within ε of each other, they must share the same label.

Global robustness is a very strong definition, and in practice, almost impossible unless the network is trivial (e.g., outputs the same label everywhere).

Problem 2.2.1.2 (Robustness: Image Classification)

Suppose that a network classifies an input image x as digit 7. We want to ensure that if the image is slightly perturbed (e.g., brightness changed by a small amount ε), the network still outputs 7.

Solution.

$$\forall x'.\ \|x - x'\| \leq \varepsilon \Rightarrow f(x') = f(x) = 7 \tag{16}$$

◊

2.2.2 Safety

Safety properties ensure that the network conforms to certain safety constraints. This is particularly important in safety-critical applications, such as autonomous vehicles or medical diagnosis systems, where incorrect outputs can lead to catastrophic consequences. %For example, in a self-driving car, we want to ensure that the car maintains a safe distance from other vehicles and pedestrians under all scenarios.

Example 2.2.2.1 (Collision Avoidance System)

A safety property in a collision avoidance system such as an autonomous vehicle might be that if the intruder is distant and significantly slower than us, then we stay below a certain velocity threshold. Formally, this can be expressed as:

$$d_{\text{intruder}} > d_{\text{threshold}} \land v_{\text{intruder}} < v_{\text{threshold}} \Rightarrow v_{\text{us}} < v_{\text{threshold}} \tag{17}$$

where d_{intruder} is the distance to the intruder, $d_{\text{threshold}}$ is a predefined safe distance, v_{intruder} is the speed of the intruder, and v_{us} is our speed.

Unlike robustness properties, which are often desirable in all networks, safety properties are often *specific* to the application domain. For example, a safety property for an autonomous vehicle may not be relevant for a surgical robot.

◊

Exercise 2.2.2.2 (Safety: Self-Driving Car)

A network controlling a self-driving car on a highway might require that: If the car in front is at least 160 meters away and moving slower than us, then we should not accelerate.

♡

2.2.3 Other properties

2.2.3.1 Consistency

Consistency requires that a NN behaves consistently when given semantically equivalent or related inputs.

Example 2.2.3.1 (Logical Consistency in LLMs)

Consider queries q_1, q_2, and q_3 about a person's age:

q_1 : How old is person X?

q_2 : What year was X born if they are currently Y years old?

q_3 : Will X be Z years old in 2025?

Thus, if the LLM outputs age Y for q_1, then q_2 should output `current_year - Y`, and the answer to q_3 should be logically consistent with the stated age. This prevents scenarios where an LLM claims someone is 30 years old but was born in 1985 when the current year is 2024.

◊

2.2.3.2 Monotonicity

Monotonicity ensures that the NN maintains a consistent ordering relationship between inputs and outputs: an increase in certain input features always leads to a non-decreasing output value. This property is important in applications where domain knowledge dictates logical ordering constraints, such as fairness-aware systems, medical diagnosis, and scientific applications where physical laws impose natural ordering relationships.

Example 2.2.3.2 (Fairness)

A network modelling the probability of admission to a university should be monotonically non-decreasing with respect to GPA and test scores, regardless of gender. Formally, for applicants with profiles (p, s, g) and (p', s', g') where p, p' are GPAs, s, s' are test scores, and g, g' are gender indicators:

$$\begin{aligned} p \leq p' \land s = s' \land g \neq g' \Rightarrow f(p, s, g) \leq f(p', s', g') \\ s \leq s' \land p = p' \land g \neq g' \Rightarrow f(p, s, g) \leq f(p, s', g') \end{aligned} \tag{18}$$

where f is the neural network computing admission probability. Additionally, for fairness:

$$f(p, s, \text{male}) = f(p, s, \text{female}) \qquad \forall p, s \tag{19}$$

ensuring that applicants with identical academic qualifications receive the same treatment regardless of gender.

◊

2.3 Counterexamples

A *counterexample* (**cex**) is a witness that falsifies the correctness property. Given the property defined in Sec. 2.1, a counterexample is an input x that satisfies the precondition P but produces an output $f(x)$ that violates the postcondition Q.

Example 2.3.1 (Counterexample to Robustness Property)

For local robustness property (Sec. 2.2.1):

$$f(x) \neq f(x') \land \|x - x'\| \leq \varepsilon \Rightarrow x' \quad \text{is a counterexample.} \tag{20}$$

The goal of NNV (Sec. 3) is to either prove that a property holds—no cex exist—or find a cex that violates the property.

◊

Example 2.3.2 (Counterexample: Monotonicity in Admission))

A network predicts admission probability to a university. Inputs: GPA p and test score s.

We want: a higher GPA with same score should not decrease admission probability.

But we observe a case violating this:

- A: GPA = 3.0, score = 1500, prediction = 0.8
- B: GPA = 3.5, score = 1500, prediction = 0.6

Using the numbers in this violating case, write the monotonicity requirement and show how this is a counterexample.

Monotonicity requirement:

$$p \leq p' \wedge s = s' \Rightarrow f(p, s) \leq f(p', s') \tag{21}$$

Counterexample:

$$3.0 \leq 3.5 \wedge 1500 = 1500 \quad \text{but} \quad f(3.0, 1500) = 0.8 > f(3.5, 1500) = 0.6 \tag{22}$$

◊

2.4 The VNN-LIB Specification Language

The VNN-LIB specification [3], [4] defines a standard format to describe neural networks and their properties. This enables the sharing of benchmarks across different tools and platforms, facilitating evaluations and comparisons of their performance. VNN-COMPs [5] uses VNN-LIB to evaluate different neural network verification tools.

Specifically, VNN-LIB defines a common format for the following components:

1. **Neural Network (or model) representation** in the ONNX format [2].
2. **Property specification** in SMT-LIB format [6] (Sec. 1.6).

2.4.1 VNN-LIB: Property Specification Language

Verification tasks involve proving that the output of a network remains within some desired post-condition σ, given inputs within a bounded set Π.

Formal Specification Let $\nu : D^{n_1 \times \ldots \times n_h} \to D^{m_1 \times \ldots \times m_k}$ be a neural network, and x and y its input and output tensors. A property is expressed as:

$$\forall x \in \Pi \to \nu(x) \in \sigma \tag{23}$$

This includes:

- **Precondition** Π: constraints on inputs.
- **Postcondition** σ: required properties of outputs.

Properties are encoded in **SMT-LIB2**, referencing input/output variable names consistent with ONNX.

Example 2.4.1.1 (ACAS XU VNN-LIB Property)

A typical ACAS XU network maps 5D input to 5D output via 6 layers of 50 ReLU neurons. A property of the network is:

$$-\varepsilon_i \leq x_i \leq \varepsilon_i \quad (0 \leq i < 5) \tag{24}$$

$$y_0 \leq y_1, \quad y_0 \leq y_2, \quad y_0 \leq y_3, \quad y_0 \leq y_4, \tag{25}$$

where x_i are the input variables, y_i are the output variables, and ε_i are small perturbation bounds for each input dimension. This property states that if the inputs are perturbed within the bounds ε_i, then the first output neuron y_0 is less than or equal to all other output neurons y_1, y_2, y_3, y_4.

The VNN-LIB code for this property is:

```
; declaring the input variables
(declare-const X_0 Real)
(declare-const X_1 Real)
(declare-const X_2 Real)
(declare-const X_3 Real)
(declare-const X_4 Real)
; declaring neuron outputs
(declare-const Y_0 Real)
(declare-const Y_1 Real)
(declare-const Y_2 Real)
(declare-const Y_3 Real)
(declare-const Y_4 Real)
; asserting the input relations
(assert (<= X_0 eps0))
(assert (>= X_0 -eps0))
(assert (<= X_1 eps1))
(assert (>= X_1 -eps1))
(assert (<= X_2 eps2))
(assert (>= X_2 -eps2))
(assert (<= X_3 eps3))
(assert (>= X_3 -eps3))
(assert (<= X_4 eps4))
(assert (>= X_4 -eps4))
; asserting the output relations
(assert (<= Y_0 Y_1))
(assert (<= Y_0 Y_2))
(assert (<= Y_0 Y_3))
(assert (<= Y_0 Y_4))
```

◊

2.5 Problems

Problem 2.5.1 (Robustness + Safety: Drone Controller)

An autonomous drone computes its thrust level using a (neural) network controller $f(w, d)$, where w is wind speed and d the distance to nearest obstacle. State the following properties in formal logic (remember to use quantifiers, e.g., $\forall$ and $\exists$):

1. Robustness: If the wind speed reading changes by at most 1 unit ($|w - w'| \leq 1$) and distance remains the same, then thrust decision should remain the same.
2. Safety: If the obstacle distance is less than 5 meters, then thrust must not be greater than 2.

Solution. Robustness:

$$\forall w, w', d, \quad |w - w'| \leq 1 \Rightarrow f(w, d) = f(w', d) \tag{26}$$

Safety:

$$\forall w, d, \quad d < 5 \Rightarrow f(w, d) \leq 2 \tag{27}$$

◊

Problem 2.5.2 (Global Consistency: Celsius and Fahrenheit)

An NN answers two related questions: Q1: "What is the temperature in Celsius?" (input t_C) Q2: "What is the temperature in Fahrenheit?" (input t_F)

State the following consistency property in formal logic (remember to use quantifiers, e.g., $\forall$ and $\exists$): If Q1 outputs y, then Q2 must output $1.8y + 32$.

Solution.

$$\forall t_C, \quad f(\text{Q2}, t_C \times 1.8 + 32) = 1.8 \cdot f(\text{Q1}, t_C) + 32 \tag{28}$$

◊

Problem 2.5.3 (Counterexample: Safety in Vehicles)

A neural network computes car acceleration a. State a safety property in formal logic (remember to use quantifiers, e.g., $\forall$ and $\exists$): if distance to obstacle $d < 10$, then $a \leq 0$. But we observe a scenario where $d = 5$ but $a = +2$.

Write the safety requirement and show how this is a counterexample.

Solution. Safety requirement:

$$\forall d, a, \quad d < 10 \Rightarrow a \leq 0 \tag{29}$$

Counterexample:

$$d = 5, \quad f(d) = a = +2 \quad \text{violates property.} \tag{30}$$

◊

Problem 2.5.4 (Z3 Properties)

Consider the neural network shown below. Recall that in this network the hidden neurons ($v_1 ... v_4$) use ReLU activation, but the output neurons (y_1, y_2) does not.

- Encode this network using Z3.
  - Try to use Z3 to evaluate the network on various inputs.
- Come up with *three* properties that this network *does not* have. Recall that a property often has the form shown in Sec. 2.1.
  - Argue why each property does not hold by providing a counterexample (Sec. 2.3).
  - (Optional, Easy) Try to use Z3 to show that the properties you came up with do not hold by asking Z3 to find counterexamples (one for each property).
- Come up with *three* properties that that this network has (i.e., as long as the precondition holds, the postcondition holds).
  - Argue why each property always holds (e.g., for any input satisfying the range X, the first layer outputs values in range Y, etc).
  - (Optional) Show that the properties you came up will always hold by using Z3 to prove that no counterexample exists (i.e., unsat).

◊

3 The Neural Network Verification (NNV) Problem

Definition 3.1 (NNV)

Given a NN N and a property φ, the NNV problem asks if φ is a valid property[2] of N.

Typically, φ is a formula of the form

$$\varphi_{\text{in}} \rightarrow \varphi_{\text{out}}, \tag{31}$$

where φ_{in} is the *precondition*, i.e., a condition over the inputs of N, and φ_{out} is the *postcondition*, i.e., a condition over the outputs of N.

An NNV tool attempts to find a *counterexample* input to N that satisfies φ_{in} but violates φ_{out}. If no such counterexample exists, φ is a valid property of N. Otherwise, φ is not valid and the counterexample can be used to retrain or debug the network [7].

♣

Example 3.2 (Safety Property for Collision Avoidance System)

In Example 2.2.2.1, we defined a safety property (Sec. 2.2.2) for a collision avoidance system:

$$d_{\text{intruder}} > d_{\text{threshold}} \land v_{\text{intruder}} < v_{\text{threshold}} \rightarrow v_{\text{us}} < v_{\text{threshold}}, \tag{32}$$

where d_{intruder} is the distance to the intruder, $d_{\text{threshold}}$ is a predefined safe distance, v_{intruder} is the speed of the intruder, and v_{us} is our speed.

Here, the precondition is

$$\varphi_{\text{in}} = d_{\text{intruder}} > d_{\text{threshold}} \land v_{\text{intruder}} < v_{\text{threshold}}, \tag{33}$$

and the postcondition is

$$\varphi_{\text{out}} = v_{\text{us}} < v_{\text{threshold}}. \tag{34}$$

◊

[2] Sec. 2 provides various examples of properties.

Example 3.3

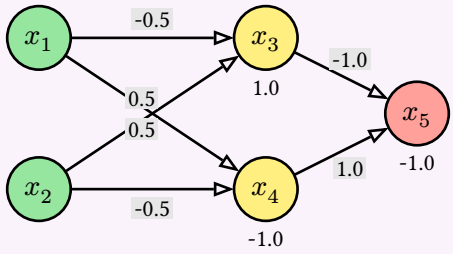


Fig. 12: A simple network (similar to Fig. 1).

For the network in Fig. 12, a *valid* property is that the output is $x_5 \leq 0$ for any inputs $x_1 \in [-1, 1], x_2 \in [-2, 2]$.

An *invalid* property is that $x_5 > 0$ for similar inputs. A counterexample showing this property violation is $\{x_1 = -1, x_2 = 2\}$, from which the network evaluates to $x_5 = -3.5$. ◊

3.1 Satisfiability Formulation and Checking

As with traditional software verification, NNV is often represented as a satisfiability problem, which can be solved using an SMT solver (e.g., Z3 [8]) or a MILP solver (e.g., Gurobi [9]).

Formulation To formulate the problem, we first define a formula α to represent the network. Typically α is a conjunction of constraints representing the affine transformation (Sec. 1.3) and activation function (Sec. 1.4) of each neuron in the network. For example, for a fully-connected neural network (Sec. 1.5.1) with L layers and N ReLU neurons per layer, α is:

$$\alpha = \bigwedge (i \in [1, L], j \in [1, N]) v_{i,j} = \text{relu}\left(\sum_{k \in [1,N]} \left(w_{i-1,j,k} \cdot v_{i-1,k} \right) + b_{i,j} \right) \tag{35}$$

where $v_{i,j}$ is the output of the j-th neuron in layer i, $w_{i-1,j,k}$ is the weight connecting the k-th neuron in layer $i-1$ to the j-th neuron in layer i, and $b_{i,j}$ is the bias of the j-th neuron in layer i. The input layer is layer 0, i.e., $v_{0,j}$ are the input variables.

With this, the NNV problem (Definition 3.1) can be formulated as checking the validity of the following formula:

$$\alpha \Rightarrow (\varphi_{\text{in}} \Rightarrow \varphi_{\text{out}}) \tag{36}$$

Eq. 36 checks if network N satisfies (implies) the property $\varphi_{\text{in}} \Rightarrow \varphi_{\text{out}}$. This validity checking can be reduced to checking the satisfiability of the formula:

$$\alpha \wedge \varphi_{\text{in}} \wedge \neg \varphi_{\text{out}} \tag{37}$$

If Eq. 37 is unsatisfiable, then φ is a valid property of N. Otherwise, φ is not valid. Moreover, we can extract a counterexample for the original problem from the satisfying assignment of Eq. 37. This formulation is commonly used by NNV tools, as it allows us to leverage powerful SMT/MILP solvers to automatically check properties and find counterexamples.

Problem 3.1.1

Let α represent our network and the robustness property $|x - y| \leq \varepsilon \Rightarrow f(x) = f(y)$. Form the satisfiability formula (Eq. 37) that we need to check.

◊

Problem 3.1.2

Let α represent our network and the property $y > 0$ for any input $x_1 \in [r_1, r_2], x_2 \in [r_3, r_4]$. Form the satisfiability formula (Eq. 37) that we need to check.

◊

Problem 3.1.3 (Validity Formulation)

Show that the formula $\alpha \Rightarrow \varphi_{\text{in}} \Rightarrow \varphi_{\text{out}}$ in Eq. 36 is valid if and only if $\alpha \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$ (Eq. 37) is unsatisfiable.

First, do this by hand (on paper) by explicitly writing out the logical equivalences step by step. Then, verify your result using Z3, i.e., show that the negation of the first formula is equivalent to the second formula.

◊

Example 3.1.4

We represent the network in Fig. 12 as a formula α:

$$\begin{aligned} x_3 &= \text{relu}(-0.5x_1 + 0.5x_2 + 1.0) \wedge \\ x_4 &= \text{relu}(0.5x_1 - 0.5x_2 - 1.0) \wedge \\ x_5 &= -x_3 + x_4 - 1.0 \end{aligned} \tag{38}$$

and the property $x_5 > 0$ for any inputs $x_1 \in [-1, 1], x_2 \in [-2, 2]$ as:

$$\begin{aligned} \varphi_{\text{in}} &= (-1 \leq x_1 \leq 1) \wedge (-2 \leq x_2 \leq 2) \\ \varphi_{\text{out}} &= (x_5 > 0) \end{aligned} \tag{39}$$

◊

Satisfiability Solving We can check the satisfiability of $\alpha \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$ (Eq. 37) using constraint solving, e.g., the Z3 SMT solver. Sec. 4.1 shows how to perform symbolic execution and use an SMT solver to automatically generate this formula from a given network and property, and check its satisfiability. Sec. 4.2 shows how to encode the problem as MILP constraints, solvable using a MILP solver (e.g., Gurobi).

Example 3.1.5

For Example 3.1.4, the formula is satisfiable, i.e., the property is invalid, and the solver returns SAT. Any satisfying assignment, e.g., $x_1 = -1$ and $x_2 = 2$, is a counterexample to the property, as it satisfies the precondition φ_{in} but violates the postcondition φ_{out}, i.e., $x_5 = -3.5$, which is < 0.

◊

Problem 3.1.6

Use Z3 to do Example 3.1.4. You might find Example 1.4.4.2 useful. Make sure that you also ask Z3 to find a counterexample violating the property (it does not have to be the same as above).

◊

Problem 3.1.7

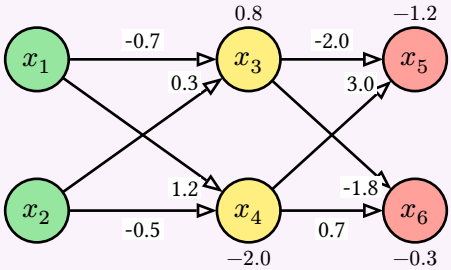


Fig. 14: A simple network with 2 inputs, 2 hidden ReLU neurons, and 2 outputs.

Consider the neural network in Fig. 14. Do the following:

1. Write the formula α representing the network.
2. Create the property φ that the output $x_5 \geq x_6$ for any inputs $x_1 \in [-1, 1]$, $x_2 \in [-1, 1]$.
3. Generate the satisfiability formula $\alpha \wedge \overline{\varphi}$ as shown in Eq. 37.
4. Use Z3 to check the satisfiability of the formula. If it is satisfiable, extract a counterexample input that violates the property.

◊

3.2 Complexity

As shown in [10], [11] ReLU-based NNV is NP-complete by reducing the 3-SAT problem to it. This means that the problem of checking whether a given ReLU-based network satisfies a property is computationally hard, and no polynomial-time algorithm is known to solve it in the general case.

4 Constraint Solving

4.1 Symbolic Execution and SMT Solving

As described in Sec. 3.1 the Neural Network Verification (NNV) problem can be formulated as a satisfiability problem. Specifically, we encode the network as a logical formula, and use a constraint solver to check that formula satisfies the property of interest.

A straightforward and automated way to do this encoding is using *symbolic execution* (SE) [12], [13], a well-known software testing technique for finding bugs. SE executes a program on symbolic inputs, i.e., inputs represented as symbols rather than concrete values, and tracks the reachability of program state as symbolic expressions, i.e., logical formulae over symbolic inputs. The satisfiability of these formulae is then checked using an SMT solver, and satisfying assignments represent inputs leading to the undesirable (buggy) program state.

4.1.1 Symbolic Execution

We can adapt traditional SE to our problem by treating the network as a program and neurons as variables and executing the network on symbolic inputs. Affine transformations can easily be represented as logical formulae because they are linear functions. Activation functions such as ReLUs are translated to disjunctions of linear functions or if-then-else statements:

$$\begin{gathered} y = \text{relu}(x) \\ y = \max(x, 0) \\ y = \text{if } x > 0 \text{ then } x \text{ else } 0 \\ (x > 0 \wedge y = x) \vee (x \leq 0 \wedge y = 0) \end{gathered} \tag{40}$$

Example 4.1.1.1

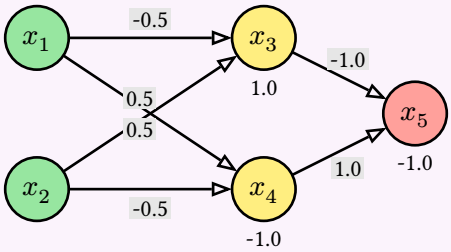


Fig. 16: A simple network with two inputs x_1, x_2, two hidden neurons x_3, x_4, and one output neuron x_5.

To create a logical formula representing the network in Fig. 16, we can symbolically execute the network on symbolic inputs x_1, x_2 and track the values of the neurons x_3, x_4, x_5 as a set (conjunction) of logical formulae. SE starts with the inputs x_1 and x_2 and computes the values of the neurons in the hidden layer, x_3 and x_4, using the affine transformations, followed by ReLUs. Finally, SE computes the output neuron x_5 as a linear combination of the hidden layer neurons.

$$\begin{aligned} x_5 &= -x_3 + x_4 - 1.0 \wedge \\ x_3 &= \max(-0.5x_1 + 0.5x_2 + 1.0, 0) \wedge \\ x_4 &= \max(0.5x_1 - 0.5x_2 - 1.0, 0) \end{aligned} \tag{41}$$

◊

4.1.2 SMT Solving

After obtaining the symbolic representation of the network, we can use an SMT solver [6] to check the satisfiability of the formula $\alpha \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$ as shown in Eq. 37, where α is the symbolic representation of the network, φ_{in} is the precondition on the inputs, and φ_{out} is the postcondition on the outputs.

The solver checks if there exists an assignment to the symbolic inputs that satisfies the formula. If such an assignment exists, it means that the property is violated, and we can extract a counterexample from the satisfying assignment. Otherwise, if no such assignment exists, the property is valid.

Example 4.1.2.1

To check that the network in Fig. 16 satisfies the property $x_5 > 0$ for any inputs $x_1 \in [-1, 1], x_2 \in [-2, 2]$ (Example 3.1.4), represented as:

$$\begin{gathered} \varphi_{\text{in}} = (x_1 \geq -1) \wedge (x_1 \leq 1) \wedge (x_2 \geq -2) \wedge (x_2 \leq 2) \\ \varphi_{\text{out}} = (x_5 > 0) \end{gathered} \tag{42}$$

We check the satisfiability of $\alpha \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$:

$$\begin{aligned} & x_5 = -x_3 + x_4 - 1.0 \wedge \\ & x_3 = \max(-0.5x_1 + 0.5x_2 + 1.0, 0) \wedge \\ & x_4 = \max(0.5x_1 - 0.5x_2 - 1.0, 0) \wedge \\ & \quad (x_1 \geq -1) \wedge (x_1 \leq 1) \wedge (x_2 \geq -2) \wedge (x_2 \leq 2) \wedge (x_5 \leq 0) \end{aligned} \tag{43}$$

In this case, the SMT solver returns `sat` and a satisfying assignment, e.g., $x_1 = -1$ and $x_2 = 2$, which is a counterexample to the property. This means that for these inputs, the output $x_5 = -3.5$ violates the property $x_5 > 0$.

◊

4.1.3 Limitations

While using symbolic execution and SMT solving is a straightforward way to verify networks, it has several practical limitations:

- **Path Explosion and Scalability**: the number of paths that the solver has to analyze can grow exponentially with the number of ReLU-based neurons and layers as each ReLU, represented as a disjunction of linear functions, has two possible outputs for any input (i.e., the input value itself or 0). This leads to the notorious *path explosion* problem and becoming intractable for large networks. Programming assignment 1 (Sec. E.1) demonstrates this issue.
- **Non-linearity:** Other non-linear activation functions (Sec. 1.4), such as Sigmoid or Tanh, might not be easily representable as disjunctions of linear functions as ReLU. This can lead to complex formulae that are hard to reason about and/or result in a large search space for the SMT solver.
- **Precision Issues:** SMT solvers may struggle with precision when dealing with floating-point arithmetic, which is common in neural networks. This can lead to incorrect results or false positives/negatives in the verification process.

For these reasons, SMT solving is mostly used on toy examples, e.g., in a classroom setting. Modern NNV tools do not use SMT solving, and instead, rely on more efficient techniques such as abstraction (Sec. 5).

4.2 MILP

Instead of using SMT solving, we can encode the NNV problem as a Mixed Integer Linear Programming (MILP) constraints (Sec. D.1), and then invoke an MILP solver such as Gurobi [9] to check their satisfiability or *feasibility* (Sec. D.3.1).

4.2.1 ReLU Encoding

We first encode non-linear activation functions like ReLU using *binary indicator* variables and linear constraints. For each neuron, we introduce a binary variable (Sec. D.2.1) that indicates whether the neuron is "active" (output equals input) or "inactive" (output is zero). This transforms the non-linear `max(z,0)` operation into a set of linear inequalities controlled by the binary variable.

Definition 4.2.1.1

We define

- z: the pre-activation value, i.e., the value that goes into ReLU
- $\hat{z}$: the post-activation value, i.e., the output of ReLU
- $a \in \{0, 1\}$: a binary indicator variable encoding whether the neuron is active ($z \geq 0$) or inactive ($z < 0$)
- l, u: lower and upper bounds on z ($l \leq z \leq u$).

♣

Encoding The MILP encoding of $\hat{z} = \max(z, 0)$ is then (Example 4.2.1.3 shows a simpler example that does not involve bounds):

$$\begin{aligned} &\hat{z} \geq z \\ &\hat{z} \geq 0 \\ &\hat{z} \leq a \cdot u \\ &\hat{z} \leq z - l(1 - a) \\ &a \in \{0, 1\} \end{aligned} \tag{44}$$

These constraints enforce $\hat{z} = z$ when $a = 1$ (active) and $\hat{z} = 0$ when $a = 0$ (inactive), which captures the two possible ReLU outputs (0 or z). Note that constraints are linear and involve both continuous variable $\hat{z}$ and binary variable a.

4.2.2 Computing Concrete Bounds

The mentioned lower and upper bounds (Sec. 4.2.1)—*concrete* values l and u such that $l \leq z \leq u$ over the input z to ReLU— are critical for ensuring that the binary indicator a can only take on values that are *valid* given the possible range of z. For example, if $u < 0$, then z is always negative, so the active phase ($a = 1$) is infeasible, and thus can be eliminated. Similarly, if $l \geq 0$, then z is always active.

Either of these cases indicates the neuron is *stable*—always active or always inactive—and thus significantly simplifies the MILP problem because ReLU can be replaced with a linear function (identity or constant 0). Of course, if $l < 0 < u$, then both active and inactive phases are possible, and the MILP solver has to consider both cases. Sec. 5 discusses abstraction techniques to approximate ReLU outputs.

Computing Bounds $l \leq e \leq u$: Given a linear expression e

$$e = a_1 x_1 + a_2 x_2 + \ldots + b, \tag{45}$$

where each variable x_i ranges over $[l_i, u_i]$, a simple way to compute the bounds $l \leq e \leq u$ is using *interval* arithmetic (Sec. 5.4):

- **For the lower bound** (l_3): For each x_i, use its upper bound u_i if $a_i < 0$, and use its lower bound l_i if $a_i \geq 0$.
- **For the upper bound** (u_3): For each x_i, use l_i if $a_i < 0$, and use u_i if $a_i \geq 0$.

This guarantees that the extreme values of e are achieved at some corner of the input box.

Example 4.2.2.1

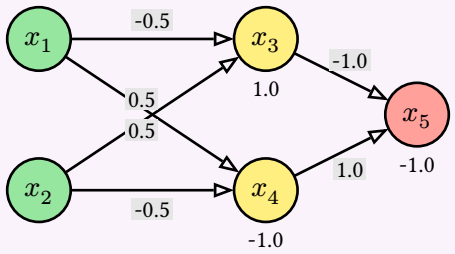


Fig. 18: A simple network (similar to Fig. 1.

Consider neuron x_3 in the network in Fig. 18. We have $x_3 = -0.5x_1 + 0.5x_2 + 1.0$ as the pre-activation value of x_3. The upper u_3 and lower l_3 bounds on x_3 over the input region $x_1 \in [-1, 1]$ and $x_2 \in [-2, 2]$ are:

$$\begin{aligned} z_3 &= -0.5x_1 + 0.5x_2 + 1.0 \\ l_3 &= -0.5(1) + 0.5(-2) + 1.0 = -0.5 \\ u_3 &= -0.5(-1) + 0.5(2) + 1.0 = 2.5 \end{aligned} \tag{46}$$

Notice that we use different pairs of values to compute the lower (1,-2) and upper (-1,2) bounds.

With the computed bounds, we encode the ReLU output of x_3:

$$\begin{aligned} &\hat{x}_3 \geq x_3 \\ &\hat{x}_3 \geq 0 \\ &\hat{x}_3 \leq a_3 \cdot u_3 \\ \Rightarrow\ &\hat{x}_3 \leq a_3 \cdot 2.5 \\ &\hat{x}_3 \leq x_3 - l_3(1 - a_3) \\ \Rightarrow\ &\hat{x}_3 \leq x_3 - (-0.5)(1 - a_3) \\ \Rightarrow\ &\hat{x}_3 \leq x_3 + 0.5(1 - a_3) \\ &a_3 \in \{0, 1\} \end{aligned} \tag{47}$$

Note that because $l_3 = -0.5$ and $u_3 = 2.5$, a_3 can be either 0 or 1, depending on the value of x_3. If we had different bounds, e.g., $l_3 = 0.1$, then a_3 would be 1, as x_3 can never be less than 0.1 (i.e., x_3 is a stable neuron because it is always active). Stable neurons replaces ReLU with either 0 or identity function, which significantly simplifies the problem.

4.2.3 Neural Network Encoding

We can encode a neural network as a set of MILP linear constraints as follows:

$$\begin{aligned}
&(a) \quad z^i = W^i \hat{z}^{(i-1)} + b^i; \\
&(b) \quad y = z^{\{L\}}; x = \hat{z}^{\{0\}}; \\
&(c) \quad \hat{z}^i_j \geq \{z\}^i_j; \quad \hat{z}^i_j \geq 0; \\
&(d) \quad a^i_j \in \{0, 1\}; \\
&(e) \quad \hat{z}^i_j \leq \{a\}^i_j u^i_j; \quad \hat{z}^i_j \leq z^i_j - l^i_j\big(1 - a^i_j\big);
\end{aligned} \tag{48}$$

where x is input, y is output, and z^i, $\hat{z}^i$, W^i, and b^i are the pre-activation, post-activation, weight, and bias vectors for layer i (j is the neuron index in layer i).

Thus, we can encode a neural network as a set of linear constraints by:

1. Define the affine transformation computing the pre-activation value for a neuron in terms of outputs in the preceding layer;
2. Define the inputs and outputs in terms of the adjacent hidden layers;
3. Assert that post-activation values are non-negative and no less than pre-activation values;
4. Define that the neuron activation status indicator variables that are either 0 or 1; and
5. Define constraints on the upper, $u^{\{(i)\}}_j$, and lower, $l^{\{(i)\}}_j$, bounds of the pre-activation value of the jth neuron in the ith layer.

Deactivating a neuron, $a^i_j = 0$, simplifies the first of the (5) constraints to $\hat{z}^i_j \leq 0$, and activating a neuron simplifies the second to $\hat{z}^i_j \leq z^i_j$, which is consistent with the semantics of $\hat{z}^i_j = \max\big(z^i_j, 0\big)$.

Example 4.2.3.1 (Full example)

We use MILP to formulate and check if the network in Fig. 18 satisfies the property $x_5 > 0$ for any inputs $x_1 \in [-1, 1], x_2 \in [-2, 2]$. We do this by checking the feasibility of the MILP constraints encoding $\alpha \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$, where α is the MILP encoding of the network, φ_{in} is the precondition, and φ_{out} is the postcondition. We do this in several steps:

1. **Encoding**: **precondition** φ_{in} and **postcondition** $\neg\varphi_{\text{out}}$:

$$\varphi_{\text{in}} : -1 \leq x_1 \leq 1; \quad -2 \leq x_2 \leq 2, \quad \neg\varphi_{\text{out}} : x_5 \leq 0 \tag{49}$$

   Hidden layer (pre- and post-activation):

$$\begin{aligned} &z_3 = -0.5x_1 + 0.5x_2 + 1.0, \quad z_4 = 0.5x_1 - 0.5x_2 - 1.0 \\ &\hat{z}_3 \geq z_3, \quad \hat{z}_3 \geq 0, \quad \hat{z}_4 \geq z_4, \quad \hat{z}_4 \geq 0 \\ &a_3, a_4 \in \{0, 1\} \\ &\hat{z}_3 \leq a_3 \cdot u_3, \quad \hat{z}_3 \leq z_3 - l_3(1 - a_3), \quad \hat{z}_4 \leq a_4 \cdot u_4, \quad \hat{z}_4 \leq z_4 - l_4(1 - a_4) \end{aligned} \tag{50}$$

   Output layer: $x_5 = -\hat{z}_3 + \hat{z}_4 - 1.0$

2. **Computing** upper and lower bounds over input ranges $x_1 \in [-1, 1], x_2 \in [-2, 2]$:

$$\begin{aligned} z_3 &\in [-0.5 \cdot 1 + 0.5 \cdot (-2) + 1, -0.5 \cdot (-1) + 0.5 \cdot 2 + 1] = [-0.5, 2.5] \\ z_4 &\in [0.5 \cdot (-1) - 0.5 \cdot 2 - 1, 0.5 \cdot 1 - 0.5 \cdot (-2) - 1] = [-2.5, 0.5] \end{aligned} \tag{51}$$

   So we set $l_3 = -0.5, u_3 = 2.5$, and $l_4 = -2.5, u_4 = 0.5$.

3. **Substituting bounds** into the constraints:

$$\begin{aligned} &\hat{z}_3 \leq a_3 \cdot 2.5, \quad \hat{z}_3 \leq z_3 - (-0.5)(1 - a_3) = z_3 + 0.5(1 - a_3) \\ &\hat{z}_4 \leq a_4 \cdot 0.5, \quad \hat{z}_4 \leq z_4 - (-2.5)(1 - a_4) = z_4 + 2.5(1 - a_4) \end{aligned} \tag{52}$$

4. **The final MILP encoding:**

$$\begin{aligned} &-1 \leq x_1 \leq 1; \quad -2 \leq x_2 \leq 2 \\ &z_3 = -0.5x_1 + 0.5x_2 + 1.0, \quad z_4 = 0.5x_1 - 0.5x_2 - 1.0 \\ &\hat{z}_3 \geq z_3, \quad \hat{z}_3 \geq 0, \quad \hat{z}_4 \geq z_4, \quad \hat{z}_4 \geq 0 \\ &a_3, a_4 \in \{0, 1\} \\ &\hat{z}_3 \leq a_3 \cdot 2.5, \quad \hat{z}_3 \leq z_3 + 0.5(1 - a_3) \quad \hat{z}_4 \leq a_4 \cdot 0.5, \quad \hat{z}_4 \leq z_4 + 2.5(1 - a_4) \\ &x_5 = -\hat{z}_3 + \hat{z}_4 - 1.0, \quad x_5 \leq 0 \\ &\text{where } z_3 \in [-0.5, 2.5], z_4 \in [-2.5, 0.5], \hat{z}_3 \in [0, 2.5], \hat{z}_4 \in [0, 0.5]. \end{aligned} \tag{53}$$

1. **Solving** The MILP solver attempts to find a feasible (Sec. D.3.1) or satisfying assignment representing a counterexample violating the property. In this example, it might find $x_1 = -1, x_2 = 2$, which leads to:

$$\begin{aligned} &z_3 = -0.5(-1) + 0.5(2) + 1.0 = 2.5, \quad z_4 = 0.5(-1) - 0.5(2) - 1.0 = -2.5 \\ &a_3 = 1, \hat{z}_3 = 2.5 \quad \text{(neuron active)}, \quad a_4 = 0, \hat{z}_4 = 0 \quad \text{(neuron inactive)} \\ &x_5 = -2.5 + 0 - 1.0 = -3.5 \end{aligned} \tag{54}$$

Since $x_5 = -3.5$, this satisfies our search for $x_5 \leq 0$ and thus is a counterexample. ◊

Problem 4.2.3.2

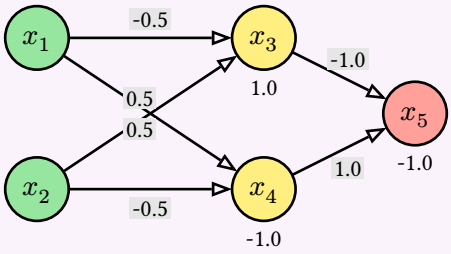


Fig. 20: A simple network (similar to Fig. 1).

Use MILP encoding and solving to check if the network Fig. 20 satisfies the property $x_5 \geq x_6$ for any inputs $x_1 \in [-1, 1], x_2 \in [-1, 1]$. Specificaly, do the following steps:

1. Write down the precondition φ_{in} and postcondition φ_{out}.
2. Write down the MILP encoding of the network.
3. Compute the upper and lower bounds of the pre-activation values of the neurons in the hidden layer.
4. Substitute the bounds into the MILP encoding.
5. Write down the final MILP encoding of $\alpha l \wedge \varphi_{\text{in}} l \wedge \neg\{\varphi_{\text{out}}\}$.
6. Check the feasibility of the MILP encoding (e.g., using Z3) and report the result.

◊

Problem 4.2.3.3

As shown in Example 4.2.2.1, the upper and lower bounds of neuron x_3 of the network in Fig. 18 are $u_3 = 2.5$ and $l_3 = -0.5$.

1. Compute the upper and lower bounds of x_4 in the same example.
2. Change the weights and/or biases of the network in Fig. 18, but keep the input ranges $-1 \leq x_1 \leq 1$ and $-2 \leq x_2 \leq 2$, such that x_3 becomes a stable neuron (always active or always inactive). Show the new weights and/or biases, and compute the new upper and lower bounds of x_3.

◊

4.2.4 Limitations

- **Scalability**: While MILP is efficient in dealing with linear constraints, it still cannot be applied directly to real-world neural networks due to the large search space. Each ReLU application to an unstable neuron introduces a binary variable (Sec. 4.2.1), leading to exponential number of possible branches, and real-world networks can have millions of such ReLUs, making the search space intractable.
- **Limited exploitation of network structure and modern hardware**: Off-the-shelf MILP solvers are designed for arbitrary MILP problems and do not exploit specific structures of neural networks and concepts such as activation patterns (Sec. 6.1) to prune the search space. Moreover, MILP solvers, even industrial-strength ones such as Gurobi [9], are primarily CPU-based and do not leverage the massive parallelism provided by modern GPUs to improve scalability.

- **Advanced Abstraction and Heuristics**: Interval analysis is efficient and commonly used for computing neuron bounds, but cannot capture dependencies between neurons, leading to precision loss in deeper networks. SOTA NNV tools use more advanced abstract domains such as zonotopes and polytopes to improve precision (Sec. 5).

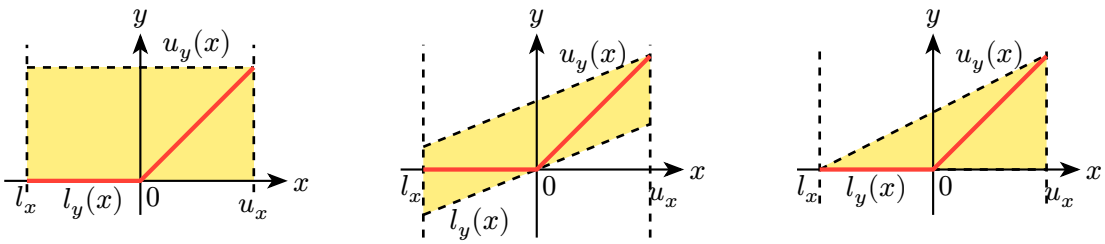
  Modern NNV tools also employ heuristics to decide which neurons to branch on and to determine stable neurons to avoid unnecessary branching. These heuristics are not available in general MILP solvers.

5 Abstractions

As mentioned in Sec. 4.2.1 the computation of upper and lower bounds of the neuron values help determine neuron stability and therefore can help scale NNV. Modern NNV tools explore different *abstraction* techniques to compute these bounds more precisely while balancing computational efficiency.

This chapter discusses the interval, zonotope, and polytope abstract domains commonly used in NNV. Each domain provides a different way to represent and compute the bounds of neurons, with its own trade-offs in terms of precision and computational complexity.

5.1 Overview and Background



(a) Interval (b) Zonotope (c) Polytope

Fig. 22: Abstractions of $\text{relu}(x) = \max(0, x)$ over $x \in [l_x, u_x]$ using (a) interval, (b) zonotope, (c) polytope abstractions.

Abstractions for ReLU We use ReLU to illustrate different abstractions. Our goal is to approximate the bounds of the post-ReLU value of a neuron given the bounds of its pre-ReLU value. For example, for a ReLU neuron $y = \max(0, x)$, we want to compute the bounds $\left[l_{y(x)}, u_{y(x)}\right]$ of y given the bounds $[l_x, u_x]$ of x.

Fig. 22 illustrates common abstractions (or *over-approximations*) for the ReLU function $y = \max(0, x)$, where $x \in [l_x, u_x]$ and $y \in \left[l_{y(x)}, u_{y(x)}\right]$. The values of ReLU are shown as points on the red line, which is non-convex and consists of two linear segments: one for $x < 0$ (where $y = 0$) and another for $x \geq 0$ (where $y = x$). To compute the bounds $l_{y(x)}$ and $u_{y(x)}$, we can use different abstractions:

1. **Interval Abstraction**: Interval represents the post-ReLU value as an interval $\left[l_{y(x)}, u_{y(x)}\right]$ where $l_{y(x)} = 0$ and $u_{y(x)} = u_x$. Interval is a simple and efficient abstraction, but it can be *imprecise*, e.g., it does not capture the relationship between the input and output of ReLU. %, and simply assumes that the output can take any value between 0 and the upper bound of the input.

   In Fig. 22(a), the yellow rectangle (box) represents the interval abstraction in the 2D space of (x, y). As can be seen, this box is too large of an over-approximation (too coarse or loose), as it includes many points that are not achievable by ReLU, e.g., the point $(0.5, 0.0)$ is not on the red line.

2. **Zonotope Abstraction**: Zonotopes, commonly represented using center and generators, can capture linear relationships between variables more effectively.

In Fig. 22(b), the a parallelogram—a zonotope in 2D—is arguably tighter than the interval in Fig. 22(a). It captures the linear relationship between x and y and excludes points that are not achievable by ReLU, e.g., the point $(0.5, 0.0)$ that was included in the interval abstraction is not included in the zonotope. However, it still includes non-ReLU points and is also not strictly better than interval, e.g., the point $(0.5, -0.5)$ is included in the zonotope but not in the interval abstraction (or ReLU).

3. **Polytope Abstraction**: Polytopes can represent arbitrary linear constraints and provide very precise bounds. We can construct a polytope that tightly encloses the non-convex shape of the ReLU function.

   In Fig. 22(c), the polytope is shown as a trapezoid that captures the linear segments of ReLU. The lower bound is $l_{y(x)} = 0$ and the upper bound is $u_{y(x)} = u_x$.

5.2 Geometric Representations

Our abstract domains, e.g., interval, zonotope, and polytopes, are all geometric shapes. There are two common ways to describe a geometric shape:

- **Corner-based representation**: Define the shape by listing all of its corner points (*vertices*). For example, a rectangle in 2D is defined by its four corners, and a box in 3D is defined by its eight corners. This representation is intuitive and directly shows the boundaries of the shape.
- **Generator-based representation**: Define the shape using a *center point* and several direction vectors (*generators*) that define how the shape can stretch or tilt. Instead of listing all corners, it specifies how to reach any point in the shape by starting from the center and moving along the generators within certain limits. This representation is often more compact, especially for higher-dimensional shapes.

Fig. 23 uses corner-based and generator-based representations to represent a 2D rectangle, which is an interval abstraction over two variables. The left side shows the corner-based view, where the rectangle is defined by its 4 corners. The right side shows the generator-based view, where the rectangle is represented as a zonotope with a center point c and two orthogonal generators g_1 and g_2.

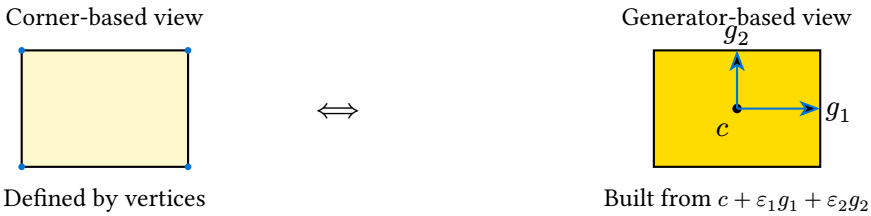


Fig. 23: Two equivalent descriptions of a 2D interval abstraction (a rectangle). Left: defined by its four corners (independent variable bounds). Right: represented as a zonotope with center c and orthogonal generators g_1, g_2.

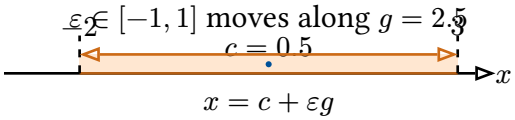


Fig. 24: An interval $[-2, 3]$ is a 1D zonotope with center $c = 0.5$, generator $g = 2.5$, and parameter $\varepsilon \in [-1, 1]$. Varying ε covers the entire interval.

Example 5.2.1 (Interval as Zonotope)

An interval for a single variable is a line segment defined by its two endpoints. Equivalently, it can be represented as a zonotope with a center point and a single generator:

$$[l, u] = \{c + \varepsilon g \ \texttt{mid} \ \varepsilon \in [-1, 1]\} \tag{55}$$

where $c = \frac{l+u}{2}$ is the center, $g = \frac{u-l}{2}$ is the generator, and ε controls how far we move along the generator.

◊

Example 5.2.2 (Interval $[-2, 3]$ as Zonotope)

Consider the interval $x \in [-2, 3]$, shown in Fig. 24. We compute:

$$c = \frac{-2+3}{2} = 0.5, \quad g = \frac{3-(-2)}{2} = 2.5, \quad \varepsilon \in [-1, 1]. \tag{56}$$

Thus, any point x in this interval can be written as $x = 0.5 + \varepsilon \cdot 2.5$, where $\varepsilon \in [-1, 1]$. For example:

$$\varepsilon = -1 \Rightarrow x = -2, \quad \varepsilon = 0 \Rightarrow x = 0.5, \quad \varepsilon = +1 \Rightarrow x = 3. \tag{57}$$

As ε varies between -1 and $+1$, we cover the entire interval $[-2, 3]$.

◊

5.2.1 Transformer Functions

The concept of computing abstractions is central to program analysis, e.g., through *abstract interpretation* techniques. It allows us to reason about the behavior of a program without evaluating it on all concrete inputs—which may be infinite. Instead, we use an *abstract domain*, such as interval, to summarize sets of concrete values, enabling sound and scalable approximation.

5.2.1.1 Abstraction Functions

In abstract interpretation, we have the *abstraction function*

$$\alpha : D \to D^a, \tag{58}$$

which maps a concrete value from the domain D to an element in a finite or simpler abstract domain D^a.

Example 5.2.1.1 (Odd/Even)

The `odd/even` or parity abstraction is defined as:

$$\alpha_{\text{parity}}(x \in \mathbb{Z}) = \begin{cases} \text{even} & \text{if } x \bmod 2 = 0 \\ \text{odd} & \text{if } x \bmod 2 = 1 \end{cases} \tag{59}$$

Even though $\mathbb{Z}$ is infinite, this abstraction maps all integers to a finite set $\{\text{odd}, \text{even}\}$.

◊

5.2.1.2 Transformer Functions

Once we have values in the abstract domains, we often define an *abstract transformer function*

$$f^a : D^a \to D^a, \tag{60}$$

for each operation f to reason about its behavior on abstract values.

Example 5.2.1.2

Consider the function $f(x) = x + 1$. We define the abstract transformers f^a for different abstract domains D^a:

- **Odd/Even abstraction**: $D^a = \{\text{odd}, \text{even}\}$. Then:

$$f^a(\text{odd}) = \text{even}, \quad f^a(\text{even}) = \text{odd} \tag{61}$$

- **Sign abstraction**: $D^a = \{\text{neg}, \text{zero}, \text{pos}\}$. Then:

$$f^a(\text{neg}) = \{\text{neg}, \text{zero}\}, \quad f^a(\text{zero}) = \text{pos}, \quad f^a(\text{pos}) = \text{pos} \tag{62}$$

- **Interval abstraction**: $D^a = \{[a, b] \texttt{ mid } a \leq b \in \mathbb{Z} \cup \{-\infty, +\infty\}\}$. Then:

$$f^a([a, b]) = [a + 1, b + 1] \tag{63}$$

Notice that the input and output of the abstract transformer f^a are both in the abstract domain D^a (e.g., odd/even, sign, or interval).

◊

5.3 Abstract Domains

We now introduce several abstract domains that are commonly used in NNV. Each domain has its own abstract transformer functions to compute the bounds of neurons.

Note that the input to the transformer functions (Sec. 5.2.1) are the abstract values and the output is also an abstract value. For example, for interval transformers, the input is an interval $[l, u]$ and the output is also an interval $[l', u']$. Similarly for zonotope transformers, the input is a zonotope defined by center and generators, and the output is also a zonotope.

5.4 Interval

Interval is a very simple abstraction which represents the possible values of a variable as an interval $[l, u]$, where l is the lower bound and u is the upper bound. For example, the set of values $\{-2.5, -8.2, -10.7, 2, 4.7, 5.1\}$ can be represented as $[-10.7, 5.1]$.

Definition. The interval for one variable v is defined as:

$$v \in [l, u] = \{v \in \mathbb{R} \mid l \leq v \leq u\} \tag{64}$$

For n variables, the interval becomes a box (like a rectangle in 2D, a cuboid in 3D, or a hyperrectangle in nD):

$$[v_1, v_2, ..., v_n] \in [l_1, u_1] \times [l_2, u_2] \times ... \times [l_n, u_n] \tag{65}$$

5.4.1 Affine Transformer

For the linear or affine function f in Sec. 1.3

$$f(v_1, v_2, ..., v_n) = \sum_{i=1}^{n} w_i v_i + b \tag{66}$$

where w_i is the weight for the input v_i, n is the number of output nodes from the previous layer and b is the bias term, the abstract transformer f^a is:

$$f^a([l_1, u_1], ..., [l_n, u_n]) = [f_L^a, f_U^a] = \left[b + \sum_{i=1}^{n} \min(w_i l_i, w_i u_i), ; b + \sum_{i=1}^{n} \max(w_i l_i, w_i u_i) \right]. \tag{67}$$

Example 5.4.1.1 (Affine Transformer)

Consider the network in Fig. 1 with inputs $x_1 \in [1, 2]$ and $x_2 \in [-1, 3]$. The affine function for the neuron x_3 is:

$$f(x_1, x_2) = -0.5x_1 + 0.5x_2 + 1.0 \tag{68}$$

Then the interval for x_3 can be computed as:

$$\begin{aligned} f^a([1, 2], [-1, 3]) &= [1 + \min(-0.5 \cdot 1, -0.5 \cdot 2) + \min(0.5 \cdot (-1), 0.5 \cdot 3), \\ &\quad 1 + \max(-0.5 \cdot 1, -0.5 \cdot 2) + \max(0.5 \cdot (-1), 0.5 \cdot 3)] \\ &= [1 - 1.0 - 0.5, 1 - 0.5 + 1.5] \\ &= [-0.5, 2.0] \end{aligned} \tag{69}$$

◊

5.4.2 ReLU Transformer

For $\text{relu}(x) = \max(0, x)$, the abstract transformer is defined as:

$$\text{relu}^a([l, u]) = [\text{relu}(l), \text{relu}(u)] = [\max(0, l), \max(0, u)] \tag{70}$$

This is equivalent to three cases shown in Fig. 25:

1. If $u < 0$, then $\text{relu}^a([l, u]) = [0, 0]$. If inputs are negative, the output is also negative.
2. If $l \geq 0$, then $\text{relu}^a([l, u]) = [l, u]$. If inputs are positive, the output is exactly the same.

3. If $l < 0 < u$, then $\text{relu}^a([l, u]) = [0, u]$. If inputs are mixed, the output is approximated to $[0, u]$.

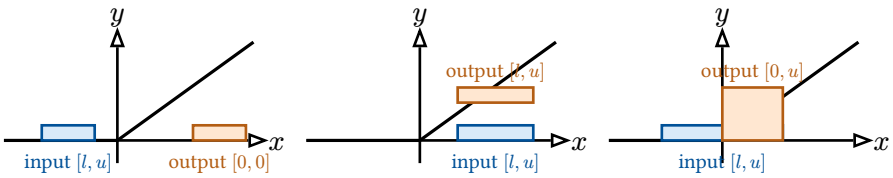


Fig. 25: ReLU transformer for intervals. Left: all inputs negative ⇒ $[0, 0]$. Middle: all inputs positive ⇒ unchanged. Right: interval crosses zero ⇒ $[0, u]$. The ReLU function clamps negative parts to zero while preserving positive regions.

Example 5.4.2.1 (ReLU Interval)

For Example 5.4.1.1, applying relu^a to neuron x_3 gives:

$$\text{relu}^a([-0.5, 2.0]) = [\text{relu}(-0.5), \text{relu}(2.0)] = [0, 2.0]. \tag{71}$$

◊

Problem 5.4.2.2 (Interval Abstraction)

Apply interval abstraction to the network in Fig. 1 with inputs $x_1 \in [1, 2]$ and $x_2 \in [-1, 3]$. Compute the bounds for all neurons. Clearly indicate the steps (e.g., results after affine and ReLU transformers).

◊

5.4.3 Efficiency and Precision

Interval is very efficient and scales well to large networks. The affine transformer only requires a linear number of multiplications and min/max operations, and ReLU reduces to only three matching cases. However, the cost of efficiency is *precision*.

Example 5.4.3.1 (Interval Overapproximation)

Suppose we have $v_1 \in [0, 1]$, $v_2 = -v_1$, and $z = v_1 + v_2$. The concrete value of z would always be 0, but interval abstraction gives $z \in [-1, 1]$, which is a very loose over-approximation. Moreover, if we apply ReLU, then the output would be $\text{relu}(z) = \text{relu}(0) = 0$, but the interval gives $\text{relu}^a([-1, 1]) = [0, 1]$, which is again a loose over-approximation.

◊

Interval overapproximation grows quickly as we propagate through many layers of a large network, i.e., it keeps "inflating" the bounds, leading to a loose approximation of the output and becoming unable to verify valid properties. For example, $z \leq 0$ is a valid property for Example 5.4.3.1, but the interval abstraction gives $z \in [0, 1]$, which is not tight (precise) enough to show this property.

Nonetheless, interval domain remains popular due to its simplicity and efficiency. In some cases, despite being imprecise, it can still successfully verify properties of neural networks, e.g., for Example 5.4.3.1 if we want to verify that $z \leq 2$, then the interval $z \in [0, 1]$ would suffice.

Problem 5.4.3.2 (Correct Abstraction)

Suppose y can take values from 0 to 3. Student A computes an interval abstraction $y \in [-2, 4]$, while student B computes $y \in [1, 2]$.

1. Which student's abstraction is correct? If both are correct, which is more precise? Explain your answer.
2. Using the correct abstraction (if both are correct, use the more precise one), can we use it to verify the property $y \leq 3$? How about $y \leq 5$? Explain your answers.

◊

5.5 Zonotope

Interval abstraction (Sec. 5.4) treats each variable independently. For example, if $x_1 \in [1, 2]$ and $x_2 \in [3, 4]$, interval assumes any combination of x_1 and x_2 is possible. But variables can correlate: e.g., when x_1 increases, x_2 also increases. Zonotopes can capture such correlations and therefore provide a tighter abstraction.

Definition.

A zonotope is often represented using a generator-based representation as illustrated in Sec. 5.2. It is built by starting from a center point and then stretching along several directions defined by generator vectors. Each generator can move forward or backward within a certain range, and together these movements sweep out the entire shape. Zonotope generalizes the idea of an interval, which itself is a zonotope (e.g., see Fig. 24).

Formally, a **zonotope** $\mathcal{Z}$ in $\mathbb{R}^n$ is defined as:

$$\mathcal{Z} = \left\{ c + \sum_{i=1}^{n} \varepsilon_i g_i \mid \varepsilon_i \in [-1, 1] \right\} \tag{72}$$

where:

- $c \in \mathbb{R}^n$ is the *center* (analogous to the midpoint of an interval),
- $g_i \in \mathbb{R}^n$ are *generator vectors* (directions of variability), and
- $\varepsilon_i \in [-1, 1]$ are independent coefficients that determine how much we move along each generator. For example, $\varepsilon_i = -1$ moves in the negative direction, $\varepsilon_i = 0$ stays at the center, and $\varepsilon_i = +1$ moves in the positive direction.

Independent ⇒ Rectangle Correlated ⇒ Parallelogram

Fig. 26: Left: independent generators produce an axis-aligned rectangle (interval abstraction). Right: correlated generators tilt the shape into a parallelogram, capturing dependency between x_1 and x_2.

Example 5.5.1 (Zonotope Parallelogram)

A common zonotope in 2D is a parallelogram. It can be defined by its center point and two generator vectors that define its sides. For instance, the parallelogram on the right side of Fig. 26 has the center $c = (1, 1)$ and the generators $g_1 = (1, 0.25)$ and $g_2 = (0.5, 1)$. This zonotope represents all points that can be reached by moving along these generators within the range $[-1, 1]$:

$$\mathcal{Z} = \{(1, 1) + \varepsilon_1(1, 0.25) + \varepsilon_2(0.5, 1) \mid \varepsilon_1, \varepsilon_2 \in [-1, 1]\} \tag{73}$$

This zonotope includes points like:

$$(1, 1) + (-1)(1, 0.25) + (1)(0.5, 1) = (-1, 2.5) \tag{74}$$

$$(1, 1) + (1)(1, 0.25) + (-1)(0.5, 1) = (3, -0.5) \tag{75}$$

◊

Example 5.5.2 (Correlated vs Independent Generators)

In Fig. 26 the parallelogram zonotope on the right captures points with correlated x_1 and x_2 values, e.g., increasing x_1 by moving along g_1 also increases x_2 due to the non-zero second component of g_1. In contrast, the rectangle zonotope on the left, which represents an interval abstraction, has independent generators, i.e., changing x_1 does not affect x_2, and therefore cannot capture the correlation.

◊

5.5.1 Computing Bounds of a Zonotope

In NNV, we often need to compute the bounds of each variable represented by a zonotope. This allows us to determine stable neurons, which in turn helps optimize the verification process.

For each coordinate (dimension) of the zonotope, we compute its lower and upper bounds as follows:

$$l_j = \min_{\varepsilon_i \in [-1,1]} \left(c_j + \sum_{i=1}^{n} \varepsilon_i g_{i,j} \right), \quad u_j = \max_{\varepsilon_i \in [-1,1]} \left(c_j + \sum_{i=1}^{n} \varepsilon_i g_{i,j} \right), \tag{76}$$

where c_j is the j-th component of the center and $g_{i,j}$ is the j-th component of the i-th generator.

A simpler way. Since each $\varepsilon_i \in [-1, 1]$, each term $\varepsilon_i g_{i,j}$ can vary between $-|g_{i,j}|$ and $+|g_{i,j}|$. Hence:

$$l_j = c_j - \sum_{i=1}^{n} |g_{i,j}|, \quad u_j = c_j + \sum_{i=1}^{n} |g_{i,j}|. \tag{77}$$

Example (1D). Given the interval zonotope:

$$x = 3 + \varepsilon_1(2), \quad \varepsilon_1 \in [-1, 1], \tag{78}$$

the bounds are computed as:

$$[l, u] = [3 - |2|, 3 + |2|] = [1, 5]. \tag{79}$$

Example (2D).

$$\mathcal{Z} = \left\{ \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \varepsilon_1 \begin{pmatrix} 0.5 \\ 1.0 \end{pmatrix} + \varepsilon_2 \begin{pmatrix} -0.3 \\ 0.2 \end{pmatrix}, \quad \varepsilon_1, \varepsilon_2 \in [-1, 1] \right\}. \tag{80}$$

Then for each coordinate:

$$\begin{aligned} l_1 &= 1 - (|0.5| + |-0.3|) = 0.2, \quad & u_1 &= 1 + (|0.5| + |-0.3|) = 1.2, \\ l_2 &= 2 - (|1.0| + |0.2|) = 0.8, & u_2 &= 2 + (|1.0| + |0.2|) = 3.2. \end{aligned} \tag{81}$$

5.5.1.1 Affine Transformer

Recall that the affine function f in Sec. 1.3 is

$$f(v_1, v_2, ..., v_n) = \sum_{i=1}^{n} w_i v_i + b, \tag{82}$$

where w_i is the weight associated with input v_i, n is the number of inputs, and b is the bias term.

More formally, given a zonotope

$$\mathcal{Z} = \left\{ c + \sum_{j=1}^{m} \varepsilon_j g_j \mid \varepsilon_j \in [-1, 1] \right\}, \tag{83}$$

the abstract transformer f^a for the affine function f is:

$$\begin{aligned} f^a(v_1, v_2, ..., v_n) &= \sum_{i=1}^{n} w_i v_i + b \\ &= \sum_{i=1}^{n} w_i \underbrace{\left(c_i + \sum_{j=1}^{m} \varepsilon_j g_{ij} \right)}_{v_i} + b \\ &= \sum_{i=1}^{n} w_i c_i + \sum_{i=1}^{n} w_i \sum_{j=1}^{m} \varepsilon_j g_{ij} + b \\ &= \underbrace{\left(\sum_{i=1}^{n} w_i c_i + b \right)}_{\text{new center}} + \sum_{j=1}^{m} \varepsilon_j \underbrace{\left(\sum_{i=1}^{n} w_i g_{ij} \right)}_{\text{new generators}} \end{aligned} \tag{84}$$

That is, the new zonotope has:

- **center:** $f^c(c) = \sum_{i=1}^{n} w_i c_i + b$
- **generators:** each generator g_j becomes $f^g(g_j) = \sum_{i=1}^{n} w_i g_{ij}$

Applying an affine transformation to a zonotope transforms its center by the same affine rule (weights and bias) and transforms each generator by the linear part (weights only). In other words, affine operations *preserve* the structure of zonotopes exactly: no approximation needed.

$$f^a(\mathcal{Z}) = \left\{ f^c(c) + \sum_{j=1}^{m} \varepsilon_j f^g(g_j) \mid \varepsilon_j \in [-1, 1] \right\} \tag{85}$$

Example 5.5.1.1 (Zonotope Affine Transformer)

Consider the network in Fig. 1 with inputs $x_1 \in [1, 2]$ and $x_2 \in [-1, 3]$. The affine function for neuron x_3 is:

$$f(x_1, x_2) = -0.5x_1 + 0.5x_2 + 1.0 \tag{86}$$

We want to compute the bounds for x_3 using zonotope abstraction.

Represent the input as a zonotope. We represent the input intervals $x_1 \in [1, 2]$ and $x_2 \in [-1, 3]$ as a zonotope $\left\{c + \sum_{i=1}^{n} \varepsilon_i g_i \mid \varepsilon_i \in [-1, 1]\right\}$.

The center c_i represents the midpoint of each input interval:

$$c_1 = \frac{1+2}{2} = 1.5, \quad c_2 = \frac{-1+3}{2} = 1.0, \tag{87}$$

and the generator g_i captures the half-width of each interval:

$$g_1 = \frac{2-1}{2} = 0.5, \quad g_2 = \frac{3-(-1)}{2} = 2.0 \tag{88}$$

Thus, the input zonotope is

$$\mathcal{Z} = \{(1.5, 1.0) + \varepsilon_1(0.5, 0) + \varepsilon_2(0, 2.0), \quad \varepsilon_1, \varepsilon_2 \in [-1, 1]\}. \tag{89}$$

Apply the affine transformation. We apply $f(x_1, x_2) = -0.5x_1 + 0.5x_2 + 1.0$ to $\mathcal{Z}$:

$$\begin{aligned} f^a(\mathcal{Z}) &= f^c(c_1, c_2) + \varepsilon_1 f^g(g_1) + \varepsilon_2 f^g(g_2) \\ &= (-0.5c_1 + 0.5c_2 + 1.0) + \varepsilon_1(-0.5g_{1x} + 0.5g_{1y}) + \varepsilon_2(-0.5g_{2x} + 0.5g_{2y}) \\ &= (-0.5(1.5) + 0.5(1.0) + 1.0) + \varepsilon_1(-0.5(0.5) + 0.5(0)) + \varepsilon_2(-0.5(0) + 0.5(2.0)) \\ &= 0.75 - 0.25\varepsilon_1 + 1.0\varepsilon_2. \end{aligned} \tag{90}$$

Hence, the output zonotope for x_3 is

$$\{0.75 + \varepsilon_1(-0.25) + \varepsilon_2(1.0), \quad \varepsilon_1, \varepsilon_2 \in [-1, 1]\}. \tag{91}$$

Compute output bounds. From the output zonotope, the bounds for x_3 are:

$$\begin{aligned} f_L^a &= c' - (|g_1'| + |g_2'|) = 0.75 - (0.25 + 1.0) = -0.5, \\ f_U^a &= c' + (|g_1'| + |g_2'|) = 0.75 + (0.25 + 1.0) = 2.0. \end{aligned} \tag{92}$$

Thus, the output range of x_3 is $[-0.5, 2.0]$.

Note that the reason for computing the bounds from the zonotope representation is to determine *stable neurons* as described in Sec. 4.2.2. If the lower bound is non-negative, ReLU will be active; if the upper bound is non-positive, ReLU will be inactive. In these cases, ReLU can be simplified to either the identity function or zero, respectively, which helps reduce overapproximation in subsequent layers.

For this simple example, both interval (Sec. 5.4.1) and zonotope abstractions yield identical bounds $[-0.5, 2.0]$. The benefit of zonotopes becomes clear in deeper layers, where they preserve linear correlations that interval abstractions lose.

◊

Problem 5.5.1.2 (Zonotope Abstraction for x_4)

Compute the bounds for x_4 in Fig. 1 using zonotope abstraction, with inputs $x_1 \in [1, 2]$ and $x_2 \in [-1, 3]$, and the affine function for x_4 is:

$$f(x_1, x_2) = 0.5x_1 - 0.5x_2 - 1 \tag{93}$$

◊

5.5.1.2 ReLU Transformer

Activation functions like ReLU are non-affine and do not preserve zonotope structure. For $\text{relu}(x) = \max(0, x)$, the shape of the input region changes at $x = 0$: $x < 0$ maps to 0 (a flat line), while $x \geq 0$ maps to x (a slanted line). Thus, we over-approximate these shapes with a new zonotope — a parallelogram — that contains the entire ReLU output region.

We can distinguish three possible cases of ReLU depending on the bounds $[l, u]$ of the input zonotope:

- **Active case** ($l \geq 0$): All values are non-negative, so $\text{relu}^a(x) = x$ (identity function), i.e., the output zonotope is the same as the input zonotope.
- **Inactive case** ($u \leq 0$): All values are negative, so $\text{relu}^a(x) = \{0\}$, i.e., the output zonotope is a single point at zero.
- **Unstable case** ($l < 0 < u$): The range crosses zero, requiring over-approximation using a parallelogram that bounds ReLU over $[l, u]$ as described below.

Building parallelogram. Given the input zonotope $x = \left\{c + \sum_{i=1}^{n} \varepsilon_i g_i \mid \varepsilon_i \in [-1, 1]\right\}$ with x ranging over $[l, u]$ where $l < 0 < u$ (i.e., the unstable case), we construct a parallelogram that bounds ReLU over this interval using two linear constraints representing its upper and lower edges:

- **Upper bound constraint:** $y \leq \lambda x - \lambda l$, where $\lambda = \frac{u}{u-l}$ is the slope of the line connecting points $(l, 0)$ and (u, u).
- **Lower bound constraint:** $y \geq \lambda x$ is the line with the same slope λ passing through $(0, 0)$.

Merging these two constraints, we get the following equation describing all points (x, y) within the parallelogram:

$$\begin{aligned} \lambda x \leq \quad & y \quad \leq \lambda x - \lambda l \\ \Leftrightarrow \quad & y = \lambda x - \alpha\lambda l, \quad \alpha \in [0, 1] \end{aligned} \tag{94}$$

The parameter α interpolates between the lower and upper bounds of the parallelogram. When $\alpha = 0$, we are on the lower bound $y = \lambda x$; when $\alpha = 1$, we are on the upper bound $y = \lambda x - \lambda l$.

Our zonotope requires $\varepsilon_i \in [-1, 1]$, thus we substitute:

$$\alpha = \frac{1 + \varepsilon_{\text{new}}}{2}, \quad \varepsilon_{\text{new}} \in [-1, 1] \tag{95}$$

For example, when $\varepsilon_{\text{new}} = -1$, we have $\alpha = 0$ (lower bound); when $\varepsilon_{\text{new}} = 1$, we have $\alpha = 1$ (upper bound).

The output zonotope after ReLU transformation is then:

$$
\begin{aligned}
y &= \lambda x - \alpha\lambda l \\
&= \lambda x - \frac{1+\varepsilon_{\text{new}}}{2} \cdot \lambda l \\
&= \lambda \underbrace{\left(c + \sum_{i=1}^{m} \varepsilon_i g_i \right)}_{x} - \varepsilon_{\text{new}} \frac{\lambda l}{2} - \frac{\lambda l}{2} \\
&= \underbrace{\left(\lambda c - \frac{\lambda l}{2} \right)}_{\text{new center}} + \underbrace{\sum_{i=1}^{m} \varepsilon_i (\lambda g_i) - \varepsilon_{\text{new}} \frac{\lambda l}{2}}_{\text{new generators}}
\end{aligned}
\tag{96}
$$

Example 5.5.1.3 (ReLU on Zonotope)

Continuing from Example 5.5.1.1, applying ReLU to the zonotope output of x_3:

$$
x_3 = 0.75 + \varepsilon_1(-0.25) + \varepsilon_2(1.0), \quad \varepsilon_1, \varepsilon_2 \in [-1, 1]. \tag{97}
$$

Determine the bounds. As shown in Example 5.5.1.1, the output zonotope (before ReLU) has bounds $[-0.5, 2.0]$. Since $l = -0.5 < 0 < 2.0 = u$, this is an unstable neuron requiring over-approximation.

Approximate slope. We compute the slope of the upper bound line of the parallelogram (connecting points $(-0.5, 0)$ and $(2.0, 2.0)$):

$$
\lambda = \frac{u - 0}{u - l} = \frac{2.0}{2.0 - (-0.5)} = \frac{2.0}{2.5} = 0.8. \tag{98}
$$

Transform zonotope. The new center becomes:

$$
c' = \lambda c - \frac{\lambda l}{2} = 0.8 \cdot 0.75 - \frac{0.8 \cdot (-0.5)}{2} = 0.6 + 0.2 = 0.8 \tag{99}
$$

Existing generators are scaled by λ:

$$
\begin{aligned}
g_1' &= \lambda g_1 = 0.8 \cdot (-0.25) = -0.2, \\
g_2' &= \lambda g_2 = 0.8 \cdot 1.0 = 0.8
\end{aligned}
\tag{100}
$$

New generator for approximation error. A new generator is introduced to capture the ReLU approximation error:

$$
g_3' = -\frac{\lambda l}{2} = -\frac{0.8 \cdot (-0.5)}{2} = 0.2 \tag{101}
$$

Result: The output zonotope is

$$
0.8 + \varepsilon_1(-0.2) + \varepsilon_2(0.8) + \varepsilon_3(0.2), \quad \varepsilon_i \in [-1, 1], \tag{102}
$$

with concrete bounds:

$$
\begin{aligned}
l' &= 0.8 - 0.2 - 0.8 - 0.2 = -0.4 \to 0 \quad \text{(clamp negative values)} \\
u' &= 0.8 + 0.2 + 0.8 + 0.2 = 2.0
\end{aligned}
\tag{103}
$$

Thus, the final bounds after ReLU are $[0, 2.0]$.

◊

6 The Branch and Bound Search Algorithm

As shown in Sec. 4, NNV can be formulated as a satisfiability problem, solvable using a constraint solver, e.g., SMT and MILP solvers. However, such solvers do not scale to large networks due to the complexity of the underlying formulae. Thus, modern NNV techniques reframe the problem to search for *activation patterns* that satisfy the constraints, and use the *Branch-and-Bound* algorithm to explore the space of possible activation patterns.

6.1 Activation Pattern Search

Neural networks with piecewise linear activation functions have a special structure that can be exploited for verification. For example, each ReLU function has two *activation statuses*: active and inactive. Each status partitions the input space into two regions—one where the neuron is active and one where it is inactive. Within each region, the network behavior can be encoded as *a linear constraint*, which can be efficiently analyzed.

Thus, the NNV problem reduces to checking that none of these linear regions contains a counterexample. More specifically, we can rewrite the satisfiability formulation as a disjunction:

$$\bigvee_{p \in P} (\alpha_p \wedge \varphi_{\text{in}} \wedge \neg \varphi_{\text{out}}) \tag{104}$$

where P is the set of all possible *activation patterns*—boolean assignments of activation statuses of all neurons—of the network, and α_p is the formula α restricted to the linear region defined by the activation pattern p. This means that α_p includes additional constraints that fix the activation status of each neuron according to p. For example, if p specifies that neuron n_i is active, then α_p includes the constraint $z_i \geq 0$, indicating that the pre-activation value z_i of n_i is non-negative. Similarly, if n_i is inactive, then α_p includes $z_i < 0$.

As long as one of the disjuncts in Eq. 104 is satisfiable, i.e., a counterexample exists in one of the linear regions, the entire formula is satisfiable, indicating that the property is invalid. Conversely, if all disjuncts are unsatisfiable, the property is valid because no counterexample exists.

This allows us to break down the verification problem into smaller subproblems, each searching for an activation pattern that satisfies the formula. Modern NNV techniques all adopt this idea and search for a satisfying activation pattern to find a counterexample.

6.1.1 Activation Patterns

Definition 6.1.1.1 (Activation Pattern)

Let N be a neural network with ReLU neurons $n_1, ..., n_m$. An *activation pattern* is a Boolean assignment of the activation status of a subset (or all) of the neurons. If a neuron n_i is active, we set $b_i =$ `true` (meaning $z_i \geq 0$); if it is inactive, $b_i =$ `false` (meaning $z_i < 0$).

Definition 6.1.1.2 (Complete Activation Pattern)

A *complete activation pattern* assigns an activation status to **every** neuron in the network:

$$p = \langle b_1, b_2, ..., b_m \rangle \in \{ \texttt{true}, \texttt{false} \}^m. \tag{105}$$

♣

Definition 6.1.1.3 (Partial Activation Pattern)

A *partial activation pattern* assigns activation statuses to only a **subset** of the neurons:

$$q = \langle b_1, b_2, *, b_4, *, ..., b_m \rangle \in \{ \texttt{true}, \texttt{false}, * \}^m, \tag{106}$$

where $*$ means "undetermined" or "don't care". Each partial activation pattern represents multiple complete activation patterns.

♣

Example 6.1.1.4

Consider a network with three ReLU neurons n_1, n_2, n_3. A complete activation pattern might be

$$p = \langle \texttt{true}, \texttt{false}, \texttt{true} \rangle, \tag{107}$$

which means n_1 and n_3 are active while n_2 is inactive.

In contrast, a partial activation pattern such as

$$q = \langle \texttt{true}, \texttt{false}, * \rangle \tag{108}$$

represents **both** complete patterns $\langle \texttt{true}, \texttt{false}, \texttt{true} \rangle$ and $\langle \texttt{true}, \texttt{false}, \texttt{false} \rangle$.

◊

Definition 6.1.1.5 (Empty Activation Pattern)

An *empty* activation pattern is a partial pattern that assigns **no** activation statuses to any neurons:

$$p = \langle *, *, ..., * \rangle \in \{ \texttt{true}, \texttt{false}, * \}^m. \tag{109}$$

It thus represents all 2^m complete activation patterns.

♣

Problem 6.1.1.6

Consider an NN with 3 ReLU neurons:

1. How many complete activation patterns are there?
2. How many partial patterns of size 2 (i.e., fixing 2 neurons but leaving 1 unspecified) are there?
3. Give an explicit example of one partial pattern and list all the complete patterns it represents.

Solution.

1. $2^3 = 8$ complete patterns.
2. Choose 2 neurons to fix: $\binom{3}{2} = 3$ ways. For each, $2^2 = 4$ assignments. Total $= 12$ partial patterns.
3. Example: $p' = \{n_1 = \mathsf{true}, n_3 = \mathsf{false}\}$ represents

$\{n_1 = \mathsf{true}, n_2 = \mathsf{true}, n_3 = \mathsf{false}\}$ and $\{n_1 = \mathsf{true}, n_2 = \mathsf{false}, n_3 = \mathsf{false}\}$.

◊

Problem 6.1.1.7

Suppose we have a network with $m = 10$ ReLU neurons. Consider a partial pattern p' that fixes 4 neurons.

1. How many complete patterns extend p'?
2. Suppose we restrict p' further by adding 2 more neuron assignments. How many extensions now?
3. Generalize: if p' fixes k neurons out of m, how many complete patterns are there?

Solution.

1. $2^{10-4} = 2^6 = 64$.
2. $2^{10-6} = 2^4 = 16$.
3. General: 2^{m-k}.

◊

6.1.2 Set Notation of Activation Patterns

We can use *set notation* to represent activation patterns more concisely by listing only the neurons whose activation statuses are fixed. For example, the activation pattern

$$p = \langle \texttt{true}, \texttt{false}, *, \texttt{true}, \texttt{false} \rangle \tag{110}$$

can be represented as

$$p = \{n_1 = \texttt{true}, n_2 = \texttt{false}, n_3 = *, n_4 = \texttt{true}, n_5 = \texttt{false}\}, \tag{111}$$

or more compactly

$$p = \{n_1, \overline{n_2}, n_4, \overline{n_5}\}, \tag{112}$$

where n_i indicates that neuron n_i is active, $\overline{n_i}$ indicates that it is inactive, and the absence of n_3 indicates that its activation status is unspecified.

Thus, given a set of neurons, we can construct an activation pattern by specifying which neurons are active and which are inactive. If the set includes all neurons, it is a complete pattern; otherwise, it is a partial pattern (and if it is an empty set $\emptyset$, it is the empty pattern as in Definition 6.1.1.5).

6.1.3 State Space Reduction

Once having an activation pattern p, we can simplify the formula α representing the network by replacing each ReLU function with a linear constraint according to the activation status specified in p. For example, if p specifies that neuron n_i is active, we replace $\mathrm{relu}(z_i)$ with $z_i \geq 0$. Otherwise, if n_i is inactive, we replace $\mathrm{relu}(z_i)$ with $z_i < 0$. This gives us the formula α_p corresponding to the linear region defined by p.

A complete activation pattern p defines a unique linear region of the network because with a complete pattern, α_p becomes a purely linear formula. In contrast, a partial activation pattern q defines multiple linear regions, one for each complete pattern it represents. With q, α_q may still contain some ReLU functions that are not fixed by q. However, since q fixes the status of some neurons, we can simplify α by replacing those neurons' ReLU functions with linear constraints.

In any case, given an activation pattern, we can reduce the complexity of the satisfiability check by fixing the activation status of some neurons. Thus, checking the satisfiability of $\alpha_p \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$ is easier than checking that of $\alpha \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$.

Example 6.1.3.1

Recall the network from Fig. 12 can be represented as the formula α

$$\begin{aligned} \hat{x}_3 &= -0.5x_1 + 0.5x_2 + 1.0 \\ \hat{x}_4 &= 0.5x_1 - 0.5x_2 - 1.0 \\ x_3 &= \text{relu}(\hat{x}_3) \\ x_4 &= \text{relu}(\hat{x}_4) \\ x_5 &= -x_3 + x_4 - 1.0, \end{aligned} \tag{113}$$

where $\hat{x}_3$ and $\hat{x}_4$ are the pre-activation values of the ReLU neurons x_3 and x_4, respectively. Thus, if we fix the the activation status of x_3 and x_4, we can simplify α by replacing the ReLUs with linear constraints.

- A **complete** activation pattern specifies the status of both ReLU neurons. For instance,

  $$p = \{x_3, \overline{x_4}\} \tag{114}$$

  means x_3 is active while x_4 is inactive; that is, $\hat{x}_3 \geq 0$ (so $x_3 = \hat{x}_3$) while $\hat{x}_4 < 0$ (so $x_4 = 0$). In this case, α reduces to a single linear constraint:

  $$x_5 = -(-0.5x_1 + 0.5x_2 + 1.0) + 0 - 1.0 = 0.5x_1 - 0.5x_2 - 2.0. \tag{115}$$

  This reduction significantly simplifies the satisfiability check: we no longer have to reason about the non-linearity of ReLU, and can directly check whether the linear constraint is satisfiable with the input and output constraints.

- A **partial** pattern specifies only some activation statuses. For example,

  $$q = \{x_3\} \tag{116}$$

  means $\hat{x}_3 \geq 0$ but places no restriction on $\hat{x}_4$. Here, α reduces to:

  $$\begin{aligned} \hat{x}_4 &= 0.5x_1 - 0.5x_2 - 1.0 \\ x_4 &= \text{relu}(\hat{x}_4) \\ x_5 &= -(-0.5x_1 + 0.5x_2 + 1.0) + x_4 - 1.0. \end{aligned} \tag{117}$$

  Because q does not fix the status of x_4, we cannot simplify the ReLU for x_4. Thus, the formula still contains a ReLU which splits x_4 into two linear regions. Note that this is still simpler than the original α because we have simplified the ReLU for x_3.

◊

Problem 6.1.3.2

Consider the network in Fig. 12. Suppose we fix the activation pattern $p = \{x_3, x_4\}$.

1. Provide the constraints α induced by p.
2. Consider an invalid property $x_5 > 0$ for inputs $x_1 \in [-1, 1]$, $x_2 \in [-2, 2]$. Find an activation pattern that satisfies the formula $\alpha_p \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$. In other words, find a pattern that contains a counterexample to the property.

◊

1 **Input**: network $\mathcal{N}$, property $\varphi_{\text{in}} \Rightarrow \varphi_{\text{out}}$
2 **Output**: unsat if property is valid, otherwise (sat, cex)

3 ActPatterns $\leftarrow \{\emptyset\}$ //initialize verification problem
4 **while** ActPatterns $\neq \emptyset$
5 $\sigma_i \leftarrow$ Select(ActPatterns) // process problem i^{th}
6 **if** Deduce($\mathcal{N}, \varphi_{\text{in}} \Rightarrow \varphi_{\text{out}}, \sigma_i$) // check satisfiability
7 **if** (found a valid cex)
8 **return** (sat, cex)
9 $v_i \leftarrow$ Decide($\mathcal{N}, \sigma_i$)
  // create new activation pattern
10 ActPatterns $\leftarrow$ ActPatterns $\cup \{\sigma_i \cup \{v_i\}; \sigma_i \cup \{\neg v_i\}\}$
11 **return** "unsat"

Algorithm 1: Branch and Bound (BaB) Algorithm for Neural Network Verification

6.2 The Branch and Bound Algorithm

Many modern NNV tools adopt the Branch-and-Bound (BaB) approach to explore the space of possible neuron activation patterns (Sec. 6.1). BaB splits (*branch*) the verification problem into smaller subproblems by fixing activation statuses of neurons, and uses abstraction (*bound*) techniques to compute upper and lower bounds on the output values for these subproblems. If the bounds, computed using abstraction (Sec. 5), indicate infeasible (cannot contain a counterexample), the subproblem is pruned from the search space. This process continues until either a counterexample is found or all subproblems are exhausted, proving the property holds.

Note that because BaB splits ReLU neurons into active and inactive cases, it is also called *"neuron-splitting"*, which contrasts with *"input-splitting"* techniques (Sec. 9.1) that partition the input space.

Reference BaB Algorithm Algorithm 1 shows BaB, a reference [14] Branch and Bound architecture for NNV. BaB takes as input a ReLU-based network $\mathcal{N}$ and a formulae $\varphi_{\text{in}} \Rightarrow \varphi_{\text{out}}$ representing the property of interest. BaB maintains a set of activaion patterns (ActPatterns) that represent the current activation pattern of the network. Initially, ActPatterns is initialized with an empty activation pattern (Line 3).

In each BaB iteration i (Line 4), the algorithm selects and removes an activation pattern σ_i from ActPatternss (Line 5). It then calls Deduce to *quickly* determine if the current problem, i.e., the original satisfiability problem with activation pattern σ_i, is feasible. For example, it can use interval abstraction (Sec. 5) to quickly computes the bounds of the output values with respect to σ_i, e.g., by replacing ReLU functions according to the activation statuses specified in σ_i (Sec. 6.1.3).

Deduce vs. LP The difference between Deduce and LP is that the former uses abstraction to compute over-approximated bounds to determine feasibility, while the latter uses an LP solver

to check exact satisfiability. Abstraction is (very) quick but may yield false positives (declaring feasible when it is not), while LP is precise but more computationally expensive. Their combination allows BaB to efficiently prune infeasible branches while accurately checking for counterexamples.

Example 6.2.1 (BaB Illustration)

Consider verifying that a network has the property

$$(x_1, x_2) \in [-2.0, 2.0] \times [-1.0, 1.0] \Rightarrow (y_1 > y_2), \tag{118}$$

i.e., no counterexample exists where $y_1 \leq y_2$ for inputs in the given range.

First, BaB initializes ActPatterns with an empty activation pattern $\emptyset$ (i.e., no neurons fixed). Then BaB enters its main loop.

1. **First iteration.** BaB selects the only available activation pattern $\emptyset$, i.e., $\sigma_i = \emptyset$, and calls *Deduce* to check the feasibility of the problem.

   Here, *Deduce* determines that the problem is feasible — the computed bounds of y_1 and y_2 indicate that $y_1 \leq y_2$ can be satisfied. Next, BaB calls an LP solver to check satisfiability. The LP solver returns unknown — the problem is too complex without fixing any neuron status. Thus we need to refine by *splitting* a neuron.

   BaB selects a neuron to split (*Decide*). Assume it chooses v_4, spawning two independent subproblems: one with v_4 active and one with v_4 inactive. Both activation patterns $\{v_4\}$ and $\{\overline{v_4}\}$ are added to ActPatterns.

2. **Second iteration.** For the subproblem with pattern $\{v_4\}$, *Deduce* finds it feasible but the LP solver is inconclusive, so v_2 is split, adding $\{v_4, v_2\}$ and $\{v_4, \overline{v_2}\}$. For the subproblem with $\overline{v_4}$, *Deduce* determines infeasibility and prunes this branch.

3. **Third iteration.** For pattern $\{v_4, v_2\}$, BaB splits v_1, adding $\{v_4, v_2, v_1\}$ and $\{v_4, v_2, \overline{v_1}\}$. For pattern $\{v_4, \overline{v_2}\}$, *Deduce* determines infeasibility; pruned.

4. **Fourth iteration.** Both remaining patterns $\{v_4, v_2, v_1\}$ and $\{v_4, v_2, \overline{v_1}\}$ are determined infeasible and pruned.

5. **Fifth iteration.** ActPatterns is empty; BaB returns UNSAT, proving the property holds.

Notice that while there are $2^4 = 16$ possible activation patterns for the four ReLU neurons, BaB only explores 6 of them — infeasibility pruning avoids exploring the rest. ◊

Problem 6.2.2 (BaB Detailed Steps)

Do Example 6.2.1 but come up with concrete values for each step, e.g., what are the bounds computed by *Deduce*, what does the LP solver return, etc., to illustrate the process in more detail. ◊

Problem 6.2.3 (BaB Counterexample)

Do Example 6.2.1 again but make the property invalid and find a counterexample. It is also OK to change the problem (e.g., the input bounds or the property itself) to make it invalid.

◊

Problem 6.2.4 (BaB with Interval Abstraction)

Apply BAB on the network in Fig. 1 to verify the property $x_5 > 0$ for any inputs $x_1 \in [-1, 1], x_2 \in [-2, 2]$. Use interval abstraction to compute bounds. For LP solving, use any LP solver or solve manually. Show all steps: iterations, activation patterns, bounds computed, LP results, etc.

◊

Problem 6.2.5 (BaB Understanding)

Test your understanding of BAB and activation patterns by answering the following questions:

1. In the BAB algorithm, what happens if *Deduce* always returns infeasible?
2. What happens if *Deduce* always returns feasible but the LP solver always returns satisfiable?
3. What is the maximum number of activation patterns that BAB may need to explore in the worst case for a network with m ReLU neurons?
4. What is the minimum number of activation patterns that BAB may need to explore in the best case for a network with m ReLU neurons? What are these patterns?

◊

6.2.1 Beyond the Basic

What is described above is the basic and minimal BAB algorithm. Modern NNV tools implement many optimizations and strategies to improve the performance of the search. For example, they apply various engineering tricks to eliminate easy cases or find easy counterexamples (Sec. 7) before running the full BAB search.

Even the BAB search itself is optimized to avoid exploring the entire search space. For example, if the *Deduce* step determines infeasibility, a smarter BAB variant (e.g., NEURALSAT) can analyze the conflict and add a new clause to prevent the same activation pattern from being selected again. This is similar to conflict-driven clause learning (CDCL) in modern SAT solvers and is implemented in NEURALSAT [15]. In addition, heuristics are used to select, e.g., which neuron to split next (*Decide*). We explore these optimizations and strategies in later chapters (e.g., Sec. 10).

7 Adversarial Attacks

A full branch and bound (BaB) search (Sec. 6) is typically expensive and slow on large networks. Thus most NNV tools implement optimizations to improve performance. A common optimization is to use *adversarial attack* techniques to find counterexamples before running BaB. Examples of such techniques include randomized attacks [16] and gradient-based attacks [17].

Modern verification tools have two phases: (i) attempt to *falsify* the property with adversarial attacks, and (ii) if no counterexample is found, attempt to *verify* the property using search algorithms such as BaB (Sec. 6). Of course, during the verification phase, the verifier may also discover counterexamples that the falsify phase misses.

7.1 Random Search Attack

Random search is a simple adversarial attack method. It randomly samples points in the allowed input ranges and checks if any samples violate the property; if so, a counterexample is found.

Example 7.1.1 (Random Search)

Suppose the network input ranges are:

$$-1 \leq x_1 \leq 1, \quad -2 \leq x_2 \leq 2 \tag{119}$$

and the network output is:

$$y = 2x_1 - 1.5x_2 + 1. \tag{120}$$

We wish to find a counterexample to the property $y > 0$, i.e., find inputs (x_1, x_2) in the given ranges producing $y \leq 0$.

- Try 1: $x_1 = 0.2, x_2 = -0.5$: $y = 2 \times 0.2 - 1.5 \times (-0.5) + 1 = 0.4 + 0.75 + 1 = 2.15 > 0$. Not a violation.
- Try 2: $x_1 = -1, x_2 = 2$: $y = 2 \times (-1) - 1.5 \times 2 + 1 = -2 - 3 + 1 = -4 < 0$. **Counterexample found**: $(x_1 = -1, x_2 = 2)$.

◊

Problem 7.1.2 (Random Search Exercise)

Consider the input ranges $-1 \leq x_1 \leq 1, \quad -1 \leq x_2 \leq 1$ and network output $y = 3x_1 + 2x_2 - 2$.

Use random search to find a counterexample to the property $y > 0$:

1. Randomly sample three valid points and compute y at each point.
2. Check if any sample violates the property. Note, if none violates, that's OK; just report the results.
3. Based on your samples, discuss whether random search is likely to be efficient for this problem.

◊

7.2 Projected Gradient Descent (PGD)

PGD is a powerful and widely used adversarial attack that builds on the idea of *gradient descent*. Gradient descent repeatedly moves the input a small amount in the direction that most rapidly decreases some objective by following the negative gradient.

Specifically, at each step PGD computes the gradient of the network output with respect to the input and takes a small gradient-descent step that pushes the input toward a stronger property violation. Next, PGD *projects* (clips) the updated input back into the allowed domain if the step goes outside the input bounds.

We reuse Example 7.1.1 to illustrate PGD. The input ranges are $-1 \leq x_1 \leq 1, -2 \leq x_2 \leq 2$, the output is $y = 2x_1 - 1.5x_2 + 1$, and the goal is to find a counterexample producing $y \leq 0$.

1. **Initialize.** Choose a valid starting input, e.g., $\left(x_1^{(0)}, x_2^{(0)}\right) = (0, 0)$. This satisfies the input ranges but does not violate $y > 0$ since $y = 1 > 0$. Proceed.
2. **Iterative update.** For each step $t = 0, 1, 2, ...$:

   a. **Compute the gradient.** The gradient $\nabla_x y$ indicates how sensitive y is to each input:

   $$\nabla_x y = \left(\frac{\partial y}{\partial x_1}, \frac{\partial y}{\partial x_2}, ..., \frac{\partial y}{\partial x_n}\right) \tag{121}$$

   It represents the direction of steepest *increase* in y. To minimize y (make violation $y \leq 0$ more likely), move in the *opposite* direction $-\nabla_x y$.

   For $y = 2x_1 - 1.5x_2 + 1$: $\frac{\partial y}{\partial x_1} = 2$, $\frac{\partial y}{\partial x_2} = -1.5$, so $\nabla_x y = (2, -1.5)$.

   b. **Gradient update.** Move distance η (the *step size*) in the negative gradient direction:

   $$x_1^{(t+1)} = x_1^{(t)} - \eta \cdot 2, \quad x_2^{(t+1)} = x_2^{(t)} - \eta \cdot (-1.5) \tag{122}$$

   With $\eta = 0.5$ from $(0, 0)$: $x_1^{(1)} = -1.0$, $x_2^{(1)} = +0.75$.

   c. **Project (Clip) to valid range.** If new values are outside bounds, clip them:

   $$x_1^{(t+1)} = \max\left(-1, \min\left(x_1^{(t+1)}, 1\right)\right), \quad x_2^{(t+1)} = \max\left(-2, \min\left(x_2^{(t+1)}, 2\right)\right) \tag{123}$$

   The candidate $(-1.0, +0.75)$ is already within range; no clipping needed.

   d. **Check for violation.** Compute $y = 2(-1) - 1.5(0.75) + 1 = -2.125 < 0$. PGD finds counterexample $(x_1, x_2) = (-1, 0.75)$.

Example 7.2.1 (PGD with Clipping)

Using step size $\eta = 1.0$ from $(0, 0)$:

$$\begin{aligned} x_1^{(1)} &= 0 - 1.0 \times 2 = -2.0 \\ x_2^{(1)} &= 0 - 1.0 \times (-1.5) = +1.5 \end{aligned} \tag{124}$$

$x_1^{(1)} = -2.0$ is outside $[-1, 1]$, so clip to $x_1^{(1)} = -1$. $x_2^{(1)} = 1.5$ is within $[-2, 2]$; unchanged.

Projected input: $(-1, 1.5)$. Output: $y = 2(-1) - 1.5(1.5) + 1 = -3.25 < 0$. Counterexample found.

◊

Example 7.2.2 (Gradient Computation)

Consider $y = x_1^2 + 3x_2$. The gradient is:

$$\nabla_x y = \left(\frac{\partial y}{\partial x_1}, \frac{\partial y}{\partial x_2} \right) = (2x_1, 3). \tag{125}$$

At $(x_1, x_2) = (1, 2)$: $\nabla_x y = (2, 3)$. Near $(1, 2)$, increasing x_1 raises y twice as fast as increasing x_2 would.

◊

Problem 7.2.3 (Compute Gradients)

For each output function below, compute $\nabla_x y = \left(\frac{\partial y}{\partial x_1}, \frac{\partial y}{\partial x_2} \right)$ and explain what the gradient tells you about how y changes:

1. $y = 2x_1 - x_2 + 1$
2. $y = -x_1^2 + 4x_2$
3. $y = 3x_1 x_2 - x_2^2$

◊

Problem 7.2.4 (PGD Iterations)

Let $y = 2x_1 - 3x_2 + 1$, with $-1 \leq x_1, x_2 \leq 1$. Starting from $(0, 0)$ with step size $\eta > 0$, use PGD to minimize y and violate $y > 0$. Find two step sizes η_1 and η_2 such that:

1. With η_1, PGD finds a counterexample in *one* iteration.
2. With η_2, PGD finds a counterexample in *two or more* iterations.

Solution. The gradient is $\nabla_x y = (2, -3)$. PGD update: $x_1^{(t+1)} = x_1^{(t)} - 2\eta$, $x_2^{(t+1)} = x_2^{(t)} + 3\eta$.

Case A: $\eta_1 = 0.5$.

- Iteration 1: $x_1^{(1)} = 0 - 0.5(2) = -1$, $x_2^{(1)} = 0 + 0.5(3) = 1.5$. Clip x_2 to 1. Check: $y = 2(-1) - 3(1) + 1 = -4 < 0$. **Counterexample** $(-1, 1)$ in one iteration.

Case B: $\eta_2 = 0.07$.

- Iteration 1: $x_1^{(1)} = -0.14$, $x_2^{(1)} = 0.21$. Check: $y = 2(-0.14) - 3(0.21) + 1 = 0.09 > 0$. Continue.
- Iteration 2: $x_1^{(2)} = -0.28$, $x_2^{(2)} = 0.42$. Check: $y = 2(-0.28) - 3(0.42) + 1 = -0.82 < 0$. **Counterexample** $(-0.28, 0.42)$ in two iterations.

◊

8 Proof Generation and Checking

As NNV tools become more complex (e.g., state-of-the-art tools have 20K+ lines of code), they are more prone to bugs. VNN-COMP'23 [18] showed that 3 of the top 7 participants produced unsound results by claiming unsafe networks are safe, i.e., they return UNSAT on problems that are actually SAT.

While checking counterexamples is straightforward (we evaluate the network on the input), checking UNSAT results—proving no counterexample exists—is more challenging as it requires tracing the reasoning steps of the verifier. In this chapter, we discuss work [19] on generating and checking proofs of UNSAT results generated by BaB-based NNV tools.

8.1 Proof Generation for Branch and Bound Algorithms

The BAB algorithm splits the problem into smaller subproblems and uses abstraction to compute bounds to prune the search space. This structure allows proof generation capabilities to be added with minimal overhead to existing BAB{}-based NNV tools.

PROOFGEN extends the basic BAB algorithm with proof generation. The key idea is to introduce a **proof tree** and record the branching decisions into it. The proof tree has a binary structure, where each node represents a neuron and its left and right edges represent its activation decision (active or inactive). At the end of verification, the proof tree is returned as the proof of UNSAT.

```
Algorithm: ProofGen (BaB with Proof Generation)
Input:  NN N, property phi_in => phi_out
Output: (unsat, ProofTree) if valid, else (sat, counterexample)
```

```
ActPatterns = {ø}     // initialize with empty activation pattern
ProofTree   = {}      // initialize empty proof tree

while ActPatterns is not empty:
  σ_i = Select(ActPatterns)
  if Deduce(N, phi_in, phi_out, σ_i):        // feasible
    cex = LP(N, phi_in, phi_out, σ_i)
    if cex: return (sat, cex)                // counterexample found
    v_i = Decide(N, σ_i)                    // pick neuron to split
    ActPatterns += {σ_i ∪ {v_i}, σ_i ∪ {v̄_i}}
  else:                                      // infeasible — prune
    ProofTree += {σ_i}                       // record in proof

return (unsat, ProofTree)
```

Example 8.1.1 (ProofGen Illustration)

We reuse Example 6.2.1 to illustrate ProofGen. The goal is to verify the property $(x_1, x_2) \in [-2.0, 2.0] \times [-1.0, 1.0] \Rightarrow (y_1 > y_2)$.

ProofGen first splits neuron v_4, creating two subproblems. The inactive-v_4 subproblem is determined to be UNSAT (leaf node 3). The active-v_4 subproblem is further split on v_2. The inactive-v_2 subproblem is UNSAT (leaf node 5). The active-v_2 subproblem is split on v_1. Both resulting subproblems (leaf nodes 6 and 7) are UNSAT. Since all branches lead to UNSAT, the property is valid and the proof tree is returned.

◊

Problem 8.1.2 (Proof Tree Construction)

Consider the following network:

$$\begin{aligned} \hat{x}_3 &= x_1 - 2x_2 + 1, \quad x_3 = \text{relu}(\hat{x}_3) \\ \hat{x}_4 &= -x_1 + x_3 + 0.5, \quad x_4 = \text{relu}(\hat{x}_4) \\ y &= -x_3 + 2x_4 \end{aligned} \tag{126}$$

The input region is $(x_1, x_2) \in [-1, 1] \times [-1, 1]$ and the property to verify is $y \leq 5$.

A verifier uses BaB with neuron splitting on x_3 and x_4 in order:

1. First split on x_3.
2. For the active-x_3 branch, split on x_4.

The solver determines: (a) inactive x_3 is UNSAT, (b) active x_3 + inactive x_4 is UNSAT, (c) both active is also UNSAT.

Do the following:

1. Draw the *proof tree*, labeling each split node with the neuron being split.
2. For each split, explain: which constraints are added (e.g., $\hat{x}_3 \geq 0$ or $\hat{x}_3 \leq 0$), how equations simplify, and why the subproblem is UNSAT.
3. Explain why the proof tree is a correctness proof — why showing all leaf nodes are UNSAT proves $y \leq 5$ is valid.

◊

8.2 Proof Language

Rather than recording the generated proof tree in a verifier-specific format, it is more desirable to have a standard format that is human-readable, compact, efficiently generated by verification tools, and independently processable by proof checkers.

ProofLang is designed to meet these requirements. It is inspired by the SMTLIB format [6] used for SMT solving, which has also been adopted by the VNNLIB language [4] to specify neural network properties (see Sec. 2.4 for examples).

A ProofLang proof is composed of **declarations** and **assertions**. The grammar is:

```
<proof>        ::= <declarations> <assertions>
<declarations> ::= <declaration> | <declaration> <declarations>
<declaration>  ::= (declare-const <input-vars>  Real)
                 | (declare-const <output-vars> Real)
                 | (declare-pwl   <hidden-vars> <activation>)
<activation>   ::= ReLU | LeakyReLU | ...
<assertions>   ::= <assertion> | <assertion> <assertions>
<assertion>    ::= (assert <formula>)
<formula>      ::= (<op> <term> <term>)
                 | (and <formula>+) | (or <formula>+)
<term>         ::= <input-var> | <output-var> | <hidden-var> | <constant>
<op>           ::= < | <= | > | >=
<input-var>    ::= X_<constant>
<output-var>   ::= Y_<constant>
<hidden-var>   ::= N_<constant>
```

Input variables (prefixed `X`) and *output variables* (prefixed `Y`) are declared as reals. *Hidden variables* (prefixed `N`) correspond to internal neurons and are declared with their piecewise-linear activation function (e.g., ReLU). Assertions over input variables are *preconditions* and those over output variables are *postconditions*. The hidden constraints encode the activation patterns of the proof tree, where each `and` clause represents one tree path.

Below is a ProofLang proof for the example in Example 6.2.1:

```
; Declare variables
(declare-const X_0 X_1 Real)
(declare-const Y_0 Y_1 Real)
(declare-pwl N_1 N_2 N_3 N_4 ReLU)

; Input constraints (precondition)
(assert (>= X_0 -2.0))
(assert (<= X_0  2.0))
(assert (>= X_1 -1.0))
(assert (<= X_1  1.0))

; Output constraint (negated postcondition)
(assert (<= Y_0 Y_1))

; Hidden constraints (proof tree paths — one per leaf)
(assert (or
    (and (<  N_4 0))
    (and (<  N_2 0) (>= N_4 0))
```

```
    (and (>= N_2 0) (>= N_1 0) (>= N_4 0))
    (and (>= N_2 0) (<  N_1 0) (>= N_4 0))
))
```

Example 8.2.1 (ProofLang Format)

The `(and (< N_4 0))` clause corresponds to the rightmost path of the proof tree with $\overline{v_4}$ (leaf 3). The `(and (< N_2 0) (>= N_4 0))` clause corresponds to the path with $v_4 \wedge \overline{v_2}$ (leaf 5).

◊

ProofLang intentionally omits network weights and biases — these are available in the standard ONNX format [2], which any ProofLang checker can access independently. The explicit DNF (disjunctive normal form) structure enables easy parallelization of proof checking.

8.3 Proof Checker

ProofCheck is a verifier-independent proof checker for ProofLang proofs generated by BaB{}-based NNV tools.

8.3.1 The Core Algorithm

The goal of ProofCheck is to verify that each *leaf* node of the proof tree is UNSAT. To check a leaf, ProofCheck forms an MILP problem (Sec. 4.2) consisting of the NNV constraints (Eq. 37) — network, input condition, negated output — augmented with the activation-pattern constraints encoded by the tree path to that leaf. It then invokes an LP solver; infeasibility certifies that leaf.

```
Algorithm: ProofCheck
Input:  Network N, property phi_in => phi_out, ProofTree
Output: certified if proof valid, else uncertified

if not RepOK(ProofTree): raise Error("Invalid proof tree")

model = CreateStabilizedMILP(N, phi_in, phi_out)
node  = null

while ProofTree is not empty:
  node  = Select(ProofTree, node)         // get next leaf to check
  model = AddConstrs(model, node)         // add leaf's activation constraints
  if Feasible(model):
    return uncertified                    // LP found a satisfying point
  // (leaf is infeasible — continue)

return certified
```

ProofCheck first validates the proof tree structure (Sec. 8.2). It then creates an MILP model for the network and property. The main loop selects leaf nodes one at a time, adds their activation-pattern constraints to the model, and calls the LP solver. If any leaf is found feasible, the proof is invalid. After all leaves are verified infeasible, ProofCheck returns CERTIFIED.

Example 8.3.1.1 (ProofCheck on BaB Example)

For the ProofLang proof in Sec. 8.2, ProofCheck must certify the four leaf nodes 3, 5, 6, and 7. Suppose it first selects node 3, which has the single constraint $\overline{v_4}$, i.e., $0.6v_1 + 0.9v_2 - 0.1 \leq 0$. It conjoins this with the NNV constraints for the input region and network, then invokes the LP solver. The LP is infeasible — leaf 3 is certified. ProofCheck continues for the remaining three leaves and returns certified once all are verified.

◊

8.3.2 Optimizations

While the core algorithm is minimal, it can be inefficient for large proofs. ProofCheck employs several optimizations.

Neuron Stabilization.

A primary challenge in NNV is the presence of large numbers of piecewise-linear constraints (e.g., ReLU), which generate many branches and yield large proof trees. In the MILP formulation this creates many disjunctions that are hard to solve. ProofCheck uses *neuron stabilization* [20] to identify neurons that are *stable* (always active or always inactive) for all inputs satisfying the property precondition. For stable neurons, the disjunctive ReLU constraint is replaced by a simpler linear constraint, reducing MILP complexity.

Pruning Leaf Nodes. A child node in the proof tree adds constraints to its parent's constraints (the child's activation pattern is strictly more constrained). Therefore, if a parent node is unsat, all its descendants must also be unsat. ProofCheck exploits this by using a **backtracking** mechanism: after certifying a leaf l, it checks the parent p; if p is unsat, the sibling subtree of l is pruned. This backjumping can proceed up to N levels at a time. The default $N = 2$ balances pruning benefit against the cost of solving less-constrained (harder) parent problems.

Parallelization. The DNF structure of a ProofLang proof tree is inherently parallel — each tree path is an independent sub-proof. ProofCheck uses a parameter k to check k leaf nodes concurrently using multiprocessing.

9 Common Engineerings and Optimizations

In addition to adversarial attacks (Sec. 7), NNV tools often employ a range of optimizations and engineering techniques to improve performance. This chapter discusses some of those common techniques.

9.1 Input Splitting

Many verifiers use a technique called *input splitting* to quickly deal with networks with verification problems involving low-dimensional networks, such as those in the ACAS Xu benchmark where the networks have a small number of inputs (e.g., ≤ 50). The idea is to split the original verification problem into subproblems, each checking whether the network produces the desired output from a smaller input region and returns UNSAT if all subproblems are verified and SAT if a counterexample is found in any subproblem. Input splitting avoids BaB search (Sec. 6)—which performs *neuron splitting*—and is often used to quickly eliminate easy cases.

Moreover, each subproblem now has a smaller input region, thus the verifier can often verify them more quickly than the original problem. Finally, because each task can be solved independently, the verifier can solve them in parallel to further speed up the verification process.

Example 9.1.1

Given a problem with input region $\{x_1 \in [-1, 1], x_2 \in [-2, 2]\}$, input splitting splits the input region into four subregions: $\{x_1 \in [-1, 0], x_2 \in [-2, 0]\}$, $\{x_1 \in [-1, 0], x_2 \in [0, 2]\}$, $\{x_1 \in [0, 1], x_2 \in [-2, 0]\}$, and $\{x_1 \in [0, 1], x_2 \in [0, 2]\}$. The verifier then checks if the network produces the desired output from each of these subregions.

Note that the formula $-1 \leq x_1 \leq 1 \wedge -2 \leq x_2 \leq 2$ representing the original input region is equivalent to the formula $(-1 \leq x_1 \leq 0 \vee 0 \leq x \leq 1) \wedge (-2 \leq x_2 \leq 0 \vee 0 \leq x_2 \leq 2)$ representing the combination of the created subregions.

◊

9.2 Bounds Tightening

9.2.1 Input Bounds Tightening

When the verifier splits on a neuron (e.g., $\hat{x}_i \leq 0$ or $\hat{x}_i > 0$), it adds new linear constraints that further restrict the feasible input region. As a result, the original input bounds (e.g., $-1 \leq x_1 \leq 1$) may no longer reflect the true range of values that satisfy all current constraints. **Input Bound tightening** recomputes the smallest possible ranges for each input variable under these constraints. These tighter input bounds propagate through the network and yield tighter neuron and output bounds, which in turn helps prune subproblems earlier.

Example 9.2.1.1

Recall the network from Fig. 12 can be represented as:

$$\begin{aligned} \hat{x}_3 &= -0.5x_1 + 0.5x_2 + 1.0 \land \\ \hat{x}_4 &= 0.5x_1 - 0.5x_2 + 1.0 \land \\ x_3 &= \text{relu}(\hat{x}_3) \land \\ x_4 &= \text{relu}(\hat{x}_4) \land \\ x_5 &= -x_3 + x_4 - 1.0, \end{aligned} \tag{127}$$

and input property φ_{in}: $-1 \leq x_1 \leq 1 \land -2 \leq x_2 \leq 2$.

Suppose we make a split on x_3, which gives us two subproblems, represented by the constraints: $-0.5x_1 + 0.5x_2 + 1.0 \leq 0$ and $-0.5x_1 + 0.5x_2 + 1.0 > 0$. We can use the new constraint to tighten the input bounds of x_1 and x_2 for each subproblem.

For the first subproblem, we can create an optimization problem to tighten the bounds. For example, to tighten the upper bounds of x_1, we can create an optimization problem as follows:

$$\begin{aligned} \text{maximize } & x_1 \quad \text{s.t.} \\ & -0.5x_1 + 0.5x_2 + 1.0 \leq 0 \\ & -1 \leq x_1 \leq 1 \\ & -2 \leq x_2 \leq 2 \end{aligned} \tag{128}$$

We can apply the same approach to tighten the lower bounds of x_1 using a minimization problem. Similarly, we can create optimization problems to tighten both the bounds of x_2.

◊

Note that this approach works well for networks with small inputs, e.g., the ACAS Xu benchmark with 5 inputs, and is adopted by many modern NNV tools, including Marabou and nnenum. For larger input dimensions networks, it will take much time as we need to run $2\times$ the number of inputs to tighten all the input bounds (for each input, we need to tighten the upper and lower bounds individually).

To reduce the time complexity, we can prioritize the input variables to tighten first, e.g., deciding the variable with the largest bounds first, and only tighten up to a maximum number of inputs, e.g., 10 inputs.

9.2.2 Neuron (Hidden) Bounds Tightening

Instead of tightening the input bounds, we can tighten the bounds of the hidden neurons. There are a couple of advantages of tightening the bounds of the hidden neurons:

1. In some networks, numbers of hidden neurons are *fewer* than the number of inputs, e.g., the MNISTFC 2x256 network has $28 \times 28 = 784$ inputs and only 512 hidden neurons.
2. We don't necessarily tighten all hidden neurons, as some hidden neurons are already stable, i.e., their lower bounds are greater than 0 or their upper bounds are less than 0.

3. More importantly, if a hidden neuron is stabilized after tightening its bounds, e.g., its lower bound becomes greater than 0 or its upper bound becomes less than 0, we no longer need to branch on that particular neuron.

Example 9.2.2.1

Let's revisit Example 9.2.1.1; after splitting x_4, we can tighten the upper bounds of x_4 (suppose that $l_{\{x_4\}} < 0 < u_{\{x_4\}}$, e.g., unstable and not split yet) by solving the following optimization problem:

$$\begin{aligned} \text{maximize } x_4 \quad \text{s.t.} \\ 0.5x_1 - 0.5x_2 + 1.0 \leq 0 \\ -1 \leq x_1 \leq 1 \\ -2 \leq x_2 \leq 2 \end{aligned} \tag{129}$$

If the optimization result yields a maximum value of x_4 less than 0, e.g., $u_{\{x_4\}} < 0$, we can conclude that x_4 is stabilized and no longer need to branch on x_4, thus reducing the number of branches made by the verifier. There will be many tweaks to make the optimization problem more efficient, e.g., setting an early stopping condition: if the maximum value of x_4 is less than 0, we can stop the optimization. This is because, if the upper bound of x_4 is already less than 0, x_4 will be considered as *inactive* no matter what the *optimal* upper bound is.

Note that in both cases optimization problems are independent (between inputs/neurons and lower/upper bounds) and thus can be solved in parallel, e.g., optimizing both upper and lower bounds of x_1 (input) and x_4 (neuron) simultaneously.

◊

9.3 Batch Processing

Matrix Form for Interval Computation

While the min/max formulation, e.g., for abstraction Sec. 5.4, is intuitive, it can be efficiently implemented using matrix operations. We can decompose the weight matrix W into positive and negative components:

$$W^+ = \max(W, 0) \qquad W^- = \min(W, 0) \tag{130}$$

where the max and min operations are applied element-wise.

Then the interval bounds can be computed as:

$$\begin{aligned} f_L^a &= W^+ \cdot \boldsymbol{l} + W^- \cdot \boldsymbol{u} + \boldsymbol{b} \\ f_U^a &= W^+ \cdot \boldsymbol{u} + W^- \cdot \boldsymbol{l} + \boldsymbol{b} \end{aligned} \tag{131}$$

where $\boldsymbol{l} = [l_1, l_2, ..., l_n]^T$ and $\boldsymbol{u} = [u_1, u_2, ..., u_n]^T$ are the vectors of lower and upper bounds.

Example 9.3.1

We can verify that the matrix form produces identical results to the min/max approach using the same network from Example 5.4.1.1.

Given the weight $w_1 = -0.5$, $w_2 = 0.5$, and bias $b = 1.0$, we decompose: $W^+ = [\max(-0.5, 0), \max(0.5, 0)] = [0, 0.5]$ $W^- = [\min(-0.5, 0), \min(0.5, 0)] = [-0.5, 0]$

For inputs $x_1 \in [1, 2]$ and $x_2 \in [-1, 3]$, the lower vector $\boldsymbol{l} = [1, -1]$ and the upper vector $\boldsymbol{u} = [2, 3]$:

$$\begin{aligned} f_L^a &= W^+ \cdot \boldsymbol{l} + W^- \cdot \boldsymbol{u} + b \\ &= [0, 0.5] \cdot [1, -1] + [-0.5, 0] \cdot [2, 3] + 1 \\ &= 0 \cdot 1 + 0.5 \cdot (-1) + (-0.5) \cdot 2 + 0 \cdot 3 + 1 \\ &= 0 - 0.5 - 1.0 + 0 + 1 = -0.5 \end{aligned} \tag{132}$$

$$\begin{aligned} f_U^a &= W^+ \cdot \boldsymbol{u} + W^- \cdot \boldsymbol{l} + b \\ &= [0, 0.5] \cdot [2, 3] + [-0.5, 0] \cdot [1, -1] + 1 \\ &= 0 \cdot 2 + 0.5 \cdot 3 + (-0.5) \cdot 1 + 0 \cdot (-1) + 1 \\ &= 0 + 1.5 - 0.5 + 0 + 1 = 2.0 \end{aligned} \tag{133}$$

This gives us $f^{a([1,2],[-1,3])} = [-0.5, 2.0]$, which matches exactly the result from the min/max approach in Example 5.4.1.1.

◊

The matrix formulation of interval arithmetic is equivalent to the min/max approach but enables efficient batch processing of multiple input regions simultaneously. This is particularly useful when splitting input regions into multiple subproblems and computing abstraction in parallel.

Consider an input region that needs to be split for improving abstraction precision. For example, we can split the input region $\{x_1 \in [-1, 1], x_2 \in [-2, 2]\}$ along the first dimension into two subproblems:

1. Subproblem 1: $\{x_1 \in [-1, 0], x_2 \in [-2, 2]\}$
2. Subproblem 2: $\{x_1 \in [0, 1], x_2 \in [-2, 2]\}$

Using the matrix form in Example 9.3.1, we can process both subproblems simultaneously by stacking the bounds into matrices:

$$\boldsymbol{L} = \begin{pmatrix} -1 & -2 \\ 0 & -2 \end{pmatrix}, \quad \boldsymbol{U} = \begin{pmatrix} 0 & 2 \\ 1 & 2 \end{pmatrix} \tag{134}$$

where each row represents the lower and upper bounds for one subproblem.

Next, for a linear layer with weight matrix W and bias vector $\boldsymbol{b}$, we can compute the bounds for all subproblems simultaneously:

$$\begin{aligned} \boldsymbol{O}_L &= \boldsymbol{L} \cdot (W^+)^T + \boldsymbol{U} \cdot (W^-)^T + \boldsymbol{b} \\ \boldsymbol{O}_U &= \boldsymbol{U} \cdot (W^+)^T + \boldsymbol{L} \cdot (W^-)^T + \boldsymbol{b} \end{aligned} \tag{135}$$

Example 9.3.2

Consider the network from Example 5.4.1.1 with weight $W = [-0.5, 0.5]$ and bias $b = 1.0$. We split the input region $\{x_1 \in [-1, 1], x_2 \in [-2, 2]\}$ into two subproblems as described above.

First, we decompose the weight: $W^+ = [0, 0.5]$ $W^- = [-0.5, 0]$ $\boldsymbol{b} = [1.0, 1.0]$ The bias vector is the same for both subproblems, thus just duplicate it for each subproblem. Then we compute bounds for both subproblems:

$$\boldsymbol{L} \cdot (W^+)^T = \begin{pmatrix} -1 & -2 \\ 0 & -2 \end{pmatrix} \begin{pmatrix} 0 \\ 0.5 \end{pmatrix} = \begin{pmatrix} -1.0 \\ -1.0 \end{pmatrix} \tag{136}$$

$$\boldsymbol{U} \cdot (W^-)^T = \begin{pmatrix} 0 & 2 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} -0.5 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.0 \\ -0.5 \end{pmatrix} \tag{137}$$

$$\boldsymbol{O}_L = \begin{pmatrix} -1.0 \\ -1.0 \end{pmatrix} + \begin{pmatrix} 0.0 \\ -0.5 \end{pmatrix} + \begin{pmatrix} 1.0 \\ 1.0 \end{pmatrix} = \begin{pmatrix} 0.0 \\ -0.5 \end{pmatrix} \tag{138}$$

Similarly for upper bounds:

$$\boldsymbol{U} \cdot (W^+)^T = \begin{pmatrix} 0 & 2 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 0 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 1.0 \\ 1.0 \end{pmatrix} \tag{139}$$

$$\boldsymbol{L} \cdot (W^-)^T = \begin{pmatrix} -1 & -2 \\ 0 & -2 \end{pmatrix} \begin{pmatrix} -0.5 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.5 \\ 0.0 \end{pmatrix} \tag{140}$$

$$\boldsymbol{O}_U = \begin{pmatrix} 1.0 \\ 1.0 \end{pmatrix} + \begin{pmatrix} 0.5 \\ 0.0 \end{pmatrix} + \begin{pmatrix} 1.0 \\ 1.0 \end{pmatrix} = \begin{pmatrix} 2.5 \\ 2.0 \end{pmatrix} \tag{141}$$

Therefore:

- Subproblem 1: $x_3 \in [0.0, 2.5]$
- Subproblem 2: $x_3 \in [-0.5, 2.0]$

We can verify these results by computing each subproblem individually using the original formulation, which yields identical results.

◊

9.4 GPU Processing

By using matrix operations, we can leverage GPUs natively to speed up the computation. As expected, the computation time is reduced significantly using GPUs when the size of matrices (weights, bias, inputs, etc.) becomes larger Fig. 27.

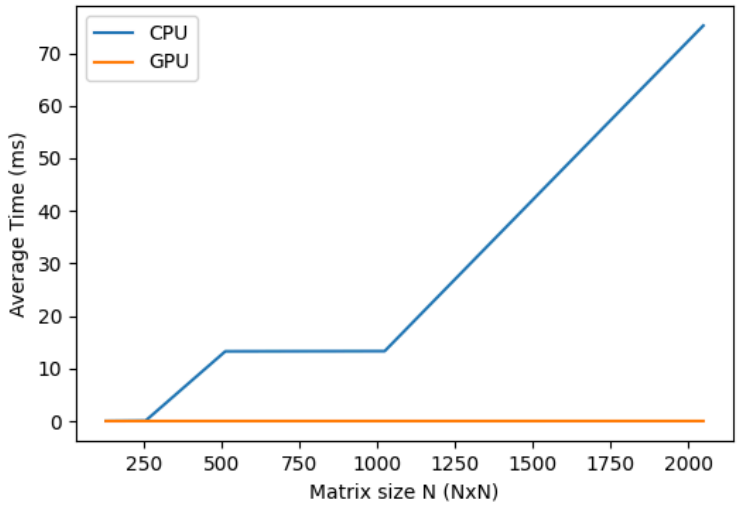


Fig. 27: GPU Benchmarking Results.

```python
import torch

# weights and bias
W = torch.tensor([[0, 0.5], [-0.5, 0]]).cuda()
b = torch.tensor([1.0, 1.0]).cuda()

# input bounds
L = torch.tensor([[-1, -2], [0, -2]]).cuda()
U = torch.tensor([[0, 2], [1, 2]]).cuda()

# decompose weights
W_plus = W.clamp(min=0)
W_minus = W.clamp(max=0)

# compute bounds
O_L = L @ W_plus.T + U @ W_minus.T + b
O_U = U @ W_plus.T + L @ W_minus.T + b
```

10 The NeuralSAT Algorithm

NeuralSAT [15], [20] is a relatively new competitor in NNV, but it has quickly become a strong contender, consistently placed among the top tools at NNV competitions (Sec. A).

At its core, NeuralSAT is BaB, but follows a DPLL(T) framework [21] and includes specialized optimizations and heuristics to improve its search. Thus, NeuralSAT is essentially an SMT solver (Sec. B.4) with respect to a theory, in this case, the theory of DNNs.

10.1 Overview

placeholder

Fig. 28: *TODO: NeuralSAT overview figure not yet added*

Fig. 28 gives an overview of NeuralSAT, which consists of standard DPLL components (light shades) and the theory solver (dark shade). NeuralSAT first constructs a propositional formula over Boolean variables that represent the activation status of neurons (*Boolean Abstraction*). Clauses in the formula assert that each neuron, e.g., neuron i, is active or inactive, e.g., $v_i \vee \overline{v_i}$. This representation enables using standard DPLL to search for truth values satisfying these clauses and a neural network-specific theory solver to check the feasibility of truth assignments—*activation patterns* (Sec. 6.1)—with respect to the constraints encoding the network and the property of interest.

NeuralSAT now enters an iterative process to find activation pattern (truth assignment) satisfying the activation clauses. First, NeuralSAT assigns a truth value to an unassigned variable (*Decide*), detects unit clauses caused by this assignment, and infers additional assignments (*Boolean Constraint Propagation*). Next, NeuralSAT invokes the theory solver or T-solver (*Deduction*), which uses LP solving and abstraction to check the feasibility of the constraints of the current assignment with the property of interest.

If the T-solver confirms feasibility, NeuralSAT continues with new assignments (*Decide*). Otherwise, NeuralSAT detects a conflict (*Analyze Conflict*) and learns clauses to remember it and backtrack to a previous decision (*Backtrack*). If NeuralSAT detects local optima, it would restart (*Restart*) the search by clearing all decisions that have been made, but save the conflict clauses learned so far to avoid reaching the same state in the next runs. Restarting especially benefits challenging NNV problems by enabling better clause learning and exploring different decision orderings.

This iterative process repeats until NeuralSAT can no longer backtrack, and returns UNSAT, indicating the network has the property, or it finds a total assignment for all boolean variables, and returns SAT.

10.2 Illustration

Example 10.2.1 (NeuralSAT Verification Run)

We use NeuralSAT to prove that for inputs $x_1 \in [-1, 1], x_2 \in [-2, 2]$ the network in Fig. 1 produces the output $x_5 \leq 0$. NeuralSAT takes as input the formula α representing the network:

$$\begin{aligned} x_3 &= \text{ReLU}(-0.5x_1 + 0.5x_2 + 1) \quad \wedge \\ x_4 &= \text{ReLU}(x_1 + x_2 - 1) \quad \wedge \\ x_5 &= -x_3 + x_4 - 1.0 \end{aligned} \tag{142}$$

and the formula φ representing the property:

$$\varphi : -1 \leq x_1 \leq 1 \wedge -2 \leq x_2 \leq 2 \quad \Rightarrow \quad x_5 \leq 0. \tag{143}$$

To prove $\alpha \Rightarrow \varphi$, NeuralSAT needs to show that *no* values of x_1, x_2 satisfying the input properties would result in $x_5 > 0$. In other words, NeuralSAT needs to return `unsat` for:

$$\alpha \wedge -1 \leq x_1 \leq 1 \wedge -2 \leq x_2 \leq 2 \wedge x_5 > 0. \tag{144}$$

Notation: In the following, we write $x \mapsto v$ to denote that the variable x is assigned with a truth value $v \in \{T, F\}$. This assignment can be either decided by `Decide` or inferred by `BCP`. We also write $x@dl$ and $\overline{x}@dl$ to indicate the respective assignments $x \mapsto T$ and $x \mapsto F$ at decision level dl.

Boolean Abstraction First, NeuralSAT creates two Boolean variables v_3 and v_4 to represent the activation status of the hidden neurons x_3 and x_4, respectively. For example, $v_3 = T$ means x_3 is `active` and thus gives the constraint $-0.5x_1 + 0.5x_2 + 1 > 0$. Similarly, $v_3 = F$ means x_3 is `inactive` and therefore gives $-0.5x_1 + 0.5x_2 + 1 \leq 0$. Next, NeuralSAT forms two clauses $\{v_3 \vee \overline{v_3}; v_4 \vee \overline{v_4}\}$ ensuring that these variables are either `active` or `inactive`.

DPLL(T) Iterations NeuralSAT searches for a satisfying *activation pattern*—truth assignment for the Boolean variables to satisfy the clauses and the constraints they represent with respect to the formula in Eq. 144. For this example, NeuralSAT uses four iterations, summarized in Tab. 3, to determine that no such assignment exists and the problem is thus UNSAT.

Iter	**BCP**	**Deduction**		**Decide**	**Analyze Conflict**	
		Constraints	Bounds		Bt	Learned Clauses
Init	-	$I = -1 \leq x_1 \leq 1;$ $-2 \leq x_2 \leq 2$	$-1 \leq x_1 \leq 1;$ $-2 \leq x_2 \leq 2$	-	-	$C = \{v_3 \vee \overline{v_3}; v_4 \vee \overline{v_4}\}$
1	-	I	$x_5 \leq 1$	$\overline{v_4}@1$	-	-
2	-	$I; x_4 =$ `off`	$x_5 \leq -1$	-	0	$C = C \cup \{v_4\}$
3	$v_4@0$	$I; x_4 =$ `on`	$x_3 \geq 0.5; x_5 \leq 0.5$	$v_3@0$	-	-
4	-	$I; x_3 =$ `on` $; x_4 =$ `on`	-	-	**-1**	$C = C \cup \{\overline{v_4}\}$

Tab. 3: NeuralSAT's run producing UNSAT. The notation $x@dl$ and $\overline{x}@dl$ mean the assignments $x \mapsto T$ and $x \mapsto F$ at decision level dl, respectively.

In *iteration 1*, as shown in Fig. 28, NeuralSAT starts with `BCP`, which has no effects because the current clauses and (empty) assignment produce no unit clauses. In `Deduction`, NeuralSAT uses an LP solver to determine that the current set of constraints, which contains just the initial input bounds, is feasible[3]. NeuralSAT then

10.3 NeuralSAT vs. BaB

placeholder

Fig. 29: *TODO: NeuralSAT vs BaB combined search tree figure not yet added*

placeholder

Fig. 30: *TODO: NeuralSAT search tree (smaller space) not yet added*

placeholder

Fig. 31: *TODO: BaB search tree (larger space) not yet added*

Note that this process of selecting and assigning (guessing) values to variables representing neurons is the *branching* phase in BaB. It is also commonly called *neuron splitting* because it splits the search tree into subtrees corresponding to the assigned values (e.g., see Sec. 10.3).

As mentioned in Sec. 3.2, ReLU-based NNV is NP-complete, and for difficult problem instances NNV tools often have to exhaustively search a very large space, making scalability a main concern for modern NNV approaches.

Fig. 29 shows the difference between NeuralSAT and another NNV tool (e.g., using the popular Branch-and-Bound (BaB) approach) in how they navigate the search space. We assume both tools employ similar abstraction and neuron splitting. Fig. 31 shows that the other tool performs splitting to explore different parts of the tree (e.g., splitting v_1 and explore the branches with $v_1 = T$ and $v_1 = F$ and so on). Note that the other tool needs to consider the tree shown regardless if it runs sequentially or in parallel.

In contrast, NeuralSAT has a smaller search space shown in Fig. 30. NeuralSAT follows the path v_1, v_2 and then $\overline{v_2}$ (just like the tool on the right). However, because of the learned clause $v_2 \vee v_3$, NeuralSAT performs a BCP step that sets v_3 (and therefore prunes the branch with $\overline{v_3}$ that needs to be considered in the other tree). Then NeuralSAT splits v_4, and like the other tool, determines infeasibility for both branches. Now NeuralSAT's conflict analysis determines from learned clauses that it needs to backtrack to v_3 (yellow node) instead of v_1. Without learned clauses and non-chronological backtracking, NeuralSAT would backtrack to decision v_1 and continues with the $\overline{v_1}$ branch, just like the other tool in Fig. 31.

Thus, NeuralSAT was able to generate non-chronological backtracks and use BCP to prune various parts of the search tree. In contrast, the other tool would have to move through the exponential search space to eventually reach the same result.

11 The Reluplex Algorithm

Reluplex [10] is a classical BaB (Sec. 6) approach for NNV. The technique extends the *simplex* method [22] to support the ReLU activation function (**Reluplex** = **Relu** + Sim**plex**). Reluplex has been succeeded by the more efficient Marabou tool [23]. However, the core ideas of Reluplex are still relevant and thus presented here as another example of BaB{}-based NNV techniques.

[3] We use the terms feasible, from the LP community, and satisfiable, from the SAT community, interchangeably.

11.1 Algorithm Overview

Reluplex extends the classical simplex method to handle the non-linear ReLU constraints $\hat{x} = \max(x, 0)$. Like simplex, Reluplex maintains a set of *basic* variables whose values are determined by other (non-basic) variables, and updates them through pivot operations.

Initialization The constraints representing the network are first rewritten into a basic form by introducing basic or *slack* variables. Reluplex also forms the lower and upper bounds for each variable based the input constraints and semantics ReLU, e.g., for a ReLU variable $\hat{x}_i$, the lower bound is 0.

All variable values are then initially set to some guess, such as 0, even if these values violate bounds. This initial configuration acts as the starting point for Reluplex to iteratively refine the variable values to satisfy all constraints.

Fixing bound violations Reluplex repeatedly checks all variables to see if any violate their bounds. If a non-basic variable violates its bounds, it can be directly updated to a valid value; the update is then propagated into the basic variables using the defining linear equations. If a basic variable violates its bounds, Reluplex cannot adjust it directly; instead it performs a *pivot*, swapping that basic variable with one of the non-basic variables appearing in its defining equation. After pivoting, the out-of-bounds variable becomes non-basic and can then be directly fixed.

Handling ReLU violations When all variable bounds are satisfied, Reluplex next checks the ReLU relations $\hat{x}_i = \max(x_i, 0)$. If a pair $(x_i, \hat{x}_i)$ is inconsistent (e.g., $x_i > 0$ but $\hat{x}_i = 0$), then the algorithm repairs it in the same way: if the violated variable is non-basic, it is simply updated; if it is basic, a pivot is performed so it can be updated.

Termination If no variable violates bounds or ReLU relations, the current configuration is a feasible assignment that satisfies all constraints, and Reluplex returns sat. If Reluplex explores all possible pivoting options without finding any feasible configuration, it concludes unsat.

11.2 Illustration

Example 11.2.1 (RELUPLEX on a Simple Network)

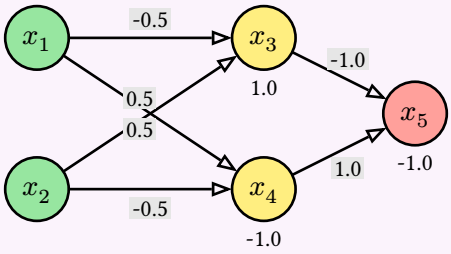


Fig. 32: A simple network (similar to Fig. 1).

We demonstrate RELUPLEX using the network in Fig. 32. This network has inputs x_1, x_2, two hidden neurons x_3, x_4 with ReLU activations, and one output x_5. It can be represented by the following equations:

$$\begin{aligned} x_3 &= x_1 - x_2, \quad \hat{x}_3 = \max(x_3, 0), \\ x_4 &= x_1 + x_2, \quad \hat{x}_4 = \max(x_4, 0), \\ x_5 &= 0.5\hat{x}_3 - 0.2\hat{x}_4 \end{aligned} \tag{145}$$

We want to check that the network has the property

$$(0 \leq x_1 \leq 0.5 \wedge -2 \leq x_2 \leq -1) \Rightarrow (x_5 < 0 \vee x_5 > 0.5). \tag{146}$$

That is, when the inputs x_1, x_2 fall within certain ranges, then the result x_5 has certain values. As shown in Sec. 3.1, we turn this into a satisfiability problem by negating the property:

$$(0 \leq x_1 \leq 0.5 \wedge -2 \leq x_2 \leq -1) \wedge 0 \leq x_5 \leq 0.5, \tag{147}$$

and checking whether there exists a counterexample satisfying the negated property.

We now use RELUPLEX to check whether there exists an assignment satisfying the negated property in Eq. 147. We start by introducing three *basic* (auxilliary or slack) variables to represent the relationships between the variables in the network:

$a_1 = x_1 - x_2 - x_3$

$a_2 = x_1 + x_2 - x_4$

$a_3 = 0.5\hat{x}_3 - 0.2\hat{x}_4 - x_5$

These basic variables are used to maintain the relationships between the variables in the constraints and are updated during the search process. In contrast, a *non-basic* variable is one that does not represent the relationship between other variables in the constraints. In this example, all variables except a_1, a_2, a_3 are non-basic.

RELUPLEX first assigns 0 to all variables in the equations in Eq. 147 – Eq. 147. This will likely violate some bounds, which RELUPLEX uses an iterative process, represented by a sequence of *configurations*, to refine.

Tab. 4 shows the initial configuration with all values assigned to 0. The lower (LB) and upper (UB) bounds of the inputs x_1, x_2 and output x_5 are specified in Eq. 147. The LBs of $\hat{x}_i$'s representing ReLUs are 0s, and the other hidden variables are unbounded (i.e., $-\infty$ to ∞).

	x_1	x_2	x_3	$\hat{x}_3$	x_4	$\hat{x}_4$	x_5	a_1	a_2	a_3
LB	0	−2	$-\infty$	0	$-\infty$	0	0	0	0	0
Val	0	**0**	0	0	0	0	0	0	0	0

Problem 11.2.2 (Reluplex by Hand)

Read the example in Example 11.2.1 again carefully. Then:

- Rewrite the steps in Example 11.2.1 on paper, showing all the calculations for each step. The goal of handwriting the steps is to help you understand how Reluplex works in detail.
- Provide feedback on this example. Is it clear? Are there any steps or terminologies that are confusing? How can we improve the explanation?

◊

Problem 11.2.3 (Basic vs. Non-Basic Variables)

Consider the description of Reluplex in Sec. 11.1 and the example in Example 11.2.1.

1. Explain the difference between a *basic* and a *non-basic* variable in Reluplex.
2. Why can Reluplex update a non-basic variable directly, but must perform a pivot before updating a basic variable? Use an example if necessary.
3. Explain why Reluplex must *still* consider ReLU violations after all bound violations have been fixed. Why is all variables (excluding ReLU ones) within bounds not sufficient? Use an example if necessary.

◊

A VNN-COMPs

As NNV matured, it became increasingly difficult to compare approaches. Papers often evaluated tools on different benchmarks, under different hardware assumptions, and using different specification languages. This fragmentation made it hard to assess the true state of the art, identify open challenges, and drive progress. This chapter describes the VNN-COMPs, which were created to address this problem by providing a common platform for evaluating and comparing NNV tools, and discusses common features of SOTA tools.

A.1 VNN-COMPs

Inspired by the success of other formal methods competitions (e.g., SAT, SMT, model checking), the International Verification of Neural Networks Competition (**VNN-COMP** [5], [18], [24]) encourages the standardization of tool interfaces and brings together the NNV community. To this end, VNN-COMP uses standardized formats for networks (ONNX [2]) and specification (VNN-LIB [4]), and evaluates tools using a common set of benchmarks on a common platform (AWS instances).

VNN-COMP'25 Results. In the latest 2025 VNN-COMP iteration [24], 7 teams participated on a diverse set of 16 regular and 6 extended benchmarks. Tab. 10 presents the overall scores and rankings of the tools (Tab. 6 in [24]) and a cactus plot of tool performance on all benchmark instances (Fig. 6 in [24]). In summary, NeuralSAT ranks 2nd overall, behind $\alpha\beta$-CROWN, a regular winner of VNN-COMPs, and ahead of PyRAT.

#	Tool	Score
1	$\alpha\beta$-CROWN	1566.9
2	NeuralSAT	1430.2
3	PyRAT	1228.4
4	CORA	987.2
5	NNV	796.4
6	nnenum	740.3
7	SobolBox	593.3

Tab. 10: VNN-COMP'25 results [24].

A.2 Common Features of SOTA Tools

Feature	Supported
Network Type	Acyclic computation graphs, e.g., Feed-forward, Residual
Layer Type	FC, CNN, MaxPool, BatchNorm, Softmax
Activation Function	ReLU, Sigmoid, Tanh, Sign, Exp
Abstract Domain	Polytope, Interval
Search Algorithm	Parallel DPLL(T)
Hardware	Multi-core CPU, GPU
Optimization	Adv. Attacks, Input splitting, Large Output Opt., MILP solving
Property	Robustness, Safety
Input	Pytorch, ONNX, VNN-LIB
Output	(sat, unsat, timeout), counter-examples

Tab. 11: Common features of SOTA NNV tools.

Network Types and Activation Functions. Currently, most NNV tools work with fully connected (FC), convolutional (CNN), residual (ResNet), and batch normalization (BatchNorm) networks. Some also support mixtures of different types, e.g., VAEs which are large residual CNN-based networks [25]. In addition to ReLU, SOTA tools support other major activation functions including sigmoid, tanh, and power [24].

Standard Input and Output Formats. VNN-COMPs and major NNV tools support input networks in the standard ONNX format [2] and properties in VNNLIB format [4]. Some SOTA tools, e.g., NeuralSAT, also work directly with networks in Pytorch. The output is reported as `unsat` (property proved), `sat` (property disproved), or `unknown` / `timeout` (property cannot be proved). NNV tools also generate counterexamples for `sat` results. However, few tools generate unsat proofs, a topic of ongoing research as discussed in Sec. 8.

Engineering Optimizations. As with most high-performance analysis tools, engineering matters! SOTA NNV tools use various engineering optimizations to improve performance. For example, they employ adversarial attack techniques such as derivative-free sampling-based [26] and gradient-based [17] methods to quickly find counterexamples. They also preprocess and apply heuristics that automatically select appropriate abstractions and algorithms based on input network structures and properties [15], [20]. They use input splitting [15], [27], [28], [29] for networks with low input dimension and neuron splitting for other networks.

Commodity Hardware. NNV tools heavily leverage the power of modern hardware, including multi-core CPUs and GPUs. For example, the parallel search in NeuralSAT uses multi-threading, allowing multiple search subspaces to be processed simultaneously on different CPU cores. Abstraction techniques, e.g., the LiRPA library [30], are often implemented to run on GPUs, which significantly speeds up the computation of neuron bounds. However, real-world networks are growing rapidly in size, and we need to explore new paradigms beyond simply leveraging commodity hardware to scale verification.

Parameter Tuning and Extensibility. NNV tools are powerful and flexible and have many parameters that can be tuned to further optimize performance for specific problems. Common

parameters include decision heuristics, restart strategies, timeouts, CPU threads for parallelism, etc. NeuralSAT, in contrast to most other SOTA tools, is designed to be fully automatic so that for end-users it *"just works"* — NeuralSAT runs on all VNN-COMP benchmarks with *zero* tuning. Some tools are designed in a modular and extensible way, allowing users to easily add new heuristics, optimizations, or new search algorithms. For example, NeuralSAT consists of a small core BnB/DPLL search algorithm, and other heuristics and optimizations are implemented as independent functions that can be easily replaced or extended (e.g., in current implementation decision heuristics or restart functionalities are less than 50 SLOC).

A Formal Methods

Neural network verification (NNV) is a rising topic that applies formal methods (FM) to ML systems, particularly neural networks (NN). This chapter explains FM techniques, how verification differs from testing and debugging, and key concepts such as specifications and common FM techniques.

A.1 What Are Formal Methods (FM)?

FM are techniques for analyzing systems, e.g., computer software, using *mathematical models and reasoning*. In FM the behavior of a program is described and analyzed using logic and mathematics rather than informal reasoning or empirical testing. This allows FM to provide emph{guarantees} about system properties. For example, an FM might prove that a sorting algorithm always produces a correctly ordered list for any input list

Compared to Testing To motivate the need for FM, consider a simple function that computes the absolute value of an integer, where the goal is to show that the function always returns a non-negative number.

```
def abs_val(x):
    if x >= 0:
        return x
    else:
        return -x
```

We might *test* `abs_val` by running it on a few inputs and checking the outputs:

```
assert(abs_val(3) = 3)
assert(abs_val(-5) = 5)
assert(abs_val(0) = 0)
...
```

The program works correctly on these inputs, returning non-negative values as expected. We could run many more tests with different inputs to increase our confidence. However, we can never test all possible integers, and therefore cannot be certain that the function works correctly for all possible inputs. This is precisely where bugs can hide, and why testing—no matter how extensive—alone cannot provide absolute guarantees about program correctness, as observed by Dijkstra:

> *"Testing can only show the presence of bugs, not their absence"*
>
> **— Edsger Dijkstra**

In contrast, an FM technique would instead reason directly about the structure of the code and atempts to prove the statement:

$$\text{For all integers } x, \text{abs\_val}(x) \geq 0. \tag{152}$$

If FM can prove this statement, we have a mathematical guarantee that the function behaves correctly for *all integers*—not just the ones we used in testing.

Thus, the *fundamental difference* between FM and testing is that FM provides mathematical guarantees about program behavior—without even running the program—, while testing can only provide empirical evidence based on a finite set of inputs that we run the program on.

A.2 Specifications

At the heart of FM is a specification—a precise description of what it means for a system to be correct (or safe, or robust, etc.). A specification defines the properties we want to verify about a system.

Software engineers often write specifications informally in natural language, e.g., an algorithm should be "correct" and "fast", or a web server should be "robust against attacks". However, these informal specifications are often ambiguous and imprecise, making them vulnerable to misinterpretation and error. For example, what is correct? How fast is fast enough? How much perturbation should a system be able to handle to be considered "robust"?

Problem 1.2.1 (Sorting Specification)

Give the formal specification for a sorting function `sort(l1)` that takes a list of integers `l1` and returns a list of integers `l2`. The specification should define what it means for the output `l2` to be a correct sorting of the input `l1`.

◊

In contrast, a formal specification must be *unambiguous* and *mathematically expressible*—typically using logic. It precisely defines the conditions under which the system is considered correct.

For example, a desirable property of neural networks used for image classification is being *robust* to small perturbations of the input image. For this property, an informal specification might say:

> *"Small changes to the input image shouldn't cause the neural network to change its classification label."*

In contrast, a formal specification of robustness would instead say:

> *"For an input image x, for all images x' such that the distance between x and x' is at most ε, the output label of a network N for x' must be the same as for x."*

This statement is more precise. It defines exactly what inputs are allowed (those within distance ε of x) and exactly what behavior is required (the output label must remain the same). Typically, we go further and express this property in formal logic:

$$\forall x', \|x - x'\| \leq \varepsilon \implies N(x') = N(x) \tag{153}$$

Once having a formal specification like this, we can apply formal verification techniques to prove or disprove it.

A.2.1 Pre and Post Conditions

A common way to express formal specifications is using preconditions and postconditions. A precondition describes the assumptions about the inputs to a function, while a postcondition describes the guarantees about the outputs given that the preconditions hold. Typically, preconditions and postconditions are expressed using logical formulae and together form the specification of the form:

$$P \implies Q, \tag{154}$$

which means: *"if the precondition P holds for the inputs, then the postcondition Q must hold for the outputs"*.

Postcondition A postcondition is a logical statement that must be true after a function has executed, assuming the precondition held. It typically describes the *expected behavior* and *outcomes* of the function with respect to its inputs. For example, the postcondition for `sqrt(x)` might state that the output y must be non-negative and that $y^2 = x$. For `idiv(a, b)`, the postcondition might state that the output q must satisfy $a = b \cdot q + r$ for some remainder r such that $0 \leq r < |b|$.

We want the postcondition to be as strong as possible, i.e., it should as precisely as possible describe the expected behavior of the function. For example, for `idiv(a, b)`, a (weak) postcondition might simply state that the output is an integer, while a (strong) postcondition would specify the exact relationship between inputs and outputs as described above. Similarly, for `sqrt(x)`, a weak postcondition might state that the output is a real number, while a strong postcondition would specify the exact relationship $y^2 = x$.

Precondition A precondition is a logical statement that must be true before a function is executed. It typically lists *assumptions* and *constraints* on the inputs. For example, a `sqrt(x)` function might have the precondition that `x` must be non-negative. Similarly, an integer division function `idiv(a, b)` might have the precondition that `b` must be non-zero to avoid division-by-zero errors.

The precondition should be as weak as possible, i.e., it should not impose unnecessary restrictions or assumptions that limit the applicability of the function. Moreover, the user might not even be aware of all the assumptions needed for the function to work correctly, and therefore a strong precondition might be unrealistic. Ideally, there should be *no* precondition at all (i.e., as weak as possible), meaning the function works for all possible inputs. For example, for `sqrt(x)` we can have a weak precondition that `x` is any real number, and the postcondition would then specify that if the input is negative the output is NaN (not a number), and otherwise the output is non-negative and its square equals `x`.

Problem 1.2.1.1 (Minimum Specification)

Give a specification for a function `m = min(l)` that takes a list of integers `l` and returns the minimum integer `m` in the list. Clearly state the precondition and postcondition for the function.

◊

Problem 1.2.1.2 (Duplicate)

Give a specification for a function `l2 = remove_duplicates(l)` that takes a list of integers `l` and returns a list of integers `l2` that contains the same integers as `l` but with all duplicates removed. Clearly state the precondition and postcondition for the function.

◊

A.3 FM Techniques

At a high level, an FM technique is an algorithm, often implemented as a software tool, that analyzes a system. The FM tool takes as *inputs*:

1. A *model* of the system to be analyzed
2. A *specification* of the desired property of the system or program

and applies a *reasoning process* to determine whether the model satisfies the specification. The tool produces an *output* indicating whether the property holds, does not hold, or is unknown.

System Model The FM tool takes as input a *model* of the system to be analyzed. This model is a mathematical abstraction of the system's behavior. It may be a program written in a language such as C or Python, a neural network in ONNX format, or other representations such as state machines or transition systems. This model, e.g., the program code, is created by the programmer and is intended to capture the desired specification of the system. However, the model might contain bugs or inaccuracies that violate the intended specification, which is what the FM tool aims to detect.

Example 1.3.1 (Clip)

Consider the following simple system that "clips" an input value into the range $[0, 10]$. This system is modeled by the following Python function:

```
def clip(x):
    if x < 0: return 0
    elif x > 10: return 10
    else: return x
```

◊

The formal specification of the `clip` system is:

$$\forall x \in \mathbb{R}, \quad 0 \leq \text{clip}(x) \leq 10. \tag{155}$$

FM Reasoning Process. Once the model and specification are given, an FM tool applies a *reasoning process*—an algorithm—to determine whether the model satisfies the specification. The next section describes several major classes of FM techniques, each with its own reasoning process.

A.3.1 Major Classes of Formal Methods Algorithms

We now introduce the major classes of formal-methods techniques. In contrast to dynamic or testing-based analysis, which execute the program on specific inputs, these formal methods *statically* analyze the program's structure and logic to reason about all possible behaviors.

We will use the `clip` function (Example 1.3.1) as a running example to demonstrate how each technique reasons about the same model and specification.

Model Checking Model checking (MC) works by *exploring all possible states* of a system and checking whether *bad states* that violate the specification are reachable. At a high level, MC constructs a state machine to model the system's behavior and systematically checks whether any execution path leads to a violation of the specification.

MC works well for systems with a finite number of states—such as hardware designs and communication protocols—because these systems have finite state spaces that can be exhaustively explored. However, MC struggles with systems involving continuous variables or infinite state spaces, e.g., a program with loops.

Example 1.3.1.1

For `clip`, MC would build a state machine representing all possible execution paths of the program for different inputs. It would then check whether any path leads to a state where the output is < 0 or > 10.

A standard (naïve) MC would struggle here because the number of states is huge (the input x is a real number). So MC would need to discretize or *bound* the input space—e.g., by considering only integer values of x from -1000 to 1000—to make the reasoning more feasible.

◊

Interactive Theorem Proving An interactive theorem prover (ITP) or *proof assistant* attempts to reason about the program using *logical inference rules* to construct a formal proof that the specification holds for all possible inputs. The reason it is called "interactive" is that the user is often involved to guide the proof process by providing lemmas, definitions, and strategies to help the prover construct the proof.

ITPs are powerful and can prove deep, rich properties about programs. However, they may require significant human guidance to construct the proofs and do not scale well to large or complex systems.

Example 1.3.1.2

For `clip`, a theorem prover might reason as follows:

1. **Case 1:** If `x < 0`, `clip` returns 0, which is in $[0, 10]$.
2. **Case 2:** If `x > 10`, `clip` returns 10, which is in $[0, 10]$.
3. **Case 3:** Otherwise, $0 \leq x \leq 10$, so `clip` returns `x`, which is in $[0, 10]$.

◊

Satisfiability Checking (SAT) and Satisfiability Modulo Theories (SMT) Solving SAT and SMT solving are *automated reasoning* techniques[4]—also referred to as *automatic theorem proving*—that reduce the verification problem to a question of *logical satisfiability*. These solvers take as input a set of logical formulae representing the program and the *negation* of the specification, and determine whether there exists an assignment of variables that makes the formulae true (satisfiable) or not (unsatisfiable). If the formulae are unsatisfiable, the property is proved. If satisfiable, the property is disproved, and the solver provides a concrete counterexample demonstrating the violation.

SAT/SMT solving is crucial to modern formal methods and software verification tools due to its automation and ability to handle complex logical constraints. However, pure SAT/SMT solving might struggle with scalability (e.g., to analyze large neural networks). This leads to the development of hybrid techniques that combine SAT/SMT solving with other methods such as abstract interpretation (described below).

Example 1.3.1.3

For `clip`, a SAT/SMT solver would encode the program as logical constraints:

$$\begin{aligned}&(x < 0 \implies y = 0)\\ &\land (x > 10 \implies y = 10)\\ &\land (0 \leq x \leq 10 \implies y = x)\end{aligned} \tag{156}$$

and the negation of the specification:

$$(y < 0) \lor (y > 10) \tag{157}$$

It would then check whether these constraints are satisfiable. In this case, the solver would find that the constraints are *unsatisfiable*, proving that `clip` always returns a value in $[0, 10]$.

◊

Abstract Interpretation Abstract interpretation (AbsInt) analyzes a program by automatically computing *over-approximations* of its behavior. Instead of tracking exact values, AbsInt tracks abstract properties such as ranges or signs of variables—e.g., instead of saying "`x = 5`", AbsInt might say "`x` $\in [0, 10]$" or "`x` is non-negative".

Because AbsInt is fully automated and uses over-approximations, it is efficient and scales well to large programs. However, over-approximation comes at the cost of precision: it can cause AbsInt to produce false positives by reporting potential violations that are not real bugs.

[4] *Automated reasoning* (AR) is the subfield of CS focused on algorithms that draw logical conclusions automatically, of which SAT/SMT solvers are a key example. FM is one major application of AR: it uses these reasoning engines to verify real systems against formal specifications.

Example 1.3.1.4

For `clip`, an AbsInt tool might reason:

1. Input `x` is in $(-\infty, +\infty)$ and output `y` is unknown initially.
2. After the `if x < 0` branch, output `y` is in $[0, 0]$.
3. After the `if x > 10` branch, output `y` is in $[10, 10]$.
4. After the else branch, output `y` is in $[0, 10]$.
5. Combining all branches, output `y` is in $[0, 10]$.

◊

For neural network verification, AbsInt is often used to compute bounds on the outputs of neural network layers given bounds on the inputs (Sec. 5). AbsInt is necessary because neural networks are very large and have complex nonlinear functions such as ReLU that are difficult to analyze exactly.

A.3.2 Soundness and Completeness in FM

Two core concepts in FM are *soundness* and *completeness*. These describe the correctness and power of an FM algorithm.[5]

Definition 1.3.2.1

An FM algorithm is **sound** if every property it claims to prove is actually true. In other words, if it says *"this property holds in this system,"* then the property truly holds. Conversely, an FM algorithm is *not* sound if it can claim that a false property is true.

♣

Definition 1.3.2.2

An FM algorithm is **complete** if it can prove every true property. That is, if the property holds, the verifier will always be able to prove it. Conversely, an FM algorithm is **incomplete** if there exist true properties that it cannot prove.

♣

Some notes:

- While a sound FM algorithm never claims a false property is true, it *may* claim that a true property is false (i.e., it can give false counterexamples). Thus, an extremely conservative FM algorithm that claims every property is false is still sound. Similarly, an FM algorithm that returns "unknown" for every property is also sound.

  Similarly, while a complete FM algorithm can prove every true property, it may also claim that a false property is true (i.e., it can produce false proofs). Thus, an extremely reckless (and untrustworthy) FM algorithm that claims every property is true is still complete. An FM algorithm that returns "unknown" for every property is also complete.

[5] Mike Hicks wrote a good blog post on this topic: http://www.pl-enthusiast.net/2017/10/23/what-is-soundness-in-static-analysis/.

- It is important to distinguish between an FM algorithm and its implementation (the software tool). An FM algorithm may be sound (or complete), but its implementation—typically written by humans (or AI nowadays)—may contain bugs that cause it to produce unsound (or incomplete) results.

Ideally, an FM algorithm should be both sound and complete: never proving a false property and able to prove every true property. However, achieving both is often computationally infeasible for complex systems. Between the two, soundness is typically prioritized in safety-critical settings because it is better to be unsure about a system (and say "unknown", or even claim that it is unsafe) than to incorrectly claim that it is safe when it is not. Thus, most practical FM algorithms are designed to be *sound but incomplete*.

B Logics

The topic of neural network verification (NNV) relies heavily on two fundamental areas: logic (to formalize and reason about properties as logical formulae) and optimization (to formulate and reason about numerical constraints). This chapter provides the necessary background on both topics in detail, starting with propositional logic and satisfiability, and then moving to linear and mixed-integer programming.

B.1 Propositional Logic and Satisfiability

Propositional, or Boolean, logic is the branch of logic that studies formulae built from Boolean variables, where each variable can take only the values `True` or `False`. These formulae are combined using logical connectives such as `and`, `or`, and `not`, and their evaluation always reduces to a single truth value.

B.1.1 Syntax

A *propositional variable* is a symbol (e.g., x, y, z). Logical formulae are built *inductively* from these variables using logical connectives as follows:

- **Variables:** Any propositional variable p is a formula.
- **Constants:** $\top$ and $\bot$ are formulae.
- **Negation:** If φ is a formula, then so is $\neg\varphi$.
- **Binary connectives:** If φ, ψ are formulae, then so are $(\varphi \wedge \psi)$, $(\varphi \vee \psi)$, and $(\varphi \Rightarrow \psi)$.

Example 2.1.1.1

$$(x \vee y) \wedge (\neg x \vee z) \tag{158}$$

is a formula involving three variables x, y, z. ◊

Special connectives. In addition to the standard connectives, we define some special connectives that are often useful:

- *Implication*: $\varphi \to \psi \equiv \neg\varphi \vee \psi$
- *Biconditional* (or *equivalence*): $\varphi \leftrightarrow \psi \equiv (\varphi \to \psi) \wedge (\psi \to \varphi)$

Notice here that we purely define the *syntax* of propositional logic, which allows us to construct valid formulae. We will discuss the *semantics* or meanings of formulae in Sec. B.1.2.

Conjunctive Normal Form (CNF). A formula is in CNF if it is a *conjunction* of *clauses*, where each clause is a *disjunction* of *literals*, where a literal is either a variable p or its negation $\neg p$.

Example 2.1.1.2

The formula

$$(x \vee y) \wedge (\neg x \vee z) \tag{159}$$

is in CNF. Each clause $(x \vee y)$, $(\neg x \vee z)$ is a disjunction of literals.

◊

Transforming to CNF. Any formula can be transformed into CNF. The transformation process generally involves the following steps:

1. Eliminate biconditionals and implications.
2. Move negations inward using De Morgan's laws.
3. Distribute disjunctions over conjunctions.

Problem 2.1.1.3

Transform the following formula into CNF:

$$(x \to y) \wedge (y \to z) \tag{160}$$

◊

B.1.2 Semantics

A logical formula in propositional logic has a truth value, which can be either `True` or `False`. For example, the formula $x \vee \neg x$ is always `True`, while the formula $x \wedge y$ is `True` if both x and y are `True`, and `False` otherwise. This is the *semantics* of the formula.

An *assignment* σ, which is a mapping from propositional variables to truth values, evaluates each formula to `True` or `False`. The truth value of a propositional formula over the connectives $\top, \bot, \neg, \wedge, \vee, \Rightarrow$ under an assignment σ is defined recursively:

$$\begin{aligned}
\sigma(\top) &= \text{True}, \\
\sigma(\bot) &= \text{False}, \\
\sigma(\neg\varphi) = \text{True} \quad &\text{iff} \quad \sigma(\varphi) = \text{False}, \\
\sigma(\varphi \wedge \psi) = \text{True} \quad &\text{iff} \quad \sigma(\varphi) = \text{True and } \sigma(\psi) = \text{True}, \\
\sigma(\varphi \vee \psi) = \text{True} \quad &\text{iff} \quad \sigma(\varphi) = \text{True or } \sigma(\psi) = \text{True}, \\
\sigma(\varphi \Rightarrow \psi) = \text{True} \quad &\text{iff} \quad \sigma(\varphi) = \text{False or } \sigma(\psi) = \text{True}.
\end{aligned} \tag{161}$$

Problem 2.1.2.1

For the formula

$$(x \vee y) \wedge (\neg x \vee z), \tag{162}$$

evaluate its truth value under all $2^3 = 8$ possible assignments of x, y, z.

◊

B.2 Satisfiability and Validity

Definition 2.2.1 (Satisfiability)

A formula φ

- is *satisfiable* if there exists **some** assignment σ that satisfies it, i.e., $\exists \sigma.\ \sigma(\varphi) = \text{True}$.
- is *unsatisfiable* if **no** assignment satisfies it, i.e., $\forall \sigma.\ \sigma(\varphi) = \text{False}$.
- is *valid* if **all** assignments satisfy it, i.e., $\forall \sigma.\ \sigma(\varphi) = \text{True}$.

♣

Example 2.2.2

The formula p is satisfiable (i.e., $\exists \sigma.\ \sigma(p) = \text{True}$).

The formula

$$(p) \wedge (\neg p) \tag{163}$$

is unsatisfiable (no assignment works). In contrast,

$$(p \vee \neg p) \tag{164}$$

is valid.

◊

Problem 2.2.3

Show that $(\varphi \Rightarrow \psi)$ is equivalent to $(\neg\varphi \vee \psi)$. (Hint: construct a truth table.)

◊

Problem 2.2.4

Assume p, q are boolean variables.

1. Can a formula be satisfiable but not valid? If yes, provide an example of such a formula over p, q.
2. Can a formula be valid but not satisfiable? If yes, provide an example of such a formula over p, q.
3. Give a formula over p, q that is satisfiable under exactly 1 assignment.
4. Let φ be a formula over p, q. If φ is valid, how many assignments satisfy it?

◊

B.2.1 SAT Checking

A *SAT solver* takes a propositional formula, typically in CNF, as input and determines its satisfiability by searching for a satisfying assignment σ that makes the formula true. If one exists it returns SAT (and optionally σ); otherwise it reports unsat.

Example 2.2.1.1

$$\begin{aligned} \varphi_1 &= (x \vee y) \wedge (\neg x \vee z) \quad \Rightarrow \text{sat} \\ \varphi_2 &= (x) \wedge (\neg x) \quad \Rightarrow \text{unsat} \end{aligned} \tag{165}$$

◊

Problem 2.2.1.2

Use the Z3 solver to check whether

$$(x \vee y) \wedge (\neg x \vee y) \wedge (x \vee \neg y) \tag{166}$$

is satisfiable. Provide a satisfying assignment if it exists.

◊

B.2.2 Validity Checking

A formula α is valid if all assignments satisfy it. This is equivalent to its negation $\neg\alpha$ being unsatisfiable:

$$\text{valid}(\alpha) \iff \text{unsat}(\neg\alpha). \tag{167}$$

Thus, to check that α is valid, we can use a SAT solver to check that $\neg\alpha$ is UNSAT.

B.2.3 Complexity of SAT

SAT checking (and therefore validity checking) of propositional formulae is NP-Complete. This means it is unlikely that an efficient (polynomial-time) algorithm exists for all SAT instances, and in the worst case any SAT solving algorithm may need to explore an exponential number of assignments. SAT solvers therefore employ heuristics and optimizations to handle practical instances efficiently, even though worst-case complexity remains exponential. We discuss DPLL, a popular SAT solving algorithm, next.

B.3 Satisfiability Modulo Theories (SMT)

SAT solvers only reason about propositional formulae, which contain only Boolean variables and logical operators such as $\wedge, \vee$. However, in verification problems we often need to reason about arithmetic constraints such as $2x + 3y \leq 7$, $x \geq 0$, $x, y \in \mathbb{Z}$.

This leads to *Satisfiability Modulo Theories (SMT)*:

- SMT generalizes propositional logic by adding background theories (e.g., linear arithmetic, bit-vectors, arrays).
- An SMT formula mixes Boolean structure with constraints from these theories.

Example 2.3.1

Is the formula $2x = 1$ satisfiable?

The answer depends on the chosen theory. Over the reals ($\mathbb{R}$) it is SAT (e.g., $x = 0.5$), but over the integers ($\mathbb{Z}$) it is UNSAT.

◊

B.4 SMT Solvers

Popular SMT solvers include Z3 (Microsoft Research), CVC5, and dReal. They combine:

- A SAT solver for the Boolean skeleton.
- A *theory solver* or T-solver (e.g., linear arithmetic) for the numeric parts.

The SAT solver guesses a truth assignment for the Boolean structure. The T-solver checks whether the corresponding arithmetic constraints are feasible. This loop continues until either a satisfying assignment is found or UNSAT is proven.

Problem 2.4.1

Decide the satisfiability of:

$$(x > 3) \land (y < 2) \land (x + y < 1). \tag{168}$$

First reason informally by hand, then confirm with an SMT solver such as Z3.

◊

B.4.1 Z3 SMT Solver

Z3 is a well-known SMT solver developed by Microsoft Research. Here we show how to use it to check propositional formulae and SMT formulae involving linear arithmetic constraints.

Installing. You can install Z3 using your package manager:

```
brew install z3          # Homebrew
pip install z3-solver    # pip
apt install z3 python3-z3  # apt (Debian-based Linux)
conda install z3solver   # conda
```

Then try its Python interface:

```
from z3 import *
x = Int('x')
y = Int('y')
solve(x > 2, y < 10, x + 2*y = 7)
```

Example 2.4.1.1 (Propositional Logic)

We use Z3 to check satisfiability of propositional formulae and generate counterexamples.

```
from z3 import *

x, y = Bools('x y')
formula = And(Or(x, y), Or(Not(x), y))
s = Solver()
s.add(formula)
print(s.check())   # output: sat
print(s.model())   # a possible model is {x=True, y=True}
```

◊

Problem 2.4.1.2

Modify the code in the propositional logic example above to check

$$(x \wedge \neg x). \tag{169}$$

Confirm that the result is `unsat`.

◊

Example 2.4.1.3 (Linear Constraints)

We now use Z3 as an SMT solver to check linear constraints.

```
from z3 import *
x, y = Reals('x y')

# example 1
s = Solver()
s.add(x > 0, y > 0, 2*x + y <= 1)
print(s.check())  # Output: unsat

# example 2
s = Solver()
s.add(x + y <= 4, x >= 0, y >= 0)
print(s.check())  # Output: sat
print(s.model())  # Output: {x=0, y=0}

# example 3
x, y = Ints('x y')
s = Solver()
s.add(Implies(x > 3, y > 2))
print(s.check())  # Output: sat
print(s.model())  # Output: {x=0, y=0}
```

◊

Problem 2.4.1.4

Use Z3 to check whether the constraints

$$x + y = 5, \quad x \geq 0, \quad y \geq 0 \tag{170}$$

are satisfiable, and if so, ask Z3 for a model.

◊

Problem 2.4.1.5

Use Z3 to check the formula given in the SMT example above. Do it for both the reals and integers.

◊

Problem 2.4.1.6

Use Z3 to formulate and check the statement *"If* $x > 3$ *then* $y > 2$ *AND* $x \leq 1$*"*. Is it satisfiable? What model does Z3 return?

◊

Example 2.4.1.7 (DNN-style checking)

Consider a one-neuron ReLU: $y = \max(0, x - 1)$. We want to check if the property *"For all* $x \leq 0$*, we have* $y = 0$*"* holds.

```
x, y = Reals('x y')
s = Solver()

# y = max(0, x-1)
s.add(y >= x-1, y >= 0)
s.add(Or(y = x-1, y = 0))  # ReLU
s.add(x <= 0, y != 0)        # Negation of property

print(s.check())  # Output: unsat, property holds
```

◊

Problem 2.4.1.8

Modify the above to check the property: *"For all* $x \geq 2$*, we have* $y \geq 1$*."* What does Z3 return?

◊

C SAT Solving Algorithms

C.1 DPLL

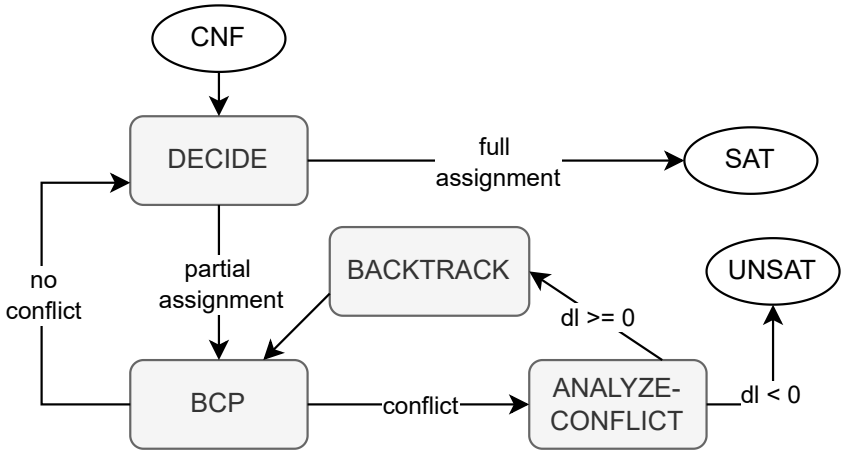


Fig. 34: The DPLL algorithm for SAT solving.

Fig. 34 gives an overview of the **DPLL** algorithm—a SAT solving technique introduced in 1961 by Davis, Putnam, Logemann, and Loveland. DPLL is an iterative algorithm that takes as input a propositional formula and (i) decides an unassigned variable and assigns it a truth value, (ii) performs Boolean constraint propagation (BCP, also called Unit Propagation), which detects single-literal clauses that either force a literal to be true or give rise to a conflict, (iii) analyzes the conflict to backtrack to a previous decision level `dl`, and (iv) erases assignments at levels greater than `dl` to try new assignments.

These steps repeat until DPLL finds a satisfying assignment and returns `sat`, or decides that it cannot backtrack (`dl = -1`) and returns `unsat`.

C.1.1 Decide

The *Decide* step selects an unassigned variable and assigns it a truth value. Decision is the main source of state-space explosion in DPLL because it uses random choices or heuristics (e.g., VSIDS) to select the variable and truth value. Thus, value assignments can be *wrong*, leading to conflicts later on.

In addition, each decision creates a new *decision level*. The decision level starts at 0 and increases by 1 with each new assignment. Decision level is used to manage backtracking and erasing of assignments when conflicts occur.

Example 3.1.1.1 (Simple Decision)

Consider the CNF:

$$\varphi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3). \tag{171}$$

All variables are initially unassigned. DPLL may choose:

$$\text{Decide: } x_1 = \text{True} \quad (\text{Decision level } 1). \tag{172}$$

◊

Example 3.1.1.2 (Decision Using a Heuristic)

Given:

$$\varphi = (x_1 \vee x_3) \wedge (x_3 \vee x_4) \wedge (\neg x_3 \vee x_2), \tag{173}$$

a heuristic such as VSIDS may choose the most frequent variable:

$$\text{Decide:} \quad x_3 = \text{False}. \tag{174}$$

◊

C.1.2 Boolean Constraint Propagation (BCP)

BCP detects *unit clauses*, i.e., clauses with exactly one unassigned literal and all other literals set to False. Such clauses force the remaining literal to be assigned True.

BCP is very useful because it can determine variable assignments directly without guessing (which is Decide as described in Sec. C.1.1). For example, if we have the clause $(\neg x_1 \vee x_2)$ and x_1 is True, then BCP forces x_2 to be True to satisfy the clause.

BCP is often invoked after each decision to propagate the consequences of the assignment.

Example 3.1.2.1 (Single Propagation)

$$\varphi = (x_1) \wedge (\neg x_1 \vee x_2). \tag{175}$$

The unit clause (x_1) forces $x_1 = \text{True}$. Now $(\neg x_1 \vee x_2)$ becomes $(\text{False} \vee x_2)$, hence $x_2 = \text{True}$.

◊

Example 3.1.2.2 (Propagation Leading to Conflict)

$$\varphi = (x_1) \wedge (\neg x_1). \tag{176}$$

BCP assigns $x_1 = \text{True}$, immediately contradicting $(\neg x_1)$.

◊

Example 3.1.2.3 (Cascade of Propagations)

$$\varphi = (x_1) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3). \tag{177}$$

BCP yields:

$$x_1 = \text{True} \Rightarrow x_2 = \text{True} \Rightarrow x_3 = \text{True}. \tag{178}$$

◊

C.1.3 Conflict Analysis and Backtracking

A *conflict* occurs when the formula is False under the current assignment σ. Since the formula is in CNF, a conflict arises when at least one clause is False (i.e., all its literals are False).

Example 3.1.3.1

$$\varphi = (\neg x_1 \vee x_2) \wedge (\neg x_2) \wedge (x_1). \tag{179}$$

Assignments $x_1 = \text{True}$ and $x_2 = \text{False}$ make $(\neg x_1 \vee x_2) = (\text{False} \vee \text{False})$, producing a conflict.

◊

Conflict Analysis. Conflict analysis explains why the conflict occurred and generates a *learned clause* that prevents the solverfrom revisiting the same conflicting assignment. Most modern solvers use the 1st-UIP (Unique Implication Point) learning scheme.

Example 3.1.3.2 (Simple Learned Clause)

$$(x_1) \wedge (\neg x_1). \tag{180}$$

The conflict directly yields the learned clause $\neg x_1 \vee x_1$. Because this is always false, the solver concludes the formula is unsatisfiable.

◊

Example 3.1.3.3 (UIP Conflict Analysis)

Assume decision levels:

$$x_1 = \text{True} \ \ (dl = 1), \quad x_2 = \text{False} \ \ (dl = 2). \tag{181}$$

Clauses:

$$(\neg x_1 \vee x_2 \vee x_3), \quad (\neg x_2 \vee x_3), \quad (\neg x_3). \tag{182}$$

Propagation yields $x_3 = \text{False}$, causing a conflict with $(\neg x_3)$. The learned clause (1st-UIP) is:

$$(\neg x_2 \vee \neg x_1). \tag{183}$$

◊

Backtracking. Given a learned clause, DPLL backtracks to the decision level at which the clause becomes unit.

Example 3.1.3.4 (Non-Chronological Backtracking)

Decision levels:

$$dl = 1 : x_1 = \text{True}, \quad dl = 2 : x_2 = \text{True}, \quad dl = 3 : x_3 = \text{False}. \tag{184}$$

Conflict analysis produces the learned clause $(\neg x_1 \vee x_3)$. The highest decision level in the clause except the most recent UIP is 1, so the solver backtracks to level 1, undoing assignments to x_2 and x_3.

◊

C.2 CDCL

Modern DPLL solving improves the original version with Conflict-Driven Clause Learning (CDCL) [31], [32], [33]. DPLL with CDCL *learns new clauses* to avoid past conflicts and backtrack more intelligently (e.g., using non-chronological backjumping). Due to its ability to learn new clauses, CDCL can significantly reduce the search space and allow SAT solvers to scale to large problems.

In the following, whenever we refer to DPLL hereafter, we mean DPLL with CDCL.

C.3 DPLL(T)

DPLL(T) [34] extends DPLL for propositional formulae to check SMT formulae involving non-Boolean variables, e.g., real numbers and data structures such as strings, arrays, and lists. DPLL(T) combines DPLL with dedicated *theory solvers* to analyze formulae in those theories.[6] For example, to check a formula involving linear arithmetic over the reals (LRA), DPLL(T) may use a theory solver that applies linear programming to check the constraints in the formula.

Modern DPLL(T)-based SMT solvers such as Z3 [8] and CVC4 [35] include solvers supporting a wide range of theories including linear arithmetic, nonlinear arithmetic, strings, and arrays [36].

[6] SMT stands for Satisfiability Modulo Theories; the T in DPLL(T) stands for Theories.

D Linear Programming (LP)

Linear Programming (LP) and its generalization Mixed-Integer Linear Programming (MILP) form the optimization backbone of many NNV techniques. This chapter provides an overview of LP and MILP in the context of NNV.

D.1 Linear Constraints and Objectives

At a high level, LP is a method for optimizing an objective with respect to certain constraints. In LP, both the objective and the constraints are *linear*.

Linear Constraints. Linear constraints are inequalities involving linear combinations of variables. A linear inequality has the general form

$$c_1 x_1 + c_2 x_2 + \ldots + c_n x_n \leq b, \tag{185}$$

where $c_i, b \in \mathbb{R}$. These constraints limit the feasible values the variables x_i can take. The set of points satisfying all constraints is called the *feasible region*.

Example 4.1.1

Consider the following linear constraints:

$$x + y \leq 4, \quad x \geq 0, \quad y \geq 0. \tag{186}$$

The feasible region is a triangle with vertices at $(0,0)$, $(4,0)$, $(0,4)$.

◊

Linear Objective Functions. A linear objective function z has the general form

$$z(x_1, x_2, \ldots, x_n) = c_1 x_1 + c_2 x_2 + \ldots + c_n x_n, \tag{187}$$

where $c_i \in \mathbb{R}$ are coefficients and $x_i \in \mathbb{R}$ are decision variables.

Optimizing Goals. The goal is to optimize (maximize or minimize) z subject to a set of linear constraints:

$$\begin{aligned} &\text{maximize} \quad z \\ &\text{subject to} \\ &\quad \{c_1 x_1 + \ldots + c_n x_n \leq b, \ldots\} \end{aligned} \tag{188}$$

Example 4.1.2

Maximize $1.5x + 2y$ subject to $x + y \leq 4$, $x \geq 0$, $y \geq 0$.

Solving for the intersection points gives the vertices of the feasible region: $(0, 0)$, $(4, 0)$, $(0, 4)$.

Evaluating the objective at each vertex:

x	y	$z = 1.5x + 2y$
0	0	0
4	0	6
0	4	8

The maximum value of z is 8, achieved at vertex $(0, 4)$.

◊

Example 4.1.3

Minimize $z = x + y$ subject to $2x + y \geq 3$, $x + 2y \geq 4$, $x \geq 0$, $y \geq 0$.

Solving the boundary equations gives the vertices of the feasible region: $(0, 3)$, $(4, 0)$, $\left(\frac{2}{3}, \frac{5}{3}\right)$.

Evaluating the objective at each vertex:

x	y	$z = x + y$
0	3	3
4	0	4
$\frac{2}{3}$	$\frac{5}{3}$	$\frac{7}{3}$

The minimum value of z is $\frac{7}{3}$, achieved at $\left(\frac{2}{3}, \frac{5}{3}\right)$.

◊

Problem 4.1.4

Maximize $z = 4x + 5y$ subject to $x + y \leq 20$, $2x + 4y \leq 72$.

1. Identify the corner points.
2. Evaluate the objective function at each corner point.

◊

D.2 Mixed-Integer Linear Programming (MILP)

LP assumes continuous variables over real numbers. A mixed-integer linear program—MILP—extends LP by requiring some variables to be integers (often binary 0 or 1). This is useful for modeling discrete decisions and logical constraints, e.g., on/off decisions, yes/no choices, and active/inactive ReLU.

Linear Constraints. In MILP, a linear constraint can involve both integer and continuous decision variables:

$$c_1x_1 + ... + c_nx_n + d_1y_1 + ... + d_my_m \leq b, \tag{189}$$

where $x_i \in \mathbb{R}$ are continuous variables, $y_j \in \mathbb{Z}$ are integer variables, and $c_i, d_j, b \in \mathbb{R}$.

Objective Function. A linear objective function in MILP:

$$z = c_1x_1 + ... + c_nx_n + d_1y_1 + ... + d_my_m. \tag{190}$$

Optimizing Goals.

$$\begin{aligned} &\text{maximize} \quad z(x_1, ..., x_n, y_1, ..., y_m) \\ &\text{subject to} \\ &\quad \{c_1x_1 + ... + c_nx_n + d_1y_1 + ... + d_my_m \leq b\} \end{aligned} \tag{191}$$

Example 4.2.1

A company sells a chair for \$50 and a table for \$120. Production costs are \$20 per chair and \$70 per table. It takes 3 hours and 5 units of wood to produce a chair, and 1 hour and 20 units of wood to produce a table.

Maximize profit without exceeding a monthly budget of 200 units of wood, 80 hours of labor, and \$800 in production costs.

Decision variables: x = number of chairs, y = number of tables.

Objective:

$$P = 50x + 120y - 20x - 70y = 30x + 50y. \tag{192}$$

Constraints:

$$\begin{aligned} 5x + 20y \leq 200 \quad &\Rightarrow \quad x + 4y \leq 40, \\ 3x + y \leq 80, \quad & \\ 20x + 70y \leq 800 \quad &\Rightarrow \quad 2x + 7y \leq 80, \\ x \geq 0, \quad y \geq 0. \quad & \end{aligned} \tag{193}$$

The feasible vertices are $(0, 0)$, $(0, 10)$, $\left(\frac{80}{3}, 0\right)$, and $\left(\frac{280}{11}, \frac{40}{11}\right)$. Evaluating P:

x	y	$P = 30x + 50y$
0	0	0
0	10	500
$\frac{80}{3}$	0	800
$\frac{280}{11}$	$\frac{40}{11}$	945.45

The maximum is \$945.45 at $\left(\frac{280}{11}, \frac{40}{11}\right)$. Since fractional production is not possible, the company should produce 24 chairs and 4 tables per week for a profit of \$920.00.

D.2.1 Encoding Binary Variables

MILP problems often involve conditions where the answer *depends* on some condition. We can encode such logical implications using *binary variables* ($z \in \{0, 1\}$) and a *large constant* M (e.g., ∞).

Example 4.2.1.1

To encode "if $z = 1$ then $x \geq 5$", use:

$$x \geq 5 - M(1 - z), \quad z \in \{0, 1\}. \tag{194}$$

This works because:

- $z = 1 \Rightarrow x \geq 5 - M(0) \Rightarrow x \geq 5$.
- $z = 0 \Rightarrow x \geq 5 - M$, which is vacuously true for large M.

◊

Problem 4.2.1.2

Encode the disjunction $x \geq 5 \vee x \leq 3$ in MILP.

◊

Solution. Use two constraints:

- $x \geq 5 - M(1 - z)$ (if $z = 1$, this enforces $x \geq 5$)
- $x \leq 3 + Mz$ (if $z = 0$, this enforces $x \leq 3$)

Example 4.2.1.3 (Encoding ReLU in MILP)

To encode the ReLU activation function $y = \max(0, x)$, use:

$$\begin{aligned} &y \geq x, \\ &y \geq 0, \\ &y \leq x + M(1 - z), \\ &y \leq Mz, \\ &z \in \{0, 1\}. \end{aligned} \tag{195}$$

This works because $z = 1 \Rightarrow y = x$ and $z = 0 \Rightarrow y = 0$.

◊

Problem 4.2.1.4

Consider again the ReLU encoding above but now x has bounds $L \leq x \leq U$. Modify the MILP encoding so that L and U are used *directly* instead of a large constant M.

◊

Solution.

$$
\begin{aligned}
&y \geq x, \\
&y \geq 0, \\
&y \leq x + (U - x)(1 - z), \\
&y \leq Uz, \quad z \in \{0, 1\}, \\
&L \leq x \leq U.
\end{aligned} \tag{196}
$$

D.3 Using Z3 to Solve LP and MILP

We can use Z3's optimization capabilities to solve both LP and MILP problems. In practice, dedicated solvers such as Gurobi or CPLEX are preferred for large problems, but Z3 is effective for demonstration purposes.

Example 4.3.1

We solve the LP example from Example 4.1.2 using Z3:

```
from z3 import *

x, y = Reals("x y")
opt = Optimize()
opt.add(x + y <= 4, x >= 0, y >= 0)

z = 1.5*x + 2*y
h = opt.maximize(z)

if opt.check() = sat:
    print("Optimal sol:", opt.model())  # [x = 0, y = 4]
    print("Max value:", opt.upper(h))   # 8
```

◊

Problem 4.3.2

Solve Problem 4.1.4 using Z3.

◊

Problem 4.3.3

Solve Example 4.2.1 using Z3.

◊

Problem 4.3.4

Find two interesting MILP problems online, formulate each as a MILP, solve by hand (showing all steps), and implement in Z3.

For each problem: write down the problem statement and cite the source, clearly state the objective and constraints, and document the Z3 code with comments.

◊

Problem 4.3.5

Minimize $z = x + y$ subject to:

$$x + 2y \geq 3, \quad 3x + y \geq 3, \quad x, y \geq 0, \quad x, y \in \mathbb{R}. \tag{197}$$

Remember to:

1. State the constraints and objective function.
2. Find the corner or candidate points.
3. Evaluate the objective at each corner point.
4. State the optimal solution.

◊

D.3.1 Using LP as Feasibility Checking

In addition to optimization, LP can be used for *feasibility checking* of linear constraints: are there any values for the variables that satisfy all the constraints? This is achieved by running the LP solver with a constant objective (e.g., "minimize 0"), in which case it tries to find *any* point in the feasible region or show that *none* exists. This makes LP solving useful for property checking and verification.

The main difference between LP feasibility and SMT satisfiability checking is the type of constraints they handle. SMT supports richer logics involving booleans, integers, nonlinear arithmetic, and other theories, while LP feasibility is limited to linear equalities/inequalities over real numbers. For NNV problems that involve only linear real arithmetic, LP feasibility is sufficient and often more efficient.

Example 4.3.1.1

For the constraints $\{x + y \leq 4, x \geq 0, y \geq 0\}$, running an LP solver with objective $\min 0$ will return a point (e.g., $x = 0, y = 0$) within the feasible region.

◊

E Programming Assignments

E.1 PA1

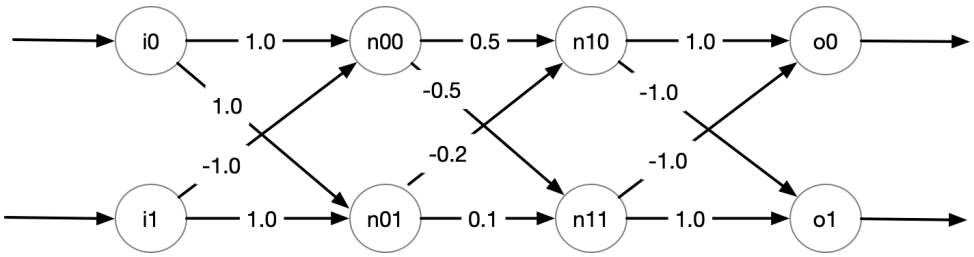
In this PA you will implement NNV using symbolic execution (SE) as described in Sec. 4.1. You can consider neural networks as a specific type of programs and thus can "execute" it. SE runs the network on symbolic inputs and returns symbolic outputs, i.e., a constraint representation. You will represent the symbolic outputs as a logical formula and use a constraint solver to check *assertions*, i.e., properties about the network. This PA has two parts: (1) implement SE on a given a network, and (2) evaluate the scalability of your SE on various networks.

You will use Python and the Z3 SMT solver (Sec. B.4.1) for this assignment. **Do not** use external libraries or any extra tools. **Do not** use AI tools (e.g., ChatGPT, GitHub Copilot, etc.) to generate code for this assignment, the purpose of this PA is for you to understand and implement symbolic execution yourself.

Finally, you will need to answer some questions about your implementation and findings in the provided `README` template. https://github.com/dynaroars/book_nnv/blob/main/pa/templates/README_PA1.md

E.1.1 Part 1: Symbolic Execution

Consider the following fully-connected network with 2 inputs, 2 hidden layers, and 2 outputs.



1. **DNN Encoding**: We can encode this DNN using Python:

```
# (weights, bias, use activation function relu or not)
n00 = ([1.0, -1.0], 0.0, True)
n01 = ([1.0, 1.0], 0.0, True)
hidden_layer0 = [n00, n01]

n10 = ([0.5, -0.2], 0.0, True)
n11 = ([-0.5, 0.1], 0.0, True)
hidden_layer1 = [n10, n11]

# don't use relu for outputs
o0 = ([1.0, -1.0], 0.0, False)
o1 = ([-1.0, 1.0], 0.0, False)
output_layer = [o0, o1]

dnn = [hidden_layer0, hidden_layer1, output_layer]
```

   In this network, the outputs of the neurons in the hidden layers (prefixed with `n`) are applied with the ReLU activation function, but the outputs of the network (prefixed with `o`) are not. These settings are controlled by the `True`, `False` parameters as shown above. Also, this example does not use `bias`, i.e., bias values are all 0.0's as shown.

   Note that all of these settings are parameterized and I deliberately use this example to show how these parameters are used (e.g., ReLU only applies to hidden neurons, but not outputs).

2. **Symbolic States**: After performing symbolic execution on the network, we obtain *symbolic states*, represented by a logical formula relating input and output variables.

```
# my_symbolic_execution is something you implement,
# it returns a single (but large) formula representing the symbolic states.
symbolic_states = my_symbolic_execution(dnn)
...
"done, obtained symbolic states for DNN in 0.1s"
assert z3.is_expr(symbolic_states)  # symbolic_states is a Z3 expression

# your symbolic states should look something like this
```

```
print(symbolic_states)
# And(n0_0 == If(i0 + -1*i1 <= 0, 0, i0 + -1*i1),
#     n0_1 == If(i0 + i1 <= 0, 0, i0 + i1),
#     n1_0 ==
#     If(1/2*n0_0 + -1/5*n0_1 <= 0, 0, 1/2*n0_0 + -1/5*n0_1),
#     n1_1 ==
#     If(-1/2*n0_0 + 1/10*n0_1 <= 0, 0, -1/2*n0_0 + 1/10*n0_1),
#     o0 == n1_0 + -1*n1_1,
#     o1 == -1*n1_0 + n1_1)
```

3. **Constraint Solving**: We will use Z3 to query various things about this network from the obtained symbolic states. Examples include:

   (a) Generating random inputs and obtain outputs. Note that these are random values (that satisfy the symbolic states), so your results might look different.

```
z3.solve(symbolic_states)
[n0_1 = 15/2,
    o1 = 1/2,
    o0 = -1/2,
    i1 = 7/2,
    n1_1 = 1/2,
    n1_0 = 0,
    i0 = 4,
    n0_0 = 1/2]
```

   (b) Simulating a concrete execution.

```
i0, i1, n0_0, n0_1, o0, o1 = z3.Reals("i0 i1 n0_0 n0_1 o0 o1")

# finding outputs when inputs are fixed [i0 == 1, i1 == -1]
g = z3.And([i0 == 1.0, i1 == -1.0])
z3.solve(z3.And(symbolic_states, g))  # you should get o0, o1 = 1, -1
[n0_1 = 0,
o1 = -1,
o0 = 1,
i1 = -1,
n1_1 = 0,
n1_0 = 1,
i0 = 1,
n0_0 = 2]
```

   (c) Checking assertions, i.e., verifying/proving properties about the network or disproving/finding counterexamples.

```
print("Prove that if (n0_0 > 0.0 and n0_1 <= 0.0) then o0 > o1")
g = z3.Implies(z3.And([n0_0 > 0.0, n0_1 <= 0.0]), o0 > o1)
print(g)  #  Implies(And(n0_0 > 0, n0_1 <= 0), o0 > o1)
z3.prove(z3.Implies(symbolic_states, g))  # proved

print("Prove that when (i0 - i1 > 0 and i0 + i1 <= 0), then o0 > o1")
g = z3.Implies(z3.And([i0 - i1 > 0.0, i0 + i1 <= 0.0]), o0 > o1)
print(g)  # Implies(And(i0 - i1 > 0, i0 + i1 <= 0), o0 > o1)
z3.prove(z3.Implies(symbolic_states, g))
# proved
```

```
print("Disprove that when i0 - i1 > 0, then o0 > o1")
g = z3.Implies(i0 - i1 > 0.0, o0 > o1)
z3.prove(z3.Implies(symbolic_states, g))
# counterexample (you might get different counterexamples)
# [n0_1 = 15/2,
# i1 = 7/2,
# o0 = -1/2,
# o1 = 1/2,
# n1_0 = 0,
# i0 = 4,
# n1_1 = 1/2,
# n0_0 = 1/2]
```

E.1.2 Tips

1. You should apply symbolic execution on this example **by hand** first before attempting any coding. This example is small enough that you can work out step by step. For example, you can do these steps:
    1. First, obtain the symbolic states by hand (e.g., on paper) for the given network.
    2. Then, put what you have in code but also for the given network. Use Z3 format (e.g., declare the inputs as symbolic values `i0 = z3.Real("i0")`, then compute the neurons and outputs, etc.).
    3. Finally, convert what you have into a general program that would work for any network input.
2. You can use the method below to construct a Z3 formula for ReLU:

    ```
    @classmethod
    def relu(cls, v):
        return z3.If(0.0 >= v, 0.0, v)
    ```

3. Two ways of doing symbolic execution to obtain symbolic states:
    1. You can follow the traditional SE method which produces symbolic execution trees in which each condition representing ReLU forks into two paths. You can decide whether to do a depth-first or breadth-first search (they will have the same results). At the end, the symbolic states are a disjunction of the path conditions (i.e., `z3.And[path_conds]`).
    2. But since we are using Z3, a much simpler way is to encode all those forked paths directly as a formula. For example:

        ```
        if (x + y > 0):
            r = 3
        else:
            r = 4
        ```

        Using traditional SE, you would have 2 paths: path 1: `x+y > 0 && r = 3` and path 2: `x+y <= 0 && r == 4`, from which you take the disjunction to get `(x+y > 0 && r == 3) || (x+y <= 0 && r == 4)`. But for this assignment,

instead of forking into two paths, you can use Z3 to encode both branches using the `If` function, e.g., `If(x+y>0, r==3, r==4)` or `r==If(x+y>0,3,4)`. The results of these two methods are exactly the same; the latter requires less work. It is up to you.

4. When `bias` is non-zero, it will simply be added to the neuron computation, i.e., `neuron = relu(sum(value_i * weight_i) + bias)`. For example, if `bias` is `0.123` then for neuron `n0_0` we would obtain `n0_0 == If(i0 + -1*i1 + 0.123 <= 0, 0, i0 + -1*i1 + 0.123)`.

E.1.3 Part 2: Evaluation

In this part you will show that symbolic execution *does not* scale on large networks.

1. You will create a function `create_random` that takes as input a list of positive integers representing the shape of the network `[#inputs, #neurons, ..., #outputs]`, and creates a neural network of that size with *random* weights and biases (e.g., between -5 and 5).
2. You will run symbolic execution on this network as you did in Part 1, get its results, and time the solving process.
3. You will do this for various sizes of networks **until your computer cannot handle it anymore** (e.g., until it takes more than a minute or two).
4. You will **save these networks** because you will reuse them to measure and compare execution time using abstraction in Part 2.
5. Finally, you will plot the results: the x-axis will show the size of the networks (for simplicity, the product of all numbers in the input list) and the y-axis shows the time.

Feel free to be creative in this process (e.g., play around with different sizes/layers), and discuss your findings in the `README` file. Below gives some code example:

```
# Create a random network from a given
# number of inputs, hidden neurons, and outputs
l = [2, 3, 4, 3, 3]  # 2 inputs, 3 hidden layers (with 3, 4, 3 hidden neurons
                     # for respective hidden layer 1, 2, 3), and 3 outputs.
dnn = create_random_nn(l)

symbolic_state = my_symbolic_execution(dnn)

# time the execution
import time
st = time.time()
_ = z3.solve(symbolic_state)
print('time to solve: ', time.time() - st)
```

E.1.4 Conventions and Requirements

Your program must have the following:

Part 1

1. A function `my_symbolic_execution(dnn)` that:
   - takes as input the network, whose specifications are given as example above.

        - **Do NOT hard-code** the network in your program (e.g., do not hardcode the example given above in your code or hard-code the weight values). This function should work with any network input.
    - returns *symbolic states* of the network represented as a Z3 expression (a formula) as shown above.
2. In your resulting formula, you must name:
    - the nth input as i_n (e.g., the first input is i_0)
    - the nth output as o_n (e.g., the second output is o_1)
    - the jth neuron at the ith layer as $n_{\{i\_j\}}$ (note that the first layer is the 0th layer)
3. A testing function `test()` where you copy and paste pretty much the complete examples given above to demonstrate that your SE works for the given example. In summary, your `test` will include:
    - the specification of the 2×2×2×2 network in the Figure
    - run the symbolic execution on the network (as shown above, it should output the dimension of the network and runtime)
    - output the symbolic states results
    - apply Z3 to the symbolic states to obtain:
        - random inputs and associated outputs
        - simulate concrete execution
        - checking the 3 assertions as shown
4. Another testing function `test2()` where you create another network:
    - The network will have:
        - the same number of 2 inputs and 2 outputs, but 3 hidden layers where each layer has 4 neurons, i.e., a 2×4×4×4×2 network.
        - non-zero bias values.
    - Then on this network, do every single task you did in `test()` (run SE on it, output results, apply Z3, etc.). You will need to have 2 assertions that are true and 1 assertion that is false (just like above).
        - Initially you might randomly generate weights and bias values, but you will likely need to manually adjust them so that you can prove some correct assertions (randomly generated values probably will not give you a meaningful network with any asserted properties).

You can be as creative as you want, but your SE must not run for too long and must not take up too much space (e.g., do not generate over 50MB of files). Use the `README` to tell me exactly how to compile/run your program, e.g., `python3 se.py`.

Part 2

1. A function `create_random(sizes: int)` that:
    - takes as input the list `sizes` and returns a network of that size as described above.
2. A testing function `test3()` that runs `create_random` on various sizes of networks, runs `my_symbolic_execution` on them as you did with `test()` and `test2()` above, and times the execution.
3. A plot (in pdf, png, or jpeg) showing the scalability of `my_symbolic_execution` on various networks. You need to use **at least 25 networks of different sizes** to generate

the plot. Use incrementally sized networks (e.g., gradually increasing the number of neurons/layers) or a good random distribution of sizes. A plot with very few datapoints will not be sufficient to show the scalability trend.

E.1.5 What to Turn In

You must submit a **zip file** containing the following files. Use the following names:

1. The zip file must be named `pa1-[yourname].zip`.
2. One single main source file `se.py`. Indicate in the `README` file how I can run it.
   - Your file should include at least a `my_symbolic_execution` function and the test functions `test()`, `test2()`, and `test3()` as indicated above.
3. The completed `README.md` file. You *must* use the provided template and answer all questions in it.
4. Nothing else; do not include any binary files or other directories like `__pycache__`, `.venv`.

E.1.6 Grading Rubric (out of 30 points)

- 12 points — for complete SE implementation.
  - 4 points for `my_symbolic_execution`
  - 2 points for `test`
  - 2 points for `test2`
  - 4 points for `create_random` and `test3`
- 2 points — for code quality and submission format.
- 16 points — for completing the `README` file.
  - 4 points for Part 1
  - 10 points for Part 2 (plot and discussion)
  - 2 points for the running instructions and screenshots

E.2 PA2

In this PA you will implement neural network (NN) verification using (1) *interval abstraction* as described in Sec. 5.4 and (2) *zonotope abstraction* as described in Sec. 5.5. While symbolic execution (PA1 Sec. E.1) computes exact symbolic representations, abstract interpretation uses over-approximations to make verification more scalable at the cost of some precision. You will implement interval and zonotope abstract transformers and compare their precision-scalability trade-offs with symbolic execution.

You will use Python and PyTorch for this assignment. **Do not** use external verification libraries (other than standard numerical libraries like `torch`, `numpy`).

This PA has three main parts:

1. implement interval abstraction for linear layers and ReLU activations.
2. implement zonotope abstraction for linear layers and ReLU activations.
3. evaluate the precision/scalability compared to symbolic execution from PA1.

A README template is provided. Please answer the questions in the template and include it in your submission. https://github.com/dynaroars/book_nnv/blob/main/pa/templates/README_PA2.md

E.2.1 Interval Abstraction

Interval abstraction is the simplest form of abstract interpretation for neural networks. Each variable is represented by an interval $[l, u]$ indicating its lower and upper bounds.

E.2.1.1 Part 1: Interval for Linear Layers

Linear transformations on intervals can be computed exactly using interval arithmetic.

```
class LinearTransformer:
    def __init__(self, W, b):
        # TODO: store weight matrix and bias vector

    def forward(self, input_lower, input_upper):
        """
        Apply affine transformation f(x) = Wx + b to intervals [l, u]

        Mathematical formulation:
        - W+: positive part of weights
        - W-: negative part of weights
        - output_lower = W+ @ input_lower + W- @ input_upper + bias
        - output_upper = W+ @ input_upper + W- @ input_lower + bias

        Args:
            input_lower: lower bounds of input intervals
            input_upper: upper bounds of input intervals

        Returns:
            (output_lower, output_upper): transformed interval bounds
        """
        # TODO:
        # 1. Split weights into positive and negative parts
        # 2. Compute output bounds using interval arithmetic
        # 3. Add bias to both bounds
        # 4. Return (output_lower, output_upper)

        return output_lower, output_upper
```

E.2.1.2 Part 2: Interval for ReLU Activations

ReLU transformation on intervals is straightforward: clamp lower bounds to 0.

```
class ReluTransformer:

    def forward(self, input_lower, input_upper):
        """
        Apply ReLU transformation to intervals: ReLU(x) = max(0, x)
        Args:
            input_lower: lower bounds of input intervals
            input_upper: upper bounds of input intervals
```

```
        Returns:
            (output_lower, output_upper): intervals after ReLU
        """
        # TODO: For each dimension:
        # 1. Clamp lower bounds to be >= 0
        # 2. Clamp upper bounds to be >= 0
        # 3. Return transformed bounds

        return output_lower, output_upper
```

E.2.1.3 Part 3: Putting It All Together

```
class IntervalAbstraction:

    def __init__(self, DNN):
        # TODO: convert DNN to corresponding interval abstraction transformers
        self.transformers = ...

    def forward(self, lb, ub):
        # propagate through layers
        for transformer in self.transformers:
            lb, ub = ...

        return (lb, ub)
```

E.2.1.4 Part 4: Test Your Implementation

In this DNN, the outputs of the neurons in the hidden layers (prefixed with `n`) are applied with the `relu` activation function, but the outputs of the DNN (prefixed with `o`) are not. These settings are controlled by the `True`, `False` parameters. Also, this example does not use `bias`, i.e., bias values are all 0.0's as shown.

```
# DNN specification
n00 = ([1.0, 1.0], 2.0, True)
n01 = ([1.0, -1.0], 2.0, True)
hidden_layer0 = [n00, n01]

n10 = ([1.0, 1.0], -0.5, True)
n11 = ([1.0, -1.0], -1.0, True)
hidden_layer1 = [n10, n11]

n20 = ([-1.0, 1.0], 7.0, False)
n21 = ([0.0, 1.0], 0.0, False)
hidden_layer2 = [n20, n21]

o0 = ([1.0, -1.0], 0.0, False)
output_layer = [o0]

dnn = [hidden_layer0, hidden_layer1, hidden_layer2, output_layer]
```

Note that all of these settings are parameterized and I deliberately use this example to show how these parameters are used (e.g., `relu` only applies to hidden neurons, but not outputs).

```
net = IntervalAbstraction(dnn)
input_lower = torch.tensor([-1, -1], dtype=torch.float)
input_upper = torch.tensor([1, 3], dtype=torch.float)
output_lower, output_upper = net(input_lower, input_upper)
print(f'{output_lower=}')
print(f'{output_upper=}')
# output_lower=tensor([-7.5000])
# output_upper=tensor([12.])
```

E.2.2 Zonotope Abstraction

E.2.2.1 Part 1: Zonotope Representation

From Bounds to Zonotope

A zonotope is represented by a tuple of a center vector and a generator matrix. Given the bounds of a zonotope, we can convert it to a zonotope.

```
def zonotope(lb, ub):
    """
    Computes zonotope from interval bounds [l, u].
    TODO:
    - compute center from bounds
    - generator from bounds
    """
    return center, generator
```

From Zonotope to Bounds

Given the center and generators of a zonotope, we can concretize the bounds of the zonotope.

```
def concrete(center, generator):
    """
    Computes interval bounds [l, u] from zonotope (center, generator).

    A zonotope Z = {c + sum_i eps_i * g_i | eps_i in [-1,1]} can be
    converted to interval bounds:
      Lower bound: l = c - sum_i |g_i|  (eps_i = -sign(g_i))
      Upper bound: u = c + sum_i |g_i|  (eps_i = +sign(g_i))

    Args:
        center: Center vector c of the zonotope
        generator: Generator matrix G

    Returns:
        (lb, ub): Interval bounds for each dimension
    """
    # TODO: compute interval bounds from zonotope
    # 1. compute lower bound
    # 2. compute upper bound
    return lb, ub
```

E.2.2.2 Part 2: Zonotope for Linear Layers

Linear (affine) transformations can be handled exactly in zonotope abstraction.

```
class LinearTransformer(nn.Module):
    def __init__(self, W, b):
        # TODO: store weights and bias

    def forward(self, center, generator):
        """
        Apply affine transformation f(x) = Wx + b to zonotope Z = (c, G)
        Result: f^a(Z) = (Wc + b, WG)

        Args:
            center: center vector c of input zonotope
            generator: generator matrix G of input zonotope

        Returns:
            (new_center, new_generator): transformed zonotope
        """
        # TODO:
        # 1. Transform center using weight matrix and bias
        # 2. Transform generator matrix using weight matrix
        # 3. Return transformed (center, generator)

        return new_center, new_generator
```

E.2.2.3 Part 3: Zonotope for ReLU Activations

ReLU activations are non-linear and require over-approximation in zonotope abstraction. For each neuron with bounds $[l, u]$, there are three cases to handle.

```
class ReluTransformer(nn.Module):

    def forward(self, center, generator):
        """
        Apply ReLU abstraction to zonotope: ReLU(x) = max(0, x)

        Three cases for each neuron i with bounds [l_i, u_i]:
        1. Active case (l_i >= 0): ReLU is identity, Z' = Z
        2. Inactive case (u_i <= 0): ReLU outputs 0, Z' = {0}
        3. Unstable case (l_i < 0 < u_i): requires over-approximation

        Args:
            center: center vector of input zonotope
            generator: generator matrix of input zonotope

        Returns:
            (new_center, new_generator): over-approximated zonotope after ReLU
        """
        # TODO:
        # 1. Compute current bounds using concrete(center, generator)
        # 2. Create new generator matrix with extra row for ReLU error
        # 3. For each neuron, handle based on bounds [l, u]:
        #    - Active (l >= 0): keep unchanged
        #    - Inactive (u <= 0): set to zero
        #    - Unstable (l < 0 < u): apply slope approximation
        # 4. Return transformed (center, generator)

        return new_center, new_generator
```

E.2.2.4 Part 4: Putting It All Together

```
class ZonotopeAbstraction(nn.Module):

    def __init__(self, DNN):
        # TODO: convert DNN to corresponding zonotope abstraction transformers
        self.transformers = ...

    def forward(self, lb, ub):
        # 1. convert bounds to zonotope
        center, generator = ...
        # 2. propagate zonotope through layers
        for transformer in self.transformers:
            center, generator = ...

        # 3. convert zonotope to bounds
        lb, ub = ...
        return (lb, ub)
```

E.2.2.5 Part 5: Test Your Implementation

```
net = ZonotopeAbstraction(dnn)  # same DNN specification as interval
input_lower = torch.tensor([-1, -1], dtype=torch.float)
input_upper = torch.tensor([1, 3], dtype=torch.float)
output_lower, output_upper = net(input_lower, input_upper)
print(f'{output_lower=}')
print(f'{output_upper=}')
# output_lower=tensor([0.1667])
# output_upper=tensor([6.1667])
```

E.2.3 Evaluation

- Make sure you use the same set of DNNs for the evaluation of both abstraction methods and the symbolic execution for fair comparison.
- Same requirement as PA1: use at least 25 DNNs of varying sizes (number of layers, number of neurons per layer, etc.) in your evaluation.
- A good proxy of precision is the width of the output bounds (i.e., upper − lower), but you can be creative with other metrics as well.
- All text in your plots must be legible.
- If the DNNs you use are too small or too few to show the trend, you will receive a low score.
- Answer the questions in the README template.

E.2.4 What to Turn In

You must submit a **zip file** containing the following files. Use the following names:

1. The zip file must be named `pa2-[yourname/groupname].zip` (1 submission per group).
2. One single main source file `pa2.py`. Indicate in the `README` file how I can run it.

    - Your code should use the provided class/function signatures as indicated above. E.g., `LinearTransformer`, `ReluTransformer`, `IntervalAbstraction`, `ZonotopeAbstraction`, `zonotope`, `concrete`.
3. If you imported anything from your previous PAs, please include those files as well (e.g., `p1.py`). We will not run your code if they are missing and you will lose points.
4. A completed `README.md` file. You *must* use the provided template and answer all questions in it.
5. Nothing else; do not include any binary files or other directories like `__pycache__`, `.venv`.

E.2.5 Grading Rubric (out of 30 points)

- 12 points — for complete interval and zonotope abstraction implementation.
    - 2 points for `LinearTransformer` for Interval Abstraction
    - 2 points for `ReluTransformer` for Interval Abstraction
    - 2 points for zonotope representation functions (`zonotope` and `concrete`)
    - 3 points for `LinearTransformer` for Zonotope Abstraction
    - 3 points for `ReluTransformer` for Zonotope Abstraction
- 16 points — for completed README with evaluation and analysis.
- 2 points — for code quality and submission format.

E.3 PA3

In this PA you will implement the branch and bound (BaB) algorithm as described in Sec. 6.2 with abstract interpretation from PA2 Sec. E.2 (using either interval Sec. 5.4 or zonotope Sec. 5.5).

While symbolic execution (PA1 Sec. E.1) provides exact analysis but does not scale, and abstract interpretation (PA2 Sec. E.2) scales well but may lose precision, Branch and Bound provides a framework that can *refine* abstract domains when needed, achieving a balance between precision and scalability.

You will use Python and PyTorch for this assignment, building upon your implementations from PA2. **Do not** use external verification libraries.

This PA has three main parts:

1. implement the core BaB algorithm with a priority *queue*-based approach.
2. implement utility functions: `deduce`, `decide`, `bound`, and `prune`.
3. evaluate the precision and scalability of BaB compared to pure abstract interpretation and symbolic execution.

A README template is provided. Please answer the questions in the template and include it in your submission. https://github.com/dynaroars/book_nnv/blob/main/pa/templates/README_PA3.md

E.3.1 Part 1: Branch and Bound Algorithm Framework

BaB systematically explores the space by maintaining a queue of subproblems and iteratively refining them until verification is complete or a counterexample is found. You will implement four core utility functions that form the building blocks of the BaB algorithm: `deduce`, `decide`, `bound`, and `prune`.

E.3.1.1 Algorithm Skeleton

```
class BranchAndBound:
    def __init__(self, network, abstraction_type):
        """
        Initialize BaB verifier
        Args:
            network: DNN to verify (same format as PA2)
            abstraction_type: Interval or Zonotope from PA2
        """
        # TODO: Initialize your abstraction from PA2
        self.abstraction = ...
        self.network = network

    def verify(self, input_lower, input_upper, property_fn, timeout=60):
        """
        Main BaB verification algorithm following Algorithm 1 from the book

        Args:
            input_lower: lower bounds of input domain
            input_upper: upper bounds of input domain
            property_fn: function that takes (output_lower, output_upper)
                         and returns VERIFIED, FALSIFIED, or UNKNOWN
            timeout: maximum time in seconds

        Returns:
            result: VERIFIED, FALSIFIED, or TIMEOUT
            counterexample: input that violates property (if FALSIFIED)
        """
        start_time = time.time()

        # Initialize verification problems (Line 5 in Algorithm 1)
        problems = Queue()
        initial_problem = (input_lower, input_upper)
        problems.put(initial_problem)

        # Main loop
        while not problems.empty():
            if time.time() - start_time > timeout:
                return TIMEOUT, None

            # Select a subproblem
            lb, ub = ...  # Hint: use problems.get()

            # TODO: Use self.deduce() to determine if the problem is feasible
            if self.deduce(lb, ub, property_fn):
                # Check satisfiability using adversarial attack
                cex = ...  # Hint: use self.attack()

                if cex is not None:  # Found valid counterexample
```

```
                    return FALSIFIED, cex

                # Decide: pick an input dimension to split on
                subproblems = ...  # Hint: use self.decide()

                # Add new subproblems to queue
                for sub_lb, sub_ub in subproblems:
                    ...  # Hint: use problems.put()

        # All subproblems are verified
        return VERIFIED, None
```

E.3.1.2 Deduce Function

The `deduce` function checks if the current problem is feasible by combining the abstract interpretation (bound).

```
def deduce(self, input_lower, input_upper, property_fn):
    """
    Deduce: combine bound + prune to check if we can make progress

    Args:
        input_lower: lower bounds of input domain
        input_upper: upper bounds of input domain
        property_fn: property function to check

    Returns:
        bool: False if verified, True otherwise (need to branch)
    """
    # TODO:
    # 1. Compute abstract bounds using self.bound() (PA2)
    # 2. Check pruning using self.prune()
    # 3. Return True if prune_result == UNKNOWN (need to branch)
    # 4. Return False if prune_result == VERIFIED (can prune)
    # 5. This function should NOT handle FALSIFIED case directly

    return prune_result == UNKNOWN
```

E.3.1.3 Attack Function

The `attack` function finds a counterexample using an adversarial attack. You can use a simpler approach like random sampling, or a more advanced approach like PGD, FGSM, etc.

```
def attack(self, input_lower, input_upper, property_fn):
    """
    Find counterexample using adversarial attack

    Uses random sampling or adversarial attacks instead of LP solving

    Args:
        input_lower: lower bounds of input domain
        input_upper: upper bounds of input domain
        property_fn: property function to check
```

```
    Returns:
        counterexample: cex that violates property, or None if not found

    Example approaches:
        # Random sampling
        for _ in range(100):
            random_input = ...
            output = self.network(random_input)
            if violates_property(output):
                return random_input

        # Or use PGD attack, FGSM, etc.
    """
    # TODO: Implement adversarial attack to find counterexample
    # 1. Generate candidate inputs within [input_lower, input_upper]
    # 2. Run network on candidates
    # 3. Check if any output violates the property
    # 4. Return first violating input, or None if none found

    return counterexample
```

E.3.1.4 Branch (Decide) Function

The `decide` function splits a domain into smaller subdomains. For this PA, we use input splitting where we divide one dimension at a time.

```
def decide(self, input_lower, input_upper):
    """
    Split input domain into 2 subproblems by splitting one dimension

    Args:
        input_lower: lower bounds of input domain
        input_upper: upper bounds of input domain

    Returns:
        list of (sub_lower, sub_upper) tuples representing subproblems

    Example:
        # Input domain: [-1, 1] x [-2, 2]
        input_lower = torch.tensor([-1.0, -2.0])
        input_upper = torch.tensor([1.0, 2.0])

        # Splitting the 1st dimension at 0:
        # Subproblem 1: [-1, 0] x [-2, 2]
        # Subproblem 2:  [0, 1] x [-2, 2]

        subproblems = decide(input_lower, input_upper)
        # subproblems = [
        #     (torch.tensor([-1.0, -2.0]), torch.tensor([0.0, 2.0])),
        #     (torch.tensor([0.0, -2.0]), torch.tensor([1.0, 2.0]))
        # ]
    """
    # TODO:
    # 1. Find desired dimension to split on
    #       (e.g., first dimension, max width, random, etc.)
    # 2. Compute split point for that dimension
    #       (e.g., midpoint, random, etc.)
```

```
    # 3. Create two subproblems by splitting at the computed point
    # 4. Return list of subproblems

    return subproblems
```

E.3.1.5 Bound Function

The `bound` function computes abstract output bounds using your abstraction from PA2 (Sec. E.2).

```
def bound(self, input_lower, input_upper):
    """
    Compute output bounds using abstract interpretation

    Args:
        input_lower: lower bounds of input domain
        input_upper: upper bounds of input domain

    Returns:
        (output_lower, output_upper): abstract bounds on network outputs

    Example:
        input_lower = torch.tensor([-1.0, -2.0])
        input_upper = torch.tensor([1.0, 2.0])

        output_lower, output_upper = bound(input_lower, input_upper)
        # output_lower = tensor([-10.0])
        # output_upper = tensor([10.0])
    """
    # TODO: Use your abstraction from PA2 (interval or zonotope)
    # to compute output bounds

    return output_lower, output_upper
```

E.3.1.6 Prune Function

The `prune` function determines whether a subproblem can be eliminated based on the property being verified.

```
def prune(self, output_lower, output_upper, property_fn):
    """
    Determine if subproblem can be pruned based on abstract bounds

    Args:
        output_lower: lower bounds on network outputs
        output_upper: upper bounds on network outputs
        property_fn: function that checks if bounds satisfy property

    Returns:
        VERIFIED: property holds for all points in this subproblem
        FALSIFIED: property violated for all points in this subproblem
        UNKNOWN: cannot determine, need to branch further

    Example:
```

```
        def safety_property(out_lb, out_ub, threshold):
            if torch.all(out_ub < threshold):
                return VERIFIED
            elif torch.all(out_lb >= threshold):
                return FALSIFIED
            else:
                return UNKNOWN

        prop_1 = lambda out_lb, out_ub: safety_property(out_lb, out_ub, 5)

        # Example 1: Can be pruned as VERIFIED
        output_lower = torch.tensor([-10.0])
        output_upper = torch.tensor([3.0])  # All outputs < 5
        result = prune(output_lower, output_upper, prop_1)
        # result = VERIFIED -> prune this subproblem

        # Example 2: Cannot be pruned
        output_lower = torch.tensor([-2.0])
        output_upper = torch.tensor([8.0])  # Outputs span threshold
        result = prune(output_lower, output_upper, prop_1)
        # result = UNKNOWN -> need to branch further
    """
    # TODO:
    # 1. Check if property_fn gives definitive answer
    # 2. Return appropriate result based on property satisfaction

    return result
```

E.3.2 Part 2: Evaluation

- Make sure you use the same set of DNNs and properties for the evaluation of all methods for fair comparison.
- Use at least 30 diverse DNN and property pairs in your evaluation.
- All text in your plots must be legible.
- If the DNNs you use are too small or too few to show the trend, you will receive a low score.
- Answer the questions in the README template.

E.3.2.1 Test Your Implementation

This function is used to test whether your BaB implementation is correct. Make sure there is *at least one perturbed dimension*, e.g., `input_lower` should not equal `input_upper`, or otherwise the input space is just a single point and no need to use abstraction.

```
def test_bab():
    """Test BaB on a DNN"""
    dnn = ...

    # Test with interval abstraction
    bab = BranchAndBound(dnn, 'interval')

    # Input domain
    input_lower = ...
    input_upper = ...
```

```
# Property:
property_fn = ...

result, counterexample, stats = bab.verify(
    input_lower, input_upper, property_fn, timeout=30
)

print(f"Result: {result}")
```

E.3.2.2 Analysis

Create two additional test functions:

1. `bab_vs_z3`: Compare BaB with Z3 on a local robustness property.
2. `bab_vs_abstraction`: Compare BaB with pure abstraction from PA2.

Note that `property_fn` can be an arbitrary function; make up your own property function (e.g., $y_0 > 0$ or $y_0 > y_1$, etc.), just make sure to use the *same property function* for different methods so that the comparison is fair.

For each testcase, you should:

1. Create networks of varying sizes.
2. Use BaB with different configurations (e.g., abstraction types, branching strategies, etc.).
3. For each network, measure:
   - Symbolic execution time/result (PA1 Sec. E.1)
   - Pure abstraction time/result (PA2 Sec. E.2)
   - BaB verification time/result
   - Use appropriate timeouts (e.g., 30s)
4. Plot results showing precision vs. scalability trade-offs.
5. Analyze when BaB is most beneficial.

E.3.3 What to Turn In

You must submit a **zip file** containing the following files. Use the following names:

1. The zip file must be named `pa3-[yourname/groupname].zip` (1 submission per group).
2. One single main source file `pa3.py`. Indicate in the `README` file how I can run it.
   - Your code should use the provided class/function signatures as indicated above. E.g., `BranchAndBound`, `deduce`, `decide`, `bound`, `prune`.
3. If you imported anything from your previous PAs, please include those files as well (e.g., `p1.py`). We will not run your code if they are missing and you will lose points.
4. A completed `README.md` file. You *must* use the provided template and answer all questions in it.
5. Nothing else; do not include any binary files or other directories like `__pycache__`, `.venv`.

E.3.4 Grading Rubric (out of 30 points)

- 12 points — Core BaB implementation.

    - 3 points for correct `verify` algorithm implementation
    - 3 points for `branch` function with proper input splitting
    - 3 points for `bound` function integration with PA2 abstractions
    - 3 points for `prune` function with correct pruning logic
- 16 points — for completed README with evaluation and analysis.
- 2 points — for code quality and submission format.

E.4 PA4

In this PA, you will use your existing implementations from previous PAs to verify properties of a real benchmark using the standard format (ONNX for networks, and VNNLIB for properties).

E.4.1 Part 1: Handling ONNX Networks

In order to handle ONNX networks, the simplest way is to use the ONNX library to load the network and convert it to a PyTorch model. You can install the conversion library `onnx2pytorch` using pip:

```
pip install onnx2pytorch
```

Then you can use the library to load the network and convert to a PyTorch model as follows:

```
import onnx
import onnx2pytorch

# load the ONNX network
model = onnx.load("path/to/your/network.onnx")

# convert to Pytorch model
# experimental=True is optional to use batch processing
model = onnx2pytorch.convert(model, experimental=True)

# since ACAS XU benchmarks contain simple feed-forward networks,
# we can simply enumerate each layer of a network
for layer_name, layer in enumerate(layers):
    if isinstance(layer, nn.Linear):
        # TODO handle linear layer
        pass
    elif isinstance(layer, nn.ReLU):
        # TODO handle ReLU layer
        pass
    elif isinstance(layer, sub):
        # handle subtraction layer
        continue  # this layer is y = x - 0, which is identity, so skip it
    else:
        # TODO handle other layers
        pass
```

E.4.2 Part 2: Handling VNNLIB Properties

E.4.2.1 Parsing VNNLIB Properties

VNNLIB properties are designed to represent properties of neural networks in a formal language. It specifies the precondition and postcondition of the network. To extract the preconditions and postconditions from a VNNLIB property, you can follow the provided code at https://github.com/dynaroars/book_nnv/blob/main/code/pa4/test.py:

```
# input shape is (1, 5) for ACAS XU benchmarks
input_shape = (1, 5)

# parse the VNNLIB property
properties = parse_vnnlib("path/to/your/property.vnnlib", input_shape)
```

E.4.2.2 VNNLIB Disjunctive Properties

Note that one VNNLIB file can contain disjunctive properties. Let's take a look at an example VNNLIB property 7 of ACAS XU benchmarks:

```
; ACAS Xu property 7

(declare-const X_0 Real)
(declare-const X_1 Real)
(declare-const X_2 Real)
(declare-const X_3 Real)
(declare-const X_4 Real)

(declare-const Y_0 Real)
(declare-const Y_1 Real)
(declare-const Y_2 Real)
(declare-const Y_3 Real)
(declare-const Y_4 Real)

; Unscaled Input 0: (0, 60760)
(assert (<= X_0 0.679857769))
(assert (>= X_0 -0.328422877))

; Unscaled Input 1: (-3.141592, 3.141592)
(assert (<= X_1 0.499999896))
(assert (>= X_1 -0.499999896))

; Unscaled Input 2: (-3.141592, 3.141592)
(assert (<= X_2 0.499999896))
(assert (>= X_2 -0.499999896))

; Unscaled Input 3: (100, 1200)
(assert (<= X_3 0.5))
(assert (>= X_3 -0.5))

; Unscaled Input 4: (0, 1200)
(assert (<= X_4 0.5))
(assert (>= X_4 -0.5))

; unsafe if strong left is minimal or strong right is minimal
```

```
(assert (or
    (and (<= Y_3 Y_0) (<= Y_3 Y_1) (<= Y_3 Y_2))
    (and (<= Y_4 Y_0) (<= Y_4 Y_1) (<= Y_4 Y_2))
))
```

This file specifies the properties in the form of $X \wedge (P_0 \vee P_1)$, where X is the precondition on input variables (X_0, X_1, X_2, X_3, X_4) and P_0 and P_1 are the postconditions in *negation form* (do not negate them!) on output variables (Y_0, Y_1, Y_2, Y_3, Y_4).

Specifically,

$$\begin{aligned} X \equiv & (-0.328422877 \leq X_0 \leq 0.679857769) \\ & \wedge (-0.499999896 \leq X_1 \leq 0.499999896) \\ & \wedge (-0.499999896 \leq X_2 \leq 0.499999896) \\ & \wedge (-0.5 \leq X_3 \leq 0.5) \\ & \wedge (-0.5 \leq X_4 \leq 0.5) \end{aligned} \tag{198}$$

$$\begin{aligned} P_0 &\equiv (Y_3 \leq Y_0) \wedge (Y_3 \leq Y_1) \wedge (Y_3 \leq Y_2) \\ P_1 &\equiv (Y_4 \leq Y_0) \wedge (Y_4 \leq Y_1) \wedge (Y_4 \leq Y_2) \end{aligned} \tag{199}$$

To deal with disjunctive postconditions ($P_0 \vee P_1$), the code from NeuralSAT simply converts $X \wedge (P_0 \vee P_1)$ to a list of conjunctive sub-properties:

$$\underbrace{X \wedge P_0}_{\text{property 1}} \vee \underbrace{X \wedge P_1}_{\text{property 2}} \tag{200}$$

Now each sub-property has the same form as PA3. To verify a sub-property, you can use the code from PA3.

E.4.2.3 Output Constraints in VNNLIB

VNNLIB always represents the output constraints in the form of $cs \cdot Y \leq \text{rhs}$ where Y is the output vector.

For example, $Y_3 \leq Y_0$ is equivalent to $-Y_0 + Y_3 \leq 0$, which is represented as:

$$\underbrace{(-1 \;\; 0 \;\; 0 \;\; 1 \;\; 0)}_{cs} \cdot \underbrace{\begin{pmatrix} Y_0 \\ Y_1 \\ Y_2 \\ Y_3 \\ Y_4 \end{pmatrix}}_{Y} \leq \underbrace{(0)}_{\text{rhs}} \tag{201}$$

If you are using the VNNLIB extraction code from NeuralSAT, each sub-property is an object with the following attributes:

- `lower_bounds`: input lower bounds (e.g., lower bounds of all input variables)
- `upper_bounds`: input upper bounds (e.g., upper bounds of all input variables)
- `cs`: coefficient matrix for output constraints
- `rhs`: constant vector for output constraints

```
# number in pop(), e.g., pop(1), means number of sub-properties
# we want to verify simultaneously, not the index of the sub-property
sub_property = properties.pop(1)  # get one sub-property

print(sub_property.lower_bounds)
# tensor([[-0.3284, -0.5000, -0.5000, -0.5000, -0.5000]])

print(sub_property.upper_bounds)
# tensor([[0.6799, 0.5000, 0.5000, 0.5000, 0.5000]])

print(sub_property.cs)
# tensor([[[-1.,  0.,  0.,  1.,  0.],   # Y_3 <= Y_0
#          [ 0., -1.,  0.,  1.,  0.],   # Y_3 <= Y_1
#          [ 0.,  0., -1.,  1.,  0.]]])  # Y_3 <= Y_2

print(sub_property.rhs)
# tensor([[0., 0., 0.]])
```

E.4.2.4 Verifying VNNLIB Properties

Remember that we have a list of disjunctive sub-properties (e.g., property 1 and property 2 for property 7). To verify the original property is UNSAT, we need to verify all sub-properties are UNSAT. But to show the original property is SAT, disproving one sub-property is enough. The high-level idea:

```
while len(properties):  # make sure to verify all sub-properties
    sub_property = properties.pop(1)  # get one sub-property
    # disprove one sub-property, return counterexample immediately
    if verify(network, sub_property) == SAT:
        return SAT, cex

    # run out of time
    if verify(network, sub_property) == TIMEOUT:
        return TIMEOUT

    # prove one sub-property
    if verify(network, sub_property) == UNSAT:
        continue  # move to next sub-property

# all sub-properties are verified, the original property is UNSAT
return UNSAT
```

E.4.3 Part 3: Putting Everything Together

Now you have all the components to build the verifier. Follow the naming conventions below:

1. A single Python file `verify.py` that accepts three command line arguments and runs the verification. This provides a unified interface for testing your implementation.

   ```
   python verify.py [path/to/network.onnx] [path/to/property.vnnlib]
   [timeout_in_seconds]

   # E.g.
   python verify.py acasxu/Network1_1.onnx acasxu/Property7.vnnlib 116
   ```

Your output should be either `UNSAT`, `SAT`, or `TIMEOUT`. If the output is `SAT`, you should also print the counterexample found.
2. Your actual implementation should be in `p4.py`.

E.4.4 Part 4: Verifying ACAS XU Benchmarks

Download the ACAS XU benchmarks from https://github.com/dynaroars/neuralbench/tree/main/instances/acasxu. In this folder, you will find:

- Folder `onnx`: contains the ONNX networks.
- Folder `vnnlib`: contains the VNNLIB properties.
- File `instances.csv`: contains list of instances of the ACAS XU benchmarks, where each line is a tuple of (network path, property path, timeout in seconds).

You can use `acasxu_instances_w_ground_truth.csv` instead of `instances.csv` in the repo. It contains the same instances as `instances.csv`, but also includes the ground truth results for each instance. It is available from https://github.com/dynaroars/book_nnv/blob/main/code/pa4/acasxu_instances_w_ground_truth.csv.

Your task is to:

1. Enumerate the first 90 instances in `instances.csv` and verify the properties; the rest are optional.
2. Report the result for each instance: either UNSAT, SAT, or TIMEOUT.
3. Some timeouts are expected; we do not expect your verifier to verify all instances.
4. Check the results with the ground truth at https://github.com/dynaroars/book_nnv/blob/main/code/pa4/acasxu_instances_w_ground_truth.csv.

You should debug your implementation using a few instances first before running all 90, as the complete run may take a long time.

E.4.5 What to Turn In

You must submit a **zip file** containing the following files. Use the following names:

1. The zip file must be named `pa4-[yourname/groupname].zip` (1 submission per group).
2. If you imported anything from your previous PAs, please include those files as well (e.g., `p1.py`). We will not run your code if they are missing and you will lose points.
3. One `verify.py` file as described above.
4. A completed `README.md` file. You *must* use the provided template and answer all questions in it.
5. It is recommended to include any logs or results your verifier generated in a separate folder. You can name the folder as you like, e.g., `results/`.
    - This is not mandatory, but it will show that you have done the experiments and help us grade your PA.
6. If you used any additional scripts/files to help with Part 4, include those as well and add instructions to the `README.md` file.
7. Nothing else; do not include any binary files or other directories like `__pycache__`, `.venv`.

E.4.6 Grading Rubric (out of 30 points)

- 20 points — Verifier implementation.
  - 4 points — for parsing ONNX networks correctly
  - 4 points — for parsing VNNLIB properties correctly
  - 6 points — for implementing the verifier (`verify.py`)
  - 6 points — for running your verifier on ACAS XU benchmarks (first 90 instances)
- 10 points — for completed README with analysis.

Bibliography

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[2] ONNX Community, "ONNX: Open Neural Network Exchange." 2017.

[3] S. Demarchi *et al.*, "Supporting Standardization of Neural Networks Verification with VNNLIB and CoCoNet.," in *FoMLAS@ CAV*, 2023, pp. 47–58.

[4] A. Tacchella, L. Pulina, D. Guidotti, and S. Demarchi, "The international benchmarks standard for the Verification of Neural Networks." [Online]. Available: https://www.vnnlib.org/

[5] C. Brix, S. Bak, T. T. Johnson, and H. Wu, "The Fifth International Verification of Neural Networks Competition (VNN-COMP 2024): Summary and Results." 2024. doi: 10.48550/arXiv.2412.19985.

[6] C. Barrett, A. Stump, C. Tinelli, and others, "The smt-lib standard: Version 2.0," in *Proceedings of the 8th international workshop on satisfiability modulo theories (Edinburgh, England)*, 2010, p. 14.

[7] X. Huang, M. Kwiatkowska, S. Wang, and M. Wu, "Safety verification of deep neural networks," in *International conference on computer aided verification*, 2017, pp. 3–29. doi: 10.1007/978-3-319-63387-9_1.

[8] L. De Moura and N. Bjørner, "Z3: An efficient SMT solver," in *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, 2008, pp. 337–340. doi: 10.1007/978-3-540-78800-3_24.

[9] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual." [Online]. Available: https://www.gurobi.com/

[10] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer, "Reluplex: An efficient SMT solver for verifying deep neural networks," in *International Conference on Computer Aided Verification*, 2017, pp. 97–117. doi: 10.1007/978-3-319-63387-9_5.

[11] M. Sälzer and M. Lange, "Reachability in simple neural networks," *Fundamenta Informaticae*, vol. 189, 2023.

[12] R. Baldoni, E. Coppa, D. C. D'elia, C. Demetrescu, and I. Finocchi, "A survey of symbolic execution techniques," *ACM Computing Surveys (CSUR)*, vol. 51, no. 3, pp. 1–39, 2018.

[13] J. C. King, "Symbolic execution and program testing," *Communications of the ACM*, vol. 19, no. 7, pp. 385–394, 1976.

[14] E. Y. Nakagawa, M. Guessi, J. C. Maldonado, D. Feitosa, and F. Oquendo, "Consolidating a process for the design, representation, and evaluation of reference architectures," in *2014 IEEE/IFIP Conference on Software Architecture*, 2014, pp. 143–152.

[15] H. Duong, T. Nguyen, and M. B. Dwyer, "NeuralSAT: A High-Performance Verification Tool for Deep Neural Networks," in *International Conference on Computer Aided Verification*, 2025, p. to appear.

[16] M. Das, R. Ray, S. K. Mohalik, and A. Banerjee, "Fast Falsification of Neural Networks using Property Directed Testing," *arXiv preprint arXiv:2104.12418*, 2021.

[17] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," *arXiv preprint arXiv:1706.06083*, 2017.

[18] C. Brix, S. Bak, C. Liu, and T. T. Johnson, "The Fourth International Verification of Neural Networks Competition (VNN-COMP 2023): Summary and Results." 2023.

[19] H. Duong, T. Nguyen, and M. Dwyer, "Generating and Checking DNN Verification Proofs," in *neurips*, , Ed., 2025, p. to appear.

[20] H. Duong, D. Xu, T. Nguyen, and M. B. Dwyer, "Harnessing Neuron Stability to Improve DNN Verification," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, July 2024, doi: 10.1145/3643765.

[21] M. Davis, G. Logemann, and D. Loveland, "A machine program for theorem-proving," *Communications of the ACM*, vol. 5, no. 7, pp. 394–397, 1962, [Online]. Available: https://dl.acm.org/doi/10.1145/368273.368557

[22] J. A. Nelder and R. Mead, "A simplex method for function minimization," *The computer journal*, vol. 7, no. 4, pp. 308–313, 1965.

[23] G. Katz *et al.*, "The marabou framework for verification and analysis of deep neural networks," in *International Conference on Computer Aided Verification*, 2019, pp. 443–452. doi: 10.1007/978-3-030-25540-4_26.

[24] K. Kaulen *et al.*, "The 6th International Verification of Neural Networks Competition (VNN-COMP 2025): Summary and Results." [Online]. Available: https://arxiv.org/abs/2512.19007

[25] H. Duong, D. Shriver, T. Nguyen, and M. B. Dwyer, "Compositional neural network verification via assume-guarantee reasoning," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[26] Y. Yu, H. Qian, and Y.-Q. Hu, "Derivative-free optimization via classification," in *Thirtieth AAAI Conference on Artificial Intelligence*, 2016.

[27] S. Wang *et al.*, "Beta-CROWN: Efficient Bound Propagation with Per-neuron Split Constraints for Complete and Incomplete Neural Network Robustness Verification," *Advances in Neural Information Processing Systems*, vol. 34, pp. 29909–29921, 2021, doi: https://doi.org/10.48550/arXiv.2103.06624.

[28] S. Bak, "nnenum: Verification of ReLU Neural Networks with Optimized Abstraction Refinement," in *NASA Formal Methods Symposium*, 2021, pp. 19–36. doi: 10.1007/978-3-030-76384-8_2.

[29] H. Zhang *et al.*, "General cutting planes for bound-propagation-based neural network verification," *Proceedings of the 36th International Conference on Neural Information Processing Systems*, 2022, [Online]. Available: https://dl.acm.org/doi/10.5555/3600270.3600391

[30] K. Xu *et al.*, "Automatic perturbation analysis for scalable certified robustness and beyond," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1129–1141, 2020, [Online]. Available: https://dl.acm.org/doi/10.5555/3495724.3495820

[31] R. J. Bayardo Jr and R. Schrag, "Using CSP look-back techniques to solve real-world SAT instances," in *Aaai/iaai*, 1997, pp. 203–208.

[32] J. P. Marques-Silva and K. A. Sakallah, "GRASP: A search algorithm for propositional satisfiability," *IEEE Transactions on Computers*, vol. 48, no. 5, pp. 506–521, 1999.

[33] J. Marques Silva and K. Sakallah, "GRASP-A new search algorithm for satisfiability," in *Proceedings of International Conference on Computer Aided Design*, 1996, pp. 220–227. doi: 10.1109/ICCAD.1996.569607.

[34] R. Nieuwenhuis, A. Oliveras, and C. Tinelli, "Solving SAT and SAT modulo theories: From an abstract Davis–Putnam–Logemann–Loveland procedure to DPLL(T)," *Journal of the ACM (JACM)*, vol. 53, no. 6, pp. 937–977, 2006, doi: 10.1145/1217856.1217859.

[35] C. Barrett *et al.*, "Cvc4," in *International Conference on Computer Aided Verification*, 2011, pp. 171–177. [Online]. Available: https://dl.acm.org/doi/10.5555/2032305.2032319

[36] D. Kroening and O. Strichman, *Decision procedures*. Springer, 2008. [Online]. Available: https://dl.acm.org/doi/10.5555/1391237